Tamper Detection and Data Integrity for Self-Describing Scientific Data

A Research Plan for the S2-D2 Framework
Using ClawHDF5 as an Experimental Platform

M. Scot Breitenfeld
The HDF Group
brtnfld@hdfgroup.org

Draft — June 17, 2026

Abstract

Scientific datasets archived for 30–100 years must remain verifiable long after they leave the original creator's control, yet no existing integrity primitive simultaneously supports sub-dataset partial verification, in-place incremental updates, verifiable subset extraction, public verifiability without a shared secret, post-quantum durability, and HPC-feasible build cost without a trusted-setup ceremony. This proposal presents a Merkle-tree provenance scheme for HDF5, implemented in ClawHDF5 [60], that satisfies all twelve derived requirements. Three research contributions are targeted: (i) a *verifiable subset extraction* protocol that lets a recipient of an arbitrary hyperslab cryptographically prove it is a true, complete subset of a signed parent dataset; (ii) a *post-quantum hybrid signing* scheme (Ed25519 + ML-DSA-65) benchmarked at HDF5 scale, addressing the harvest-now, forge-later threat over multi-decade archival horizons; and (iii) a *game-based security argument and adversarial red-team study* evaluating the construction against eight threat classes on real scientific datasets. The pure-Rust ClawHDF5 platform enables rapid iteration before design decisions are transferred to the C HDF5 library.

Contents

1 Introduction and Motivation

The S2-D2 project [70] aims to comprehensively secure self-describing data formats and libraries, with HDF5 [18] as the primary target for integration. The project is organized around four research thrusts: Task 1 develops a systematic threat model and security policy framework for self-describing data; Task 2 addresses plugin and library verification through digital signatures and trust chains; Task 3 secures the data itself via encryption and authentication mechanisms; and Task 4 constructs the S2-D2 runtime framework, including the Security Task Scheduler, to execute security operations with minimal performance overhead. This proposal contributes primarily to Task 3, with a secondary connection to Task 2 through shared trust infrastructure (the KeyStore and `h5sign` signing tool developed under Task 2's plugin-signature work, HDFGroup/hdf5#6198).

The problem. Scientific HDF5 datasets routinely reach tens of gigabytes to tens of terabytes—with individual simulation snapshots (e.g., IllustrisTNG cosmological runs) exceeding 75 TB—while institutional archives and multi-mission collections reach petabyte scale. These datasets are retained for decades and consumed by readers who access only small hyperslabs at a time. They live on shared parallel filesystems, in cloud object stores, on archival tape, and in cross-institutional transfers; at each stage they are exposed to silent bit-rot, accidental corruption, hostile modification by privileged adversaries (compromised storage nodes, malicious insiders, in-transit attackers), and—over the multi-decade archival horizon—future quantum adversaries who may forge classical signatures on data harvested today. The present work asks how such datasets can be *authenticated* so that any reader, in any of these settings, can verify that the bytes they read match the bytes the original signer committed to. The integrity of these datasets is increasingly critical not only for archival fidelity but for downstream *machine-learning pipelines*: when HDF5 files serve as training corpora, even small-scale data poisoning—corrupting as little as 0.1 % of training samples—has been shown to induce error-rate increases exceeding 80 % [77]. Cryptographic authentication enables consumers to detect and localize such corruption before it propagates into trained models or irreproducible simulation results.

The gap in HDF5 today. Current integrity mechanisms in HDF5 operate at either too coarse or too fine a granularity, and none provide cryptographic authentication of raw data. The file format's v2 metadata checksums (Jenkins lookup3) protect individual metadata pages but do not cover raw chunk data. The fletcher32 chunk filter detects random bit errors but is a non-cryptographic CRC easily defeated by an adversary. ClawHDF5 includes a basic provenance feature that computes a single SHA-256 hash per dataset, but this requires rehashing the entire dataset to verify integrity—infeasible for terabyte-scale data where a researcher

may access only a small hyperslab. Neither the C HDF5 library nor ClawHDF5 currently provides a mechanism for *partial verification* (verifying a subset of chunks without touching the rest), *incremental signing* (extending or updating a dataset without invalidating existing signatures), *tamper localization* (identifying which specific chunks were modified), or *post-quantum-secure provenance* appropriate to the 30–100-year archival horizon of scientific data.

Approach of this document. Tamper detection is a deep, mature area of cryptography and systems research, with primitives ranging from simple CRCs through cryptographic hashes, MACs, authenticated encryption, classical and post-quantum signatures, hash trees, authenticated data structures, accumulators, polynomial and vector commitments, transparency logs, trusted hardware attestation, and proofs of data possession or retrievability. Rather than presuppose a solution, the proposal first surveys this design space (§3) and then distills the requirements imposed by HDF5's chunked storage model, hyperslab access patterns, and archival timescales (§4). From that comparison we conclude that a hash-tree (Merkle-tree) construction signed at the root best satisfies the full set of derived requirements at terabyte scale— primarily because R8 (no trusted setup) and R9 (post-quantum upgrade path) rule out the strongest algebraic alternatives for multi-decade archives—and we devote the remainder of the document (§5 onward) to designing, implementing, and evaluating such a construction.

Experimental platform: why a Rust prototype rather than the C HDF5 library. This work uses ClawHDF5 [60]—a pure-Rust HDF5 implementation with zero C dependencies—as its experimental platform, in preference to the canonical C HDF5 library. Because the C library is the de facto reference implementation underpinning essentially the entire scientific-HDF5 ecosystem, and is itself the eventual production target for this work (§7, P2.6–P2.7), the choice to prototype elsewhere warrants explanation. We prototype in Rust for four reasons:

1. **Memory safety for security-critical code.** The components added here—cryptographic filters, signature handling, and a binary tree parser/serializer operating over untrusted, possibly adversarial input—are exactly the code patterns that account for the majority of memory-safety CVEs in C (buffer overruns, use-after-free, integer-overflow-driven heap corruption). Rust eliminates these classes at compile time, which is doubly valuable for a feature whose entire purpose is integrity and tamper detection: a memory-safety bug in the verifier would silently undermine the guarantee being added.
2. **Iteration velocity.** The research iterates rapidly on tree layouts, filter ordering, nonce derivation, and signing schemes. Rust's type system, integrated test harness (`cargo test`), and package manager make this iteration fast and safe relative to the C toolchain's manual memory management and hand-built test scaffolding.

3. **A mature, audited cryptographic ecosystem.** Pure-Rust implementations of precisely the primitives this work needs—BLAKE3, KangarooTwelve, Ed25519 (`ed25519-dalek`), and ML-DSA (`ml-dsa`)—are available as well-maintained crates, avoiding the FFI surface and build complexity of binding OpenSSL or liboqs from C.
4. **Risk isolation and clean integration points.** Prototyping in a separate codebase lets the research move quickly without destabilizing the production C HDF5 library the community depends on. ClawHDF5's modular crate architecture (separate crates for format parsing, I/O, filters, SIMD, and GPU compute) maps naturally onto the S2-D2 framework, and its existing filter pipeline (deflate, shuffle, fletcher32) is an extensible framework into which new security filters—encryption and Merkle hashing—insert without modifying the core I/O path or metadata cache.

Crucially, the contribution is the *design*—the on-disk format, the verification protocol, and the security argument—which is implementation-language-agnostic. The Rust prototype is the vehicle for building and measuring it quickly and safely; the path to production is the C HDF5 integration design document (P2.6) and community RFC (P2.7), which specify how the same format and API are added to the official C library. Rust is therefore the research substrate, not a rejection of C HDF5.

Scope clarification. ClawHDF5 does not currently include encryption or cryptographic signing capabilities. Its provenance feature stores per-dataset SHA-256 content hashes and creator metadata as HDF5 attributes, but this is limited to flat whole-dataset hashing. The Merkle-tree provenance and the encryption filters required for authenticated data protection will be implemented as new capabilities in this research. The work is also informed by the HDF5 SHINES project [72]—The HDF Group's NSF Safe-OSE effort (Award No. 2534078) to develop a safety, security, and privacy taxonomy for the HDF5 ecosystem—which provides the threat categorization and shared vocabulary within which these technical mechanisms operate.

2 Background

2.1 HDF5 Data Model and Chunked Storage

HDF5 organizes data into *groups* (hierarchical containers) and *datasets* (multi-dimensional arrays). Datasets may use *chunked storage*, where the array is divided into fixed-size tiles stored independently. Chunks have the option to pass through a *filter pipeline* that applies transformations (e.g., shuffle, compression, checksum) before writing to storage. Chunked storage is the dominant layout for large scientific datasets because it enables partial I/O, compression, and extensibility (resizable dimensions). While chunked storage is common, other dataset representations are

discussed in Section §10.

2.2 ClawHDF5 Architecture

ClawHDF5 is organized as the following Rust crates in a layered architecture:

- `clawhdf5-format`: `no_std`-compatible binary format parser/writer
- `clawhdf5-io`: I/O backends (buffered, mmap, async, VOL, sub-filing)
- `clawhdf5-filters`: Filter pipeline (deflate backends)
- `clawhdf5-accel`: SIMD acceleration (NEON, AVX2, AVX-512)
- `clawhdf5-gpu`: GPU compute via wgpu [79]
- `clawhdf5`: High-level ergonomic API

ClawHDF5 includes a basic provenance feature in `clawhdf5-format` that stores per-dataset SHA-256 content hashes, creator identity, timestamps, and optional source identifiers as HDF5 attributes. Integrity verification (`verify_dataset()`) rehashes the entire dataset and compares against the stored hash. Notably, ClawHDF5 does *not* currently include encryption, HMAC, or digital signature support in its filter pipeline—`clawhdf5-filters` provides deflate backends and `clawhdf5-format` implements shuffle and fletcher32. Cryptographic operations will therefore be implemented as new filters and crate modules as part of this research.

2.3 S2-D2 Framework Architecture

The S2-D2 runtime consists of three components:

1. **Data Protection Factory (DPF)**: A registry of encryption, masking, tokenization, and verification libraries with a verification layer for authorizing which libraries may be used.
2. **Resource Manager (RM)**: Detects available compute and storage resources (CPU, GPU, DPU) and estimates execution costs.
3. **Security Task Scheduler (STS)**: Dispatches security operations using asynchronous and proactive execution strategies to hide latency. (The STS prototype is being developed separately; this proposal focuses on the Merkle-tree provenance layer that defines *what* to verify.)

2.4 Asynchronous I/O for HDF5

Tang et al. [69] demonstrated transparent asynchronous I/O for HDF5 uses background threads, achieving significant latency hiding for metadata operations. This work is relevant to eventual STS integration of the provenance operations developed in this proposal, but the STS prototype itself is outside the scope of this document.

3 Survey of Tamper-Detection and Data-Integrity Methods

Tamper detection and data integrity are deep, mature research areas spanning cryptography, distributed systems, filesystems, archival storage, and blockchain. This section surveys the primitives and systems that could, in principle, be used to authenticate large scientific datasets such as those held in HDF5. We organize the discussion by primitive class and conclude each subsection with the suitability of the approach for multi-terabyte, chunked, long-lived scientific data. §4 then derives a formal set of requirements and selects a primitive on that basis.

The need for a cryptographic integrity primitive is driven both by operational security concerns and by the *FAIR data principles* [80] (Findable, Accessible, Interoperable, Reusable). The Reusable pillar requires that data are "associated with detailed provenance," and the Accessible pillar implies that a recipient of a partial dataset delivery must be able to verify authenticity without redownloading the whole. Current HDF5 practice satisfies neither demand at terabyte scale; this survey identifies which, if any, existing primitives can.

3.1 Error-detection codes (non-adversarial integrity)

Parity bits, cyclic redundancy checks (CRC-32, CRC-32C/Castagnoli used by iSCSI [65] and Btrfs, Adler-32 and CRC-32C used in Lustre RPCs, Fletcher-32), the Jenkins lookup3 hash used by HDF5 v2 metadata pages, and Reed-Solomon codes [61] detect random bit errors arising from hardware faults, transmission errors, or filesystem bugs [30]. High-performance environments increasingly adopt T10-PI (SCSI Protection Information, also known as T10-DIF) [25] for end-to-end, hardware-accelerated integrity checking from the compute node down to the physical drive. Lustre, in particular, combines RPC-level checksums with T10-PI on the backend Object Storage Targets (OSTs): checksums computed at the Lustre client are forwarded to T10-PI-capable SAS and NVMe drives on the OSTs, providing a continuous protection chain from the application through the network fabric to the physical storage media. The HDF5 fletcher32 chunk filter is in this family. However, all listed codes are invertible in closed form by an adversary. CRC-family codes are linear over GF(2) (XOR arithmetic); an attacker uses elementary linear algebra over the binary field to compute a tail correction preserving the checksum. Adler-32 and Fletcher-32 are linear over a small modulus (65521 and $2^{16} - 1$ respectively), which similarly admits a closed-form correction—no brute-force search is required in either case. They therefore detect *accidental* corruption only, not *adversarial* tampering, and cannot serve as the primary integrity primitive for an authentication scheme.

3.2 Cryptographic hash functions

Cryptographic hashes (SHA-1; SHA-2 family including SHA-256 and SHA-512 [52]; SHA-3 / Keccak [53]; BLAKE2 / BLAKE3 [56]; KangarooTwelve [11]) provide preimage, second-preimage, and collision resistance suitable for adversarial integrity. SHA-256 is hardware-accelerated on modern x86 (SHA-NI) and ARMv8.4-A CPUs at ∼1.5–3 GB/s per core; BLAKE3 exploits SIMD parallelism for memory-bandwidth-class throughput in software; KangarooTwelve uses Keccak-p with a parallel sponge for similar performance. SHA-1 is broken against collision attacks [67] and must not be used for new schemes. A single hash of a multi-terabyte dataset establishes authenticity of the whole, but verification requires re-reading every byte—no sub-dataset partial verification is possible from a flat hash. ClawHDF5's existing flat-SHA-256 provenance feature is in this category.

3.3 Symmetric-key authentication: MACs, AEAD, and homomorphic hashing

Message authentication codes (HMAC [32], KMAC, AES-CMAC, GMAC, Poly1305) and authenticated encryption schemes (AES-GCM [51], ChaCha20-Poly1305 [50], AES-OCB, AEGIS) bind authenticity to a shared symmetric key. Tags are compact (16 B, i.e., a 128-bit authentication tag appended to each message or chunk). HMAC-SHA-256 and Poly1305 run at multi-GB/s per core. Per-chunk MAC or AEAD provides chunk-level adversarial integrity at minimal cost. Homomorphic MACs (e.g., Boneh–Freeman schemes [13] for network coding) extend this family by allowing intermediate nodes to aggregate keyed authenticators; verification still requires the secret key. Homomorphic hashing (e.g., Meta's LtHash [42]) is conceptually adjacent but distinct: LtHash is an *unkeyed* hash function whose outputs lie in a group with a homomorphic addition law, relying on the hardness of lattice-style problems rather than a shared secret. Crucially, LtHash *does not require a secret key to verify*—any party can recompute and check the hash—which sets it apart from MACs in the verification model. Both constructions allow intermediate nodes to incrementally aggregate authenticators when combining data packets or propagating chunk updates, avoiding the need to re-authenticate from scratch—a property relevant to streaming and in-transit integrity.

The fundamental limitation of all symmetric authenticators, however, is that verification requires possession of the secret key: a third-party auditor, a public archive mirror, or a downstream researcher receiving a derived hyperslab cannot verify without it. This directly conflicts with the FAIR Reusable pillar [80], which requires that provenance be verifiable by any party, not only the original data producer. Symmetric authentication is therefore appropriate as a *component* (e.g., AEAD ciphertext authenticity) but cannot serve as the public provenance primitive on its own. Encrypt-then-MAC composition is the canonical secure pattern [6, 7].

3.4 Classical asymmetric signatures

RSA-PSS, RSA-PKCS#1, ECDSA over P-256 or secp256k1, and Ed25519/EdDSA [26] provides non-repudiable signatures with public verifiability: anyone with the signer's public key can verify, and only the signer can produce a valid signature. Signatures are 64–512 B per object. Signing one hash per dataset (or one hash per file) is well within the cost budget of any HDF5 workflow. *Signing every chunk* is a different proposition: at $N = 10^6$–10^7 chunks (1–10 TB at 1 MB chunk size), per-chunk Ed25519 signatures already consume 64 MB–640 MB, while ML-DSA-65 (3,309 B per signature) produces 3–33 GB of signature data and commensurate signing time. BLS aggregate signatures reduce storage at the cost of a pairing-based, non-post-quantum primitive. Asymmetric signatures are well-suited as the *root authenticator* of an aggregating structure, and as such are a building block of essentially every practical authentication scheme considered below; the design question is what they sign over.

3.5 Post-quantum signatures

A cryptographically relevant quantum computer (CRQC) breaks RSA, DSA, ECDSA, and EdDSA via Shor's algorithm. Scientific datasets routinely retain value for 30–100 years, so any signature placed today must be *harvest-now, forge-later* resistant. In 2024, NIST officially standardized its first post-quantum signature schemes for general use: ML-DSA (Module-Lattice DSA, FIPS 204, formerly Dilithium) [54] and SLH-DSA (Stateless Hash-based DSA, FIPS 205, formerly SPHINCS$^+$) [55]. Stateful hash-based schemes XMSS [24] and LMS [39] have smaller signatures but require careful state management, making them awkward for distributed scientific workflows. Hybrid classical + PQ signatures (Ed25519 + ML-DSA-65 or Ed25519 + SLH-DSA-128s) are the recommended transitional design. Post-quantum signature sizes (2–50 KB) are 50–800× larger than classical signatures; this matters when many signatures must be stored, but is benign for a single-root-per-file scheme.

3.6 Hash chains and append-only transparency logs

Lamport one-time hash chains underpin a family of forward-secure authenticators in which each new value is committed by hashing it together with the previous one. Certificate Transparency [34, 35] extends this to a Merkle log of TLS certificate issuances, providing tamper-evident audit. Sigstore Rekor [48] and Trillian [20] applies the same model to software-supply-chain artifacts. Append-only Merkle Mountain Ranges [73] amortize append cost better than balanced binary trees. Append-only logs are an excellent fit when the data model is itself an append-only stream (sensor logs, simulation checkpoints), but most HDF5 workloads also *overwrite* chunks (e.g., checkpoint/restart with correction of erroneous values). A pure append-only log

model, therefore, does not capture HDF5 semantics on its own; we revisit append-only mode as an optional layer in Phase 3.

3.7 Hash trees and authenticated data structures

Hash trees (Merkle trees [41]) hash data in a balanced binary tree, producing a single root hash that commits to all leaves. Variants include Sparse Merkle Trees (SMT) [17] for keyed data with efficient non-membership proofs, Merkle–Patricia tries used by Ethereum state [81], Merkle Mountain Ranges [73] for append-friendly logs, and Verkle trees [33] (KZG-based) for shorter multiproofs at the cost of a trusted setup. Tamassia [68] formalized the broader class of *authenticated data structures* (skip lists, B-trees, persistent dictionaries) in which an untrusted responder can prove query correctness against a digest published by a trusted source. Practical deployments include ZFS block-level Merkle-style checksumming [14], Git's content-addressed Merkle DAG of blobs, trees, and commits [74], and IPFS [10]. Hash trees provide $O(\log N)$ subset proofs, build at hash speed (no public-key operations), require no trusted setup, and become post-quantum simply by choosing a PQ-secure hash. Their natural unit of integrity (the leaf) maps cleanly onto HDF5 chunks. These advantages are not free, however: a hash tree requires auxiliary storage for its $\sim N$ internal nodes, imposes $O(\log N)$ write amplification on every in-place leaf update (recomputing the path to the root), and needs a write-ordering protocol to keep the tree consistent with the data after a crash—overheads that flat hashing and symmetric MACs avoid, and that the algebraic alternatives below partially trade away for constant-size proofs. Their proofs are also $O(\log N)$ in size rather than the $O(1)$ of polynomial-commitment schemes. The major engineering questions are tree shape (balanced binary vs. SMT vs. Patricia), leaf encoding, and how to bind the tree to the underlying chunked container.

Overlay Merkle tree vs. in-format authenticated B-trees. HDF5 uses internal B-trees extensively to track chunk allocations within the file (the Version 1 B-tree in the HDF5 specification manages chunk-storage indices for each dataset). Tamassia's authenticated data structure framework applies directly to B-trees, raising the question of whether an authenticated B-tree built on HDF5's existing chunk index would subsume the need for an external Merkle overlay. Three factors rule this out for our setting. First, HDF5's internal B-trees are highly mutable, deeply coupled to the library's metadata-management code, and among the most complex and brittle sections of the C HDF5 library; augmenting their node structure to carry hash accumulators would require invasive changes to the lowest-level format layer and would break forward compatibility with existing readers. Second, the B-tree topology is determined by the HDF5 storage allocator, not by the logical chunk grid, so leaf positions do not align with chunk boundaries in a way that admits clean hyperslab-aligned proofs. Third, a companion-dataset overlay is format-agnostic: it can be constructed without modifying the library, is embarrassingly parallel across MPI ranks (each rank hashes its own chunk subset independently, and results are

reduced via $O(\log P)$ collective operations), and carries no format-compatibility risk for existing HDF5 files.

3.8 Cryptographic accumulators

RSA accumulators (Benaloh–de Mare [9]) and bilinear-pairing accumulators (Nguyen [49]) commit to a set with a constant-size digest and produce constant-size membership (and non-membership) witnesses. They appear attractive as a tree alternative. However, RSA accumulators require a trusted setup (a safe RSA modulus whose factorization is unknown), bilinear accumulators require a structured reference string typically generated by an MPC ceremony, neither variant is post-quantum, and accumulators are inherently *set-based*: they have no notion of leaf order, complicating coverage proofs over ordered hyperslabs. Witness updates after dynamic insertions and deletions are also expensive. For a scientific dataset where order is intrinsic and trusted setup is unacceptable, accumulators are not a competitive option.

3.9 Vector and polynomial commitments

KZG [28] commits to a polynomial with a constant-size group element and produces constant-size openings; it underlies many succinct proof systems. Verkle trees [33] use KZG as the inner-node primitive for a high-fanout tree with very compact multi-proofs. IPA / Bulletproofs [15] avoid trusted setup but produce $O(\log N)$ proofs and have higher commit cost. FRI-based commitments underpin STARKs [8] and are post-quantum and require no trusted setup; however, their prover overhead scales with circuit size and their proof sizes run 10–100 KB, making them uncompetitive with simple Merkle inclusion proofs for static data verification. All pairing-based schemes (KZG, Verkle, Groth16 [22]) are broken by Shor; all require an $O(N)$ multi-scalar multiplication to commit; none are competitive with hash trees for our setting at terabyte scale.

3.10 Zero-knowledge proofs and folding schemes

ZK-SNARKs (Groth16, PLONK, Marlin) and ZK-STARKs prove the correctness of arbitrary computation, including correctness of an integrity claim, with minimal proof size. Recent breakthroughs in ZK *folding schemes* (Nova, SuperNova, HyperNova [31]) have drastically reduced the overhead of Incrementally Verifiable Computation (IVC) by deferring expensive SNARK operations, enabling efficient step-by-step proofs of execution. Proofs of Data Possession [5] and Proofs of Retrievability [27] let a verifier audit remote storage without retrieving full data by sampling random blocks.

These primitives are most useful when (a) one wishes to hide intermediate values from the verifier, (b) one wishes to perform *spot-checks* rather than complete

verification, or (c) one is verifying continuous iterative computation paths. For static data subsetting—where researchers want full guarantees over the arbitrary hyperslabs they extract—ZK-SNARKs and folding schemes impose $O(N)$ proving overhead compared to the lightweight $O(k \log N)$ Merkle inclusion proofs discussed in §4. They remain candidates for orthogonal extensions (e.g., a sampling-based audit of an archive's overall integrity).

3.11 Trusted execution environments and remote attestation

Intel SGX [16], AMD SEV-SNP, ARM TrustZone, and AWS Nitro Enclaves provide hardware-isolated execution with remote attestation; TPM 2.0 [75] and Linux IMA/EVM [64] provide measured-boot integrity tied to platform-binding keys. TEE-anchored memory integrity trees are also used inside the enclave itself. These mechanisms are powerful within a controlled platform but relies on recurrent vulnerability disclosures, vendor-specific attestation services, and the integrity of a particular hardware instance—assumptions incompatible with multi-decade archival of scientific data that may be read on platforms not yet built.

3.12 Filesystem and HPC storage integrity

HPC parallel filesystems and distributed object stores have increasingly adopted block-level and end-to-end integrity mechanisms. ZFS [14] stores Fletcher4 or SHA-256 checksums for every block and arranges them in a Merkle-style hierarchy that enables silent-corruption detection and self-healing from redundant copies. Lustre employs RPC-level checksumming (Adler-32, CRC-32C) at the client and passes those checksums through to T10-PI-enabled SAS and NVMe drives on the Object Storage Targets (OSTs), providing hardware-accelerated end-to-end protection against silent data corruption from client memory to physical media. DAOS (Distributed Asynchronous Object Storage) [36], which powers exascale deployments such as Argonne National Laboratory's Aurora supercomputer, provides high-performance transactional I/O with native end-to-end data integrity and self-healing over NVMe. Btrfs uses CRC-32C per block. dm-integrity (LUKS) provides per-block AEAD; dm-verity [19] and fs-verity [12] bind kernel-enforced Merkle hashes to read-time policy; Linux IMA/EVM [64] extends measured-boot integrity to individual files.

Both ZFS and Btrfs use *copy-on-write* (COW) semantics: every mutation writes a new block rather than overwriting the original, producing an implicit sequence of immutable snapshots that enables point-in-time rollback and self-healing from redundant copies. COW is valuable for recoverability, but it does not by itself establish the *authenticity* of any given snapshot: an adversary with write access can produce a COW snapshot of tampered data.

These mechanisms are excellent within their respective trust boundaries, but the integrity guarantee evaporates the moment data leaves the filesystem (e.g., copied to S3, archived to tape, transferred between collaborating institutions, or downloaded

to a researcher's laptop). For a portable, archive-grade guarantee, the cryptographic authenticity must travel *with* the file format rather than reside exclusively in the storage substrate—precisely the FAIR Interoperability principle [80], which requires that data and its associated metadata remain usable across platforms and institutions without depending on any particular infrastructure.

3.13 Cloud object-store integrity primitives

Amazon S3 provides ETag (typically MD5, but multipart-derived) and additional checksum algorithms (CRC-32, CRC-32C, SHA-1, SHA-256) that the service computes and returns on PUT/GET [3]. S3 Object Lock, Glacier Vault Lock, and Azure Immutable Blob Storage provide WORM semantics. Server-side authenticators detect bit-rot and accidental corruption in transit, but do not establish provenance: the storage provider *is* the trust root. A malicious or compromised provider can replace both the object and its checksum. End-to-end client-side authentication is therefore needed in addition to (not instead of) server-side checksums.

3.14 Blockchain and content-addressed systems

Bitcoin [46] and Ethereum [81] provide globally consistent tamper-evident logs over decentralized infrastructure; both rely on Merkle trees as their internal data-integrity primitive. IPFS [10] and IPLD generalize content-addressed Merkle DAGs to a distributed filesystem. LOCKSS [37, 62] provides peer-to-peer digital preservation through redundant replicas with mutual integrity audits. These systems are powerful for distributed coordination, but for a closed-organization scientific archive their consensus overhead is unjustified, and the underlying integrity primitive (a Merkle DAG) can be inherited without inheriting the consensus layer.

3.15 Integrity in scientific and analytics data formats

HDF5's v2 metadata checksums (Jenkins lookup3) cover metadata pages but not raw chunk data [18]; the fletcher32 chunk filter detects non-adversarial corruption. NetCDF builds on HDF5; Ovsyannikova et al. [58] have explored external data-integrity pipelines for NetCDF without sub-file granularity. Zarr v3 [82] stores each chunk as an independent object and supports per-chunk checksum codecs (CRC-32C, SHA-256), but with no hierarchical aggregation that enables efficient subset coverage proofs. ASDF [21] stores per-block checksums without a tree. Apache Parquet [4] stores per-page CRC-32 checksums, also flat. FITS, GeoTIFF, and DICOM have weak or absent integrity primitives. None of the existing scientific or analytics formats provides Merkle-tree-based partial verification, incremental signing, or tamper localization at sub-dataset granularity, and none provides a post-quantum signing path. Emerging *virtualized* formats—Kerchunk [1] and its successor VirtualiZarr—map chunk coordinates to byte ranges inside legacy

HDF5/NetCDF files stored on S3-compatible object storage, enabling cloud-native access to archival data without format conversion; however, they inherit the integrity limitations of the underlying objects and add no cryptographic verification layer of their own. Modern array engines such as TileDB [59] provide high-performance chunked storage for multi-dimensional arrays in genomics and geospatial workloads and support at-rest AES-256-GCM encryption at the tile level, but rely on the underlying cloud store for object-level integrity and provide no hierarchical verifiable-subset capability—a verifier wishing to confirm the integrity of an arbitrary sub-array must re-read and re-authenticate the entire relevant tile set.

3.16 Provenance, lineage, and freshness

The W3C PROV data model [78] and systems such as ProvONE [44], OpenLineage [57], and PrIMe [43] provide general-purpose provenance tracking for scientific workflows. These systems track *derivation history* (which computations produced a dataset), which is complementary to but distinct from *content integrity* (whether the bytes have been modified since signing). RFC 3161 trusted timestamping [2] and transparency-log inclusion proofs [20, 35] provide *freshness* guarantees against rollback. These are layered on top of an underlying content-integrity primitive rather than replacements for it: the present work focuses on the latter, with provenance and freshness retained as integration points (§10).

3.17 Statistical and machine-learning-based anomaly detection

A complementary detection strategy employs statistical models or machine learning to identify anomalous patterns in dataset contents without relying on provenance metadata. Anomaly-based detectors applied to ML training corpora have achieved reported detection accuracy of 75–95 % for targeted data-poisoning attacks [77]. Quantum machine learning (QML) variants have also been proposed for anomaly detection in high-dimensional scientific arrays, exploiting quantum amplitude estimation to accelerate the inner-product computations underlying nearest-neighbor outlier scoring.

In forensic settings, NTFS journal artifacts—specifically the Update Sequence Number journal (`$UsnJrnl`) and the transactional log (`$LogFile`)—record every filesystem write at the volume level, enabling post-incident reconstruction of which objects were modified and by which process. These host-based forensic signals are complementary to file-format-level integrity: they provide post-hoc attribution evidence rather than preventing modification or enabling a recipient to verify integrity without trusting the host operating system.

A related design goal is *tamper tolerance*—keeping a system operational despite active attacks—as opposed to pure detection. Tamper-tolerant software (TTS) [76] employs obfuscation, self-checking guards, and graceful-degradation modes so that a system continues operating even when portions of its code or data have been mod-

ified. Periodic consistency checks compare pre-computed values against runtime state; on detecting a mismatch the system may crash, degrade, or alert. The fundamental limitation of all such approaches is that there are no *provably secure* software anti-tampering methods: a proof of tamper-resistance would require a formal model capturing all attack surfaces (impossible for physical systems), a reduction to an unsolved hard problem, and that reduction to hold even when the attacker controls the hardware [76].

These mechanisms are complementary to but cannot substitute for cryptographic authentication. An anomaly detector signals that data *appears* unusual but cannot prove *who signed it* or bind a cryptographic commitment to a specific byte sequence. A TTS system tolerates runtime code tampering but does not authenticate stored data. Forensic journal analysis provides post-hoc attribution but cannot prevent unauthorized writes or offer public verifiability. None of these mechanisms provides sub-dataset partial verification, public verifiability without a shared secret, or post-quantum durability—the requirements that govern the selection in §4. They are therefore not candidates for the primary integrity primitive but remain useful complementary signals in a defence-in-depth deployment.

Summary. The survey identifies the key tradeoffs among mechanism classes for sub-dataset content authentication of long-lived scientific data. *Cryptographic hashes* are simple, fast, and widely deployed, but verification requires re-reading the entire dataset—no partial verification is possible from a flat digest. *Per-chunk MACs or AEAD* provide efficient chunk-level authentication and support incremental updates, but verification requires the shared secret key, precluding public third-party audits. *Per-chunk asymmetric signatures* provide public verifiability and tamper localization and are a natural fit when the number of signed objects is small, but storage and signing cost scale linearly with chunk count—becoming prohibitive at millions of chunks. *Hash trees with a signed root* combine subset proofs, no trusted setup, and post-quantum upgradeability at hash speed; they are the dominant primitive in content-addressed storage systems (Git, ZFS, IPFS) for precisely these reasons, but require companion storage for the tree nodes and careful write-ordering protocols. *Polynomial commitments (KZG, Verkle)* produce constant-size or sub-logarithmic multiproofs and are actively deployed in Ethereum and blockchain scaling systems; they require a trusted-setup ceremony and their pairing-based assumptions are not post-quantum. *Zero-knowledge folding schemes* excel at proving arbitrary computation over streaming data, but impose $O(N)$ proving overhead for static-data subsetting—a poor fit when all that is needed is a membership and coverage check. *Filesystem and HPC storage mechanisms* (ZFS, fs-verity, DAOS) offer strong end-to-end integrity within their trust domain, but the guarantee does not travel with the data when files are exported or transferred. *ML-based anomaly detection and tamper-tolerant software* provide useful defense-in-depth signals but offer probabilistic, not cryptographically binding, assurances

and no public verifiability. §4 derives a formal set of requirements that discriminate among these classes for the HDF5 multi-decade archival setting and selects a primitive on that basis.

4 Requirements and Selection

4.1 Threat Model

Before deriving the requirements, we define the adversary model that motivates the design. We consider eight threats spanning three SHINES taxonomy categories [72] (TCD, FMT, EXT) plus systems-security and privacy extensions (T6–T8):

T1: Storage-level tampering (TCD). An adversary with read/write access to the storage medium (e.g., a compromised storage node, a malicious administrator, or a man-in-the-middle during file transfer) modifies chunk data, metadata, or both. The adversary may attempt *targeted modification* (altering specific chunks while leaving the rest intact) or *wholesale replacement* (substituting the entire file). The adversary does *not* possess the signing private key.

T2: Silent data corruption (FMT). Hardware-level bit-flips, firmware bugs, or filesystem errors introduce undetected modifications to stored chunks. This is a non-adversarial integrity threat that existing HDF5 v2 metadata checksums partially address for metadata pages, but not for raw data.

T3: Provenance forgery (EXT). An adversary attempts to attribute a dataset to a different creator or institution, or to claim that a modified dataset is the original. This requires non-repudiable authorship via digital signatures.

T4: Rollback / stale data attack. An adversary with storage access replaces the current version of a file (data, Merkle tree, and signed root) with a valid, properly signed *older* version of the same file. The cryptography verifies perfectly, but the researcher operates on stale data (e.g., a simulation checkpoint rolled back to a pre-correction state). This threat requires temporal ordering beyond what a static Merkle root provides.

T5: Harvest-now, forge-later (post-quantum). An adversary archives signed roots and public keys today and, once a cryptographically relevant quantum computer (CRQC) exists, recovers the Ed25519 (or other elliptic-curve) private key via Shor's algorithm and forges signatures over tampered data attributed to the original signer. Because scientific datasets are routinely retained for 30–100 years, this threat is concrete for any archive signed with classical-only signatures.

T6: Security stripping and algorithm downgrade. An adversary deletes the provenance metadata entirely (removing the `_merkle_root` attribute and the `/_merkle` companion dataset) to force a fail-open downgrade to unauthenticated access, or alters the unencrypted algorithm identifiers in the root

attribute to force the verifier to use a weaker cryptographic primitive (e.g., downgrading the hash from `blake3` to `sha256-truncated-128`, or downgrading the AEAD from `AES-256-GCM` to `AES-128-GCM`). Unlike T1–T3, no signature forgery is required; the attack exploits verifier leniency toward missing or downgraded metadata.

T7: Verification resource exhaustion (DoS). An adversary crafts a malformed companion dataset, manipulated chunk-grid metadata, or cyclic proof paths designed to force the verifier into unbounded computation or out-of-memory states during `verify_subset`. For example, a claimed Merkle path of depth 2^{63}, or a coverage certificate that maps a tiny hyperslab to astronomically many chunks, can exhaust memory on an HPC compute node before any cryptographic check occurs.

T8: Structural metadata leakage (privacy). For encrypted sparse datasets, an adversary observes the companion dataset to map the physical allocation grid—the positions of null leaves directly reveal which chunks are unallocated. Even without decrypting a single byte, the shape of the sparse data can leak sensitive structure (e.g., geographic boundaries in radar data, or expressed-gene regions in a genomic sequence).

These eight threats collectively impose concrete demands on any candidate mechanism, and they directly motivate the requirements that follow. Addressing T1 and T2 requires *chunk-level tamper localization*: the mechanism must identify *which* chunks were modified, not merely whether some modification occurred—flat-file checksums cannot meet this bar. Addressing T1 (wholesale replacement) and T3 requires a non-forgeable *asymmetric commitment*: no party lacking the creator's private key may produce a valid provenance assertion over altered content, and authorship must be non-repudiable. Defending against T4 requires *versioned, ordered commitments* whose freshness a verifier can confirm either via a prior observation or via an external trusted timestamp. T5 demands that the signing component remain hard against a large-scale quantum adversary, necessitating a *post-quantum secure signing scheme* alongside any classical component. T6 requires that cryptographic algorithm identifiers be *bound inside the signed payload*—not stored in unsigned metadata—and that the runtime enforce fail-closed behavior when provenance metadata is absent or stripped. T7 requires the verification protocol to validate all structural parameters against committed chunk-grid bounds *before* allocating memory or performing cryptographic work, so that adversarially crafted metadata cannot exhaust verifier resources. T8 requires an *optional structure-hiding mode* for sparse encrypted datasets whose sparsity pattern is itself sensitive. These properties are formalized as R1–R12 in §4.2 and drive the candidate comparison in §4.3.

4.2 Requirements

We translate the problem statement of §1 into twelve concrete requirements that any candidate primitive must meet for the HDF5 setting. Several requirements

correspond directly to FAIR data principles [80]: R1 and R5b realize the *Accessible* pillar (verifiable partial delivery without a full re-download); R6 realizes *Reusable* (any downstream collaborator can verify provenance using only a public key); and R8–R9 ensure the provenance guarantee is durable across the multi-decade archival horizon that FAIR envisions for scientific data.

R1: Sub-dataset partial verification. A reader of an arbitrary hyperslab consisting of k chunks out of N must be able to verify those k chunks at cost $O(k \log N)$ or better, without reading or hashing the remaining $N - k$ chunks. (Rehashing a multi-terabyte dataset to verify a small hyperslab is computationally prohibitive in practice.)

R2: Incremental extension. Appending new chunks to a resizable dataset must not require re-signing or re-hashing existing chunks; the cost must be $O(\log N)$ or better per appended chunk.

R3: In-place chunk update. HDF5 permits any chunk in a dataset to be overwritten (e.g., during checkpoint/restart, or correction of erroneous values). Updating chunk k must cost $O(\log N)$ in *tree update* work (recomputing the leaf hash and the $\lceil \log_2 N \rceil$ ancestor nodes on the path to the root) and $O(\log N)$ in signing-plane work, not $O(N)$. The physical I/O cost is governed separately by the filesystem or object store and may involve copy-on-write semantics; this requirement addresses only the cryptographic maintenance overhead.

R4: Tamper localization. On a verification failure, the mechanism must identify which specific chunks (or a tight superset) were modified, in $O(\log N)$ work, rather than only signaling that *some* corruption occurred.

R5a: Per-chunk authenticity. Each delivered chunk in a requested hyperslab must be cryptographically tied to the original signer's commitment, so that an adversary cannot substitute a chunk from outside the signed dataset.

R5b: Coverage and completeness proof. Beyond per-chunk authenticity, the recipient must be able to verify that the delivered chunks are *exactly* the chunks of the requested hyperslab—none silently dropped, none substituted from a different region of the same signed dataset—against a single signer commitment, without access to the rest of the dataset. Per-chunk signatures or MACs alone satisfy R5a but not R5b: an adversary can deliver a valid signed chunk in place of the requested one, and the recipient has no sublinear-bandwidth way to detect the substitution without an extra structure that, in practice, reduces to a hash tree.

R6: Public verifiability. Verification must require only the original signer's public key, not a shared secret, so that third-party auditors, archive mirrors, and downstream collaborators can verify independently.

R7: Compact subset proof. The auxiliary proof bundled with a hyperslab must be sublinear in N (target: $O(k \log N)$ or smaller), so that bandwidth is not dominated by the proof itself.

R8: No trusted setup. The construction must not require a trusted-setup ceremony or trapdoored parameters, since open scientific use cannot establish or audit such ceremonies. This requirement concerns the *cryptographic construction* itself: a Merkle tree satisfies R8 because no ceremony is needed to generate its parameters. R8 is distinct from the *operational* trust assumptions of §5.8—satisfying R8 does not mean the scheme is trust-free in deployment. Even with no trusted setup, a verifier must still obtain the signer's public key from a trusted source, the signing private key must remain secret, and the authorisation of the signer is outside the scheme's cryptographic scope.

R9: Post-quantum upgrade path. The integrity guarantee must remain valid against a future cryptographically relevant quantum adversary, either by being inherently PQ-secure or by admitting a clean upgrade (e.g., swapping the hash or signature algorithm) without re-signing existing data.

R10: HPC-feasible build cost. Initial commitment over a multi-terabyte dataset must be feasible at memory-bandwidth speeds on commodity HPC hardware, i.e., a small constant multiple of a single linear pass over the data.

R11: Distributed parallel construction. In HPC workflows, P compute nodes write disjoint chunk subsets to the same dataset concurrently via MPI-IO. Leaf-level hash computation must be *embarrassingly parallel*: each node independently hashes its own chunks, and the tree must be reducible via $O(\log P)$ rounds of MPI collective operations with no global lock on the commitment structure. Primitives that require a sequential multi-scalar multiplication (MSM) across all N chunks to commit or update—RSA accumulators, KZG polynomial commitments, ZK-SNARK circuits—do not satisfy this requirement, for the same fundamental reason as R10.

R12: Structure confidentiality for sparse encrypted datasets. For encrypted datasets, the integrity structure must not reveal the physical sparsity pattern (allocation grid) of the underlying data to an observer who cannot decrypt the payload chunks. Equivalently: an authenticated observer who knows which leaves are null vs. populated gains information about dataset geometry (e.g., geographic boundaries in sparse climate data, non-zero regions in genomic arrays) independent of the payload. The construction must either hide null-leaf positions or provide a mechanism—using a key distinct from the payload DEK—by which authorized auditors can distinguish null from populated regions without decrypting chunk content.

4.3 Comparison of candidate primitives

Table 1 maps the seven alternative classes plus Merkle tree (eight columns total) identified in the survey against these twelve requirements.

Requirement	Flat hash	Per-chk MAC	Per-chk sig.	RSA acc.	KZG / Verkle	ZKP / PoR	TEE / fs-verity	Merkle tree
R1 partial verification	×	✓	✓	✓	✓	sample[c]	✓	✓
R2 incremental extend	✓	✓	✓[d]	MSM[a]	MSM[a]	IVC[b]	✓	✓
R3 in-place update	×	✓	✓[d]	MSM[a]	MSM[a]	IVC[b]	✓	✓
R4 tamper localization	×	✓	✓	×	partial	×	✓	✓
R5a chunk authenticity	×	✓[e]	✓	✓	✓	✓	✓	✓
R5b coverage proof	×	×	manifest[f]	complex	✓	✓	×	✓[k]
R6 public verifiability	✓	×	✓	✓	✓	✓	vendor	✓
R7 subset-proof size	—	$16k$ B	$64k$ B[g]	$O(1)$	$O(1)$	$O(1)$	—	$32k \log N$ B
R8 no trusted setup	✓	✓	✓	×	×	varies	vendor	✓
R9 PQ upgrade path	✓	✓	†	×	×	STARK only	vendor	✓
R10 HPC-feasible build	✓	✓	3 h[h]	3 h[a]	30 s[a]	>24 h[b]	✓	30 s[i]
R11 distrib. parallel	✓	✓	✓	×	×	×	vendor	✓
R12 struct. confid.	×	×	×	×	×	×	×	opt-in[j]

Table 1: Candidate primitives against the twelve requirements of §4.2. Build-cost row (R10) reports order-of-magnitude build time at $N = 10^7$ chunks (10 TB at 1 MB chunks); see footnote [i] for the Merkle baseline breakdown. "vendor" = depends on platform vendor; "sample[c]" = sampling assurance only; "MSM" = $O(N)$ multi-scalar multiplication required per update; "IVC" = incrementally verifiable computation step. [a] RSA accumulator: each update is one $O(N)$ modular exponentiation, ∼30 min–3 h on commodity HW; KZG: one $O(N)$ multi-scalar multiplication over BLS12-381, ∼30 s–30 min on commodity HW (Pippenger, single core). [k] Merkle R5b: coverage proof requires the signed grid-hash + Merkle-path coverage certificate (§5.4), not raw inclusion proofs alone. [b] ZK folding step (e.g., Nova/HyperNova) takes ∼10 ms per step; 10^7 steps × 10 ms = 10^5 s ≈ 28 h (>24 h). [c] PoR/PDP provides sampling-based assurance, not complete verification. [d] Per-chunk signature update is $O(1)$ for one chunk but signing every chunk to commit the dataset is the dominant cost (R10). [e] Per-chunk MAC: authentic to the holder of the secret key only; fails R6. [f] Per-chunk signatures provide chunk authenticity but require an additional signed manifest (which is itself a Merkle root, in practice) to prove coverage and completeness against the parent dataset. [g] Per-chunk classical signatures: 64 B/chunk; ML-DSA-65 raises this to 3,309 B/chunk (FIPS 204 final). [h] ML-DSA-65 signing at ∼1 ms/chunk × 10^7 chunks ≈ 3 CPU-hours. [i] Merkle baseline: ∼30 s assumes a parallel HPC leaf-hashing pass across ∼200 cores of an HPC partition (∼200×1.5 GB/s ≈ 300 GB/s aggregate via a leadership-class parallel filesystem, e.g., DAOS on Aurora or Lustre on Frontier, for 10 TB); tree aggregation over pre-computed leaf hashes adds < 1 s on a single core. [j] Merkle R12: structure confidentiality is *opt-in* via PRF-masked null hashes ($H(\texttt{0x02} \,\|\, \mathrm{PRF}(SK, \texttt{"null"}))$, where SK is a Structure Key derived from the KEK); see §5.5. No other candidate provides even an opt-in path to structure hiding. † Per-chunk signatures upgrade to PQ by swapping the scheme but remain storage-prohibitive (33 GB/dataset at $N = 10^7$).

4.4 Selection: hash tree with a signed root

A hash tree (binary or sparse Merkle tree) over chunk hashes, with the root signed by a hybrid classical + PQ signature, satisfies all twelve requirements simultaneously. As discussed further below, other candidates satisfy subsets of R1–R12, but each has a structural gap under the combination of requirements imposed by the multi-decade archival setting—particularly R8 (no trusted setup) and R9 (post-quantum upgrade path), which are the discriminating requirements for scientific data signed today and read decades from now:

- Subset and coverage proofs of size $O(k \log N)$ (R5a, R5b, R7) come directly from the tree structure; subset extraction reduces to a Merkle multiproof, and coverage to a sibling-witness completeness check.
- Tamper localization (R4) walks from root to leaves in $O(\log N)$.
- In-place update (R3) and append (R2) recompute only the $O(\log N)$ hashes on the affected root path—$\sim$24 SHA-256 invocations ($\lceil \log_2 10^7 \rceil = 24$; $\sim 3\,\mu$s on SHA-NI) per update at $N = 10^7$.
- No trusted setup (R8); the construction is post-quantum simply by choosing a PQ-secure hash (R9) and binding the root with a PQ signature.
- Build cost (R10) is one hash per chunk plus $N - 1$ pair hashes for internal nodes. At $N = 10^7$ chunks (10 TB), leaf hashing completes in $\sim$30 s across $\sim$200 cores of an HPC partition (1.5 GB/s per core, $\sim$300 GB/s aggregate via a parallel filesystem); tree aggregation over pre-computed leaf hashes adds < 1 s on a single core—two or more orders of magnitude below the algebraic alternatives.
- Public verifiability (R6) is provided by the asymmetric root signature.

Comparison with the strongest alternatives. The strongest competitor on proof compactness is **Verkle**: a high-fanout KZG-based tree producing constant-size multiproofs (strictly better than Merkle on R7 alone), with chunk-aligned leaves satisfying R1, R3, R4, R5a, and R5b. For near-term blockchain and cloud-storage settings where a trusted ceremony is acceptable and post-quantum security is not yet required, Verkle is a compelling choice and is actively deployed in Ethereum. In the multi-decade scientific archival setting, however, Verkle does not satisfy R8 or R9: every variant requires a trusted-setup ceremony for the KZG structured reference string, and its elliptic-curve pairing is broken by Shor's algorithm. For archives signed today and read in 2060, this is an architectural gap rather than an engineering one—there is no migration path short of re-signing all existing data once a cryptographically relevant quantum computer exists.

The most compact signature alternative is **BLS-aggregated per-chunk signatures**, which collapse N per-chunk signatures into a single 48-byte aggregate and satisfy R1–R6 and R10. BLS aggregate signatures do not require a trusted setup (unlike KZG), which is a real advantage over Verkle. However, BLS12-381 pairings are not post-quantum (R9 is not met), and coverage proofs (R5b) require an addi-

tional signed manifest of delivered chunks—a structure that in practice reduces to a hash tree, yielding no storage or compute advantage over a Merkle root directly.

A second alternative that avoids pairing-based assumptions is **per-chunk PQ signatures** (e.g., ML-DSA-65 per chunk) with no aggregation. This satisfies R6, R8, and R9 and provides strong per-chunk non-repudiation. The cost is size and compute: at $N = 10^7$ chunks, ML-DSA-65 produces 33 GB of signature data and signing takes approximately three CPU-hours per dataset. This tradeoff may be acceptable for datasets with few chunks or for settings where per-object non-repudiation is architecturally required; at terabyte scale it is generally impractical.

The remaining candidates have more direct mismatches with the stated requirements. **Per-chunk MACs** and AEAD provide efficient chunk-level authentication but require a shared secret key, excluding third-party auditors who lack it (R6). **Flat hashing** is simple and fast but requires re-reading the entire dataset for any verification, with no support for partial reads, in-place updates, or tamper localization (R1, R3, R4, R5b). **RSA accumulators** share Verkle's trusted-setup and post-quantum limitations and additionally lack ordered-set semantics needed for coverage proofs. **ZK-SNARKs and folding schemes (Nova, HyperNova)** are well-suited to proving computational properties of data, and are a natural fit for settings where a verifier needs only sampling-based assurance or where intermediate values must be hidden; for static-subset authentication where full coverage must be proven, their $O(N)$ proving overhead is disproportionate to the verification task. **PoR/PDP** provide probabilistic assurance via random block sampling, which is appropriate for remote storage audits but does not meet the complete verification and tamper-localization requirements (R1, R4). **TEE-anchored schemes** (SGX, fs-verity) are a strong choice within a controlled platform but bind verification to specific vendor hardware and attestation services—assumptions that are incompatible with cross-institutional archival across decades.

Honest framing of the conclusion. Merkle is not the *only* primitive in absolute terms that satisfies R1 through R7—Verkle is competitive there, and BLS-aggregated per-chunk signatures are competitive on R1–R6. What distinguishes Merkle is that it is the only primitive achieving R1–R12 *simultaneously, without a trusted-setup ceremony and without a pairing- or factoring-based primitive that Shor's algorithm breaks.* R8 and R9 are the discriminating requirements, and they are forced by the multi-decade archival horizon of scientific data—a constraint not present in the blockchain settings where KZG, Verkle, and BLS were designed.

Trade-offs accepted by the hash-tree choice. Selecting a Merkle tree is best understood as a constrained optimization rather than the identification of a categorically dominant primitive: it is the design that meets the non-negotiable archival constraints (R8, R9) and the HPC construction constraints (R10, R11) at the lowest cost along the dimensions where it is genuinely weaker than the alternatives surveyed

above. Three such costs are explicit, and each is addressed—not eliminated—by the design that follows:

1. **Write and read amplification.** Unlike a flat hash, updating a single chunk requires recomputing and rewriting the $O(\log N)$ ancestor nodes on the path to the root, and under concurrent HPC writers the upper-level nodes near the root become contention hotspots. We accept this cost and mitigate it with the map-reduce aggregation protocol (§5.5, §7), which batches leaf updates before recomputing shared paths; the residual amplification is characterized experimentally under RQ3.
2. **Auxiliary metadata and portability.** Storing the tree in a companion dataset decoupled from the chunk payloads introduces a crash-consistency window (data written, tree not yet updated) and a portability hazard (a naïve copy that omits the `/_merkle` group breaks verification)—risks that flat hashing and per-chunk MACs do not incur. We accept this and mitigate it with the strict write-ordering protocol (§5.5) and a lightweight signed root attribute that travels with the parent dataset, preserving root-level verification even if the companion is lost.
3. **Logarithmic rather than constant proof size.** Merkle multiproofs scale as $O(k \log N)$, strictly larger than the $O(1)$ proofs of KZG and Verkle; for highly fragmented hyperslabs the proof can become a non-trivial fraction of the payload. We accept this as the direct price of avoiding trusted setup and pairing-based assumptions, and reduce it in practice via space-filling-curve (Morton/Hilbert) leaf linearization that concentrates witnesses on the hyperslab boundary (§5.4, RQ6).

Framing the selection this way—accepting auxiliary-metadata overhead, write amplification, and logarithmic (rather than constant) proofs in exchange for the post-quantum durability and trusted-setup-free construction that the multi-decade archival horizon demands—makes the choice a deliberate, multi-variable engineering trade-off rather than a claim of unconditional superiority.

The remainder of this document therefore designs, implements, and evaluates a Merkle-tree integrity scheme tailored to HDF5's chunked storage and hyperslab access model. We retain the surveyed primitives as composable layers where they fit (AEAD inside the Encrypt-then-MAC filter pipeline, §5.6; classical + PQ hybrid signatures at the root, §5.8; transparency-log publication of signed roots as a freshness layer, §5.8; W3C PROV linkage as a complementary provenance graph, §10).

Threat-model coverage of the selected design. With the selection established, we close the loop by mapping the hash-tree construction back to the threat model of §4.1. The Merkle tree detects and localizes T1 (targeted modification) and T2 at chunk granularity via partial verification—a hash-path mismatch at any leaf exposes the tampered chunk while leaving the rest of the proof intact. The hybrid

signed root defeats T1 (wholesale replacement) and T3, since the adversary cannot produce a valid signature without the private key; optional Sigstore publication provides a public, append-only audit trail for T3. To defend against T4, the signed root attribute includes a **monotonic dataset version counter** incremented on every write, plus a high-resolution timestamp; a verifier rejects any version lower than a previously observed value. Because this defense requires a prior observation, a first-time recipient cannot detect rollback from version information alone—this is why the external TSA timestamp (§5.8) is architecturally significant. The companion dataset holding the full Merkle tree is itself integrity-protected via a hash of its serialized contents included in the signed root attribute (§5.5). The hybrid PQ signature (ML-DSA-65 or SLH-DSA-128s alongside Ed25519) addresses T5 (§5.8). Algorithm identifiers bound into the signed payload R, combined with the fail-closed Strict Mode runtime (§5.5), address T6. Strict input-bound enforcement in the verification protocol (§5.9) addresses T7. The optional DEK-derived or KEK-derived PRF null hash for sparse encrypted datasets (the PRF-masked padded binary tree construction of §5.5) addresses T8.

5 Merkle-Tree Provenance

5.1 Why Integrity Verification Independent of Encryption?

A natural question is whether Merkle-tree integrity verification is necessary when encryption can provide per-chunk authentication (e.g., via AEAD ciphers like ChaCha20-Poly1305). The answer is that encryption and Merkle-tree integrity serve *different* security properties, and the most common scientific data use case requires integrity *without* encryption:

Most scientific data is unencrypted. NOAA climate records, EQSIM earthquake simulations, astronomical survey data, and genomic reference sequences are public or community-shared— canonical FAIR datasets [80] that must be Accessible and Reusable by any downstream researcher. These datasets have no confidentiality requirement, but they critically need tamper detection—a corrupted simulation checkpoint or a silently modified climate record can invalidate months of downstream analysis. The Merkle tree provides chunk-level integrity using only hashing (SHA-256 with SHA-NI achieves ∼1.5–3 GB/s per core; aggregate throughput scales linearly with core count), with no key management, no nonce derivation, and no encryption overhead.

Whole-dataset verification without decryption. Even for encrypted datasets, AEAD per-chunk authentication answers only "has *this* chunk been tampered?" after decrypting it. To answer "has *any* chunk in my terabyte dataset been modified?" without decrypting all chunks, one checks the signed Merkle root—a single 32-byte comparison. This enables zero-trust data custody: storage nodes, archival facilities, and collaborators without the decryption key can

verify dataset integrity via the Encrypt-then-MAC design (§5.6).

Tamper localization without decryption. If the root check fails, the Merkle tree localizes the corruption to specific chunks in $O(\log N)$ without reading or decrypting unaffected data. AEAD encryption alone requires decrypting every suspect chunk to find the bad one.

Provenance and non-repudiation. Encryption proves nothing about who created the data. The Ed25519-signed Merkle root provides non-repudiable authorship—critical for regulatory compliance, reproducibility audits, and data citation.

In summary, the Merkle tree is the *primary* contribution of this proposal and is valuable even—especially—when encryption is not used. The encryption filters (AES-256-CTR, ChaCha20-Poly1305) are complementary components for datasets that *do* require confidentiality; the EtM pipeline ordering ensures the Merkle tree's integrity properties are preserved in that case.

5.2 Approach

We propose constructing a Merkle hash tree [41] over HDF5 chunked datasets where:

- **Leaf nodes** are content hashes of individual chunks, computed *after* all filter pipeline operations (compression, shuffle, and encryption) so that the hash covers the exact ciphertext bytes stored on disk (see §5.6 for rationale).
- **Internal nodes** aggregate child hashes pairwise up to a single root hash per dataset.
- **File-level root** aggregates all dataset roots plus file metadata (superblock hash, group structure) into a single integrity value.
- The root may be **signed with an asymmetric key** (Ed25519 or via Sigstore [48]) to establish non-repudiable authorship.

This structure enables three operations that flat hashing cannot:

Partial verification. Verifying chunk k requires only the $O(\log N)$ sibling hashes along the path from leaf k to the root, not all N chunk hashes.

Incremental extension. Appending chunks to a resizable dataset adds new leaves and recomputes only the affected root path.

In-place update. Overwriting an existing chunk k (which HDF5 permits for any chunk in a dataset) requires recomputing only the $O(\log N)$ hashes along the path from leaf k to the root—the same cost as appending a new chunk. This is important because HDF5 workloads frequently update data in place (e.g., checkpoint/restart, correction of erroneous values). Note that in-place updates interact critically with the encryption layer: see §5.7 for the required nonce derivation scheme.

Tamper localization. Walking the tree from root to leaves identifies exactly which subtree—and ultimately which chunks—have been modified.

File-level root and metadata protection. For files containing multiple datasets, the file-level root aggregates all per-dataset roots. When a dataset is added, removed, or extended, only the affected dataset root and the file-level aggregation are recomputed—not the trees of unmodified datasets. For files with hundreds of datasets, the file-level root is computed lazily (on explicit request or at file close) rather than eagerly on every dataset modification, avoiding quadratic overhead during batch creation workflows.

Crucially, the file-level root must also cover **HDF5 metadata**, not just chunk data. An attacker who leaves array data intact but modifies an attribute (e.g., changing `units="Celsius"` to `units="Fahrenheit"`) or alters the dataspace dimensionality corrupts the scientific result while passing chunk-level Merkle verification. To prevent this, each dataset's contribution to the file-level root includes a hash of its object header metadata. Critically, this hash is computed over a **canonical serialization** of the semantic fields (datatype class and size, dataspace dimensionality and extent, chunk shape, and filter pipeline identifiers) rather than over the raw HDF5 disk bytes of the object header block. HDF5 object headers are malleable: the library routinely shifts messages, inserts padding, or migrates the header to a continuation block during ordinary operations such as `h5repack` or attribute addition. Hashing raw bytes would break the signature after any such benign operation. By extracting the specific fields into a fixed-width, big-endian binary encoding and hashing that, the signature survives all layout-preserving file operations while still detecting any semantically meaningful metadata change (e.g., rewriting `units="Celsius"` to `units="Fahrenheit"`). The canonical format is defined in the Phase 2 design document (§7). This is analogous to how Zarr v3 secures metadata via its consolidated `zarr.json` manifest [82]. The object header hash is stored in the `_merkle_root` attribute alongside the data root hash, and both are covered by the Ed25519 signature.

Parallel Merkle tree construction. In HPC workflows, many compute ranks may write to distinct chunks of the same dataset simultaneously via MPI-IO (in C HDF5) or rayon (in ClawHDF5). If each writer independently tries to update the $O(\log N)$ path from its leaf to the root in the companion dataset, the internal tree nodes become contention hotspots—particularly near the root. We address this with a **map-reduce construction protocol**:

1. **Map phase (fully parallel):** Each writer computes its leaf hash independently. This is embarrassingly parallel—no coordination required.
2. **Reduce phase (coordinated):** Writers pass their leaf hashes to a designated coordinator (rank 0, or a dedicated I/O aggregator) that computes the internal nodes level-by-level and writes the companion dataset. For initial file creation, this can use MPI collective I/O to write the companion dataset in a single pass. For incremental updates affecting a small number of chunks, only the affected paths need recomputation, and the coordinator serializes the

companion writes.

This protocol ensures that the companion dataset is written consistently without lock contention. The C HDF5 design document will specify how this integrates with HDF5's existing collective I/O infrastructure and the SWMR (Single-Writer/Multiple-Reader) access mode.

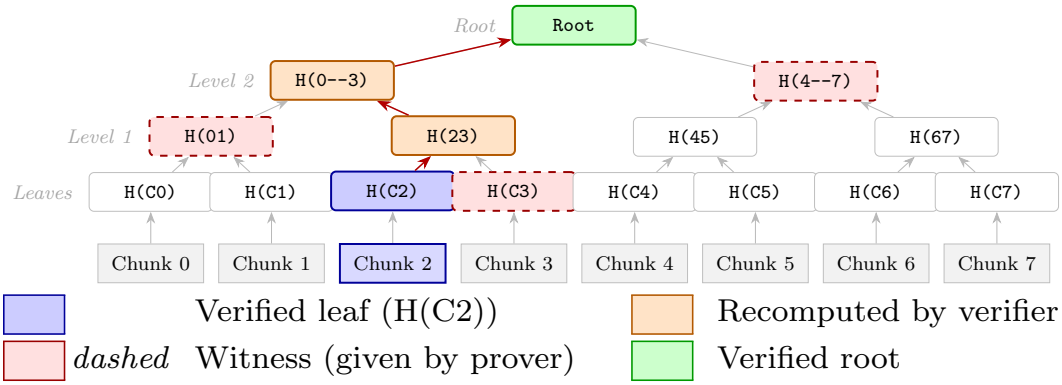

Figure 1: Merkle tree over an 8-chunk dataset. To verify Chunk 2 the verifier hashes it to get H(C2) (blue), is given the three dashed witness hashes H(C3), H(01), and H(4–7), and recomputes H(23) and H(0–3) bottom-up along the bold red arrows ($O(\log N) = 3$ steps) until reaching the signed root—without reading any other chunk data.

5.3 Hash Algorithm Selection

We propose evaluating three hash functions to span the performance–compliance–standards space:

- **SHA-256**: Widely used, FIPS-approved, hardware-accelerated via SHA-NI (x86) and SHA extensions (ARM). HMAC-SHA-256 is already identified in the S2-D2 proposal for streaming signatures.
- **BLAKE3** [56]: Designed for parallelism—the internal tree structure of BLAKE3 maps naturally to a Merkle tree, and it achieves ∼4× higher throughput than SHA-256 on modern CPUs without hardware acceleration. BLAKE3's built-in tree hashing mode could eliminate the need for an external Merkle structure for the per-dataset case.
- **KangarooTwelve** [11]: A SHA-3-family parallel hash derived from `Keccak-p`, included as a parallel alternative in environments where SHA-256 throughput is the bottleneck. Note that K12 is *not* FIPS-approved (only SHA-3-256/384/512 and SHAKE128/256 are FIPS 202 listed); it is therefore limited to non-FIPS deployments. Its 12-round permutation provides a generous security margin while exploiting SIMD and multi-core parallelism on long inputs.

The research will benchmark all three on representative scientific data to determine the performance–compliance tradeoff (SHA-256 for FIPS-required environments, KangarooTwelve for SHA-3-family parallel deployments where FIPS is not a constraint, BLAKE3 for maximum throughput where compliance is unconstrained).

5.4 Verifiable Subset Extraction

A central use case for FAIR scientific data sharing is partial data delivery: a recipient requests a hyperslab (e.g., a spatial region or temporal window) of a published dataset, and must be able to verify cryptographically that the delivered bytes are a true, unmodified subset of the signed parent dataset. A flat per-dataset hash cannot answer this question without delivering the entire dataset; per-chunk AEAD authentication can verify that each delivered chunk is intact *individually*, but cannot prove that the delivered set constitutes the requested hyperslab and that no requested chunks were silently omitted, substituted from a different version, or reordered. Note that raw Merkle inclusion proofs alone also cannot prove coverage: they establish *membership* (this chunk is in the tree), not *completeness* (all required chunks were delivered). Coverage is a property of the *grid hash + Merkle path* combination described below: the Merkle path anchors the trusted chunk-grid parameters from the object header, which then determine exactly which chunks must be present for a given hyperslab request.

This proposal elevates *verifiable subset extraction* to a first-class research contribution. We define:

`extract_subset(D, H)` $\rightarrow (\mathcal{C}, \pi)$**:** given a signed dataset D and a hyperslab H, produce the chunk set $\mathcal{C}$ covering H and a *subset proof* π.

`verify_subset(`$\mathcal{C}, H, \pi$`, root)` $\rightarrow \{\bot, \top\}$**:** accept iff (i) every chunk in $\mathcal{C}$ is authenticated by the signed root via its Merkle path, (ii) $\mathcal{C}$ exactly covers the hyperslab request H given the dataset's chunk grid, and (iii) no chunk in $\mathcal{C}$ has been substituted from a different dataset version (enforced via the per-chunk version counter, §5.7).

The proof π has three components:

(a) Merkle witnesses. The union of Merkle inclusion witnesses for every chunk in $\mathcal{C}$, with shared internal nodes deduplicated.

(b) Coverage certificate. The dataset's serialized object header (datatype, dataspace dimensionality, chunk grid parameters, and relevant attributes) together with its Merkle path to the file-level signed root. This allows the recipient to independently verify the trusted chunk-grid parameters (via the Merkle path) and then recompute which chunk indices *must* have been delivered for hyperslab H.

(c) Per-chunk version counters. One 64-bit version counter per chunk in $\mathcal{C}$, bound into each leaf hash and therefore covered by the witnesses in (a).

The object header is typically only a few kilobytes, so including it in π is negligible relative to the data payload. The verification workflow is: (i) hash the delivered

object header; (ii) verify its Merkle path against the signed file-level root (using the same proof machinery as chunk verification); (iii) extract the trusted chunk-grid parameters; (iv) use those parameters to determine which chunk indices cover H; and finally (v) verify each chunk's Merkle witness against the signed root. This chain ensures the recipient cannot be deceived by a poisoned chunk-grid specification: the grid parameters are covered by the same signature as the data.

Soundness sketch. Define an adversary that wins if it produces $(\mathcal{C}', \pi')$ accepted by `verify_subset` where $\mathcal{C}' \neq \mathcal{C}_H$ (the true covering set for H). Component (a) reduces $\mathcal{C}'$ membership soundness to second-preimage resistance of the underlying hash. Component (b) reduces coverage soundness to the integrity of the chunk-grid parameters in the object header (covered by the file-level root, §5.5). Component (c) reduces version-substitution resistance to the binding of v_k into the leaf hash (§5.7). A complete proof—including the treatment of non-rectangular hyperslabs, partial-chunk requests, and the interaction with HDF5's filter pipeline—is part of the planned work (§5.9).

Open research questions. The proof size $|\pi|$ grows with both the hyperslab shape and the dataset's chunk grid. For an r-dimensional dataset with n_i chunks per dimension and a hyperslab touching m_i chunks per dimension, naive witness aggregation (no deduplication of shared internal nodes) gives $|\pi| = O\big(\prod_i m_i \cdot \log \prod_i n_i\big) = O(M \log N)$, where $M = \prod_i m_i$ is the number of requested chunks and $N = \prod_i n_i$ is the total chunk count. However, for the *contiguous dense hyperslabs* that dominate scientific workloads (rectangular subgrids, time-window selections), shared internal Merkle nodes can be deduplicated during witness construction. In the limiting case of a fully contiguous block, the unique witnesses are concentrated at the spatial *boundary* of the hyperslab rather than its interior: proof size scales with the boundary surface area of the requested block, not its volume, yielding $O(\text{SurfaceArea}(m_1, \ldots, m_r) \cdot \log N)$ rather than $O(\text{Volume} \cdot \log N)$, for axis-aligned hyperslabs under an ideal tree layout (where $\text{SurfaceArea}(m_1, \ldots, m_r) = \sum_i \prod_{j \neq i} m_j$). **Important caveat:** the companion dataset stores the Merkle tree in a 1D linearization of the chunk grid (row-major order by default). A contiguous hyperslab in a 3D dataset does *not* correspond to a contiguous range of leaves in this linearization; its leaves are scattered, so naive deduplication may not recover the surface-area scaling. Whether an optimized layout can achieve this bound empirically—and what linearization order best matches common HPC access patterns—is a central question of RQ6 (§5.10). For strided or irregular access patterns (e.g., every 10th time step), deduplication savings are in general smaller and depend on the companion dataset's linearization order.

Space-filling curve linearization. A promising direction for closing this gap is to replace row-major order with a **space-filling curve** (SFC) to map the r-dimensional chunk grid to a 1D leaf sequence. Morton coding (Z-order curve) [45] is

particularly attractive: it is computed in $O(1)$ per chunk via bit-interleaving of the chunk coordinates, requires no additional metadata, and has been shown to preserve multi-dimensional locality in a provably hierarchical fashion—neighboring chunks in r-dimensional space map to nearby positions in the 1D Morton order. A Hilbert curve [63, 23] offers asymptotically better locality constants (proven by Haverkort and van Walderveen [23] to minimise the bounding-box dilation ratio for rectangular queries) but requires a more involved coordinate transform. Under Morton or Hilbert ordering, a contiguous 3D hyperslab corresponds to a union of *short contiguous ranges* of leaves, substantially increasing the fraction of shared internal Merkle nodes eligible for deduplication and pushing empirical proof sizes toward the theoretical $O(\text{SurfaceArea} \times \log N)$ bound. Quantifying this improvement—across chunk shapes, hyperslab geometries, and dimensionalities—is a central goal of RQ6 and would make for a compelling chart comparing proof size under row-major, Z-order, and Hilbert layouts.

Companion dataset I/O under Morton ordering. Switching to Morton (or Hilbert) linearization changes the mapping from chunk coordinates to companion dataset positions, but the companion remains a flat 1D HDF5 dataset whose internal HDF5 chunking is independent of the leaf linearization order. The coordinate-to-Morton transform is $O(1)$ per chunk via bit-interleaving of the chunk coordinates and imposes negligible CPU overhead on proof construction or verification. The effect on companion read-amplification is, counter-intuitively, *favorable*: under row-major linearization, the leaves of a rectangular r-dimensional hyperslab are scattered across the full companion, because row-major strides differ by up to $\prod_{j>i} n_j$ companion positions between successive hyperslices. Under Morton ordering, spatially adjacent data chunks map to nearby companion positions (the defining locality property of the Z-order curve), so the leaves of a compact hyperslab occupy a smaller number of 4 KB companion chunks—reducing companion I/O and increasing the fraction of sibling nodes that can be satisfied from a single companion chunk read. The residual overhead is the $O(\log N)$ ancestor nodes on the proof path, which must traverse the full tree height regardless of leaf ordering; this cost is identical under any linearization scheme. Quantifying the net reduction in companion chunk reads for Morton versus row-major layouts is explicitly scoped within RQ6.

5.5 Storage of the Merkle Tree

A dataset with N chunks produces a Merkle tree with $\sim 2N$ nodes. Each node is a 32-byte hash (SHA-256 or BLAKE3). Table 2 shows storage overhead for representative dataset sizes.

Table 2: Merkle tree storage overhead for 32-byte (SHA-256) node hashes.

Dataset Size	Chunk Size	Chunks (N)	Tree Size ($\sim 2N \times 32\,\text{B}$)	Overhead
1 MB	64 KB	16	1 KB	0.1%
1 GB	1 MB	1,024	64 KB	0.006%
1 TB	1 MB	1,048,576	64 MB	0.006%
1 TB	4 MB	262,144	16 MB	0.0015%
1 PB	16 MB	67,108,864	4 GB	0.0004%

Note that HDF5 uses fixed-size chunks for each dataset—all chunks have the same dimensions, so the Merkle tree has uniform leaves by construction. The overhead calculations above are therefore exact (not estimates) for any given chunk size and dataset size. Across different datasets within the same file, chunk sizes may vary, but each dataset's Merkle tree is independent.

To prevent second-preimage structural attacks (where a crafted chunk payload matches the 64-byte concatenation of two internal node hashes and is misinterpreted as an internal node), all hash inputs use **domain separation prefixes**: `0x00` is prepended before hashing leaf inputs, `0x01` before hashing internal node pairs, and `0x02` for unallocated sparse chunks (preventing a crafted payload from colliding with the null constant). This makes leaf, internal, and sparse-null hashes structurally incompatible at the input level, regardless of the adversary's control over chunk content.

Sparsity side-channel note. For unencrypted or publicly readable datasets, the deterministic null constant $H(\texttt{0x02} \,\|\, \texttt{null})$ is an acceptable and efficient sentinel. However, for *encrypted* datasets where confidentiality of structure is required, the positions of null leaves in the companion Merkle tree are visible to any reader and directly reveal the physical sparsity pattern of the dataset—potentially leaking sensitive information (e.g., the shape of sparse genomic or radar data implies content). If structure-hiding is required, the null hash should instead incorporate a secret value derived from the dataset encryption key (e.g., $H(\texttt{0x02} \,\|\, \text{PRF}(\text{DEK}, \texttt{"null"}))$), or the dataset must be allocated as fully dense.

Construction note: padded binary tree, not a true SMT. The construction described here—padding the leaf array with PRF-masked null sentinels to the next power of 2 and building a balanced binary tree—should not be confused with a cryptographic *Sparse Merkle Tree* (SMT) in the sense of Laurendeau and Doyen [17]. A true SMT maintains a fixed tree depth equal to the key-space width (e.g., 256 levels for a SHA-256 key), with empty subtrees collapsed via known null-subtree hashes. That construction produces inclusion and non-membership proofs of exactly 256 nodes regardless of data density—directly conflicting with R7 (compact proofs) and R10 (HPC-feasible build cost) for datasets of realistic size. The present design uses a padded balanced binary tree of depth $\lceil \log_2 N \rceil$, which achieves the same structure-hiding goal by deriving a keyed PRF value for every null padding slot

while retaining $O(\log N)$ proof paths. The term "structure-hiding padded binary tree" is used hereafter to distinguish this from a true SMT.

However, using a DEK-derived PRF introduces a **verification trade-off**: because a third-party auditor lacks the DEK, they cannot compute the PRF-masked null hashes and therefore cannot verify the Merkle tree's structure for sparse regions—they cannot even confirm which leaves are null versus populated without decrypting. To permit auditable verification without exposing chunk plaintexts, the design can distribute a separate **Structure Key** $SK = \text{KDF}(\text{KEK}, \texttt{"structure"})$ derived from the Key Encryption Key rather than the Data Encryption Key. Auditors receive SK, which lets them compute $H(\texttt{0x02} \,\|\, \text{PRF}(SK, \texttt{"null"}))$ and reconstruct the full tree topology without ever obtaining the DEK or decrypting any payload chunk. The choice between DEK-derived and KEK-derived PRF nulls is a policy decision: DEK-derived maximizes confidentiality at the cost of excluding third-party auditors; SK-derived permits structural audits while still concealing chunk content. This trade-off is noted as a configuration concern in the security design document. The Structure Key mechanism is scoped as a design option within Phase 2 (§7) alongside the choice of null-hash PRF derivation path.

An important constraint shapes the storage design: HDF5 attributes reside in the dataset's object header and are loaded whenever the dataset is opened, whether the reader needs the attribute or not. For a dataset with one million 1 MB chunks, the 64 MB Merkle tree would bloat the object header unacceptably. Compact dataset storage has the same problem—it places data inside the object header and is designed for payloads under ∼64 KB. Furthermore, storing the tree as an attribute precludes hyperslab access to individual tree nodes, defeating the $O(\log N)$ partial-verification promise.

We therefore propose a **hybrid approach** as the recommended storage design:

1. A small **HDF5 attribute** (`_merkle_root`) on the dataset containing: the root hash (32 bytes), the hash algorithm identifier (e.g., `sha256` or `blake3`), the path to the companion dataset, and a **companion integrity hash**—a hash of the serialized companion dataset that anchors the full tree's integrity to the signed root. This attribute is lightweight enough to impose negligible object header overhead and enables *instant root verification*: a reader rehashes the data chunks, recomputes the root, and compares—without loading the full tree. The companion integrity hash closes a potential attack vector where an adversary modifies internal tree nodes in the companion dataset without altering the root, which could cause partial verification to accept tampered chunks (see §4.1, threat T1).
2. A **chunked companion dataset** (e.g., `/_merkle/temperatures`) storing the complete tree as a 1D byte array with its own chunked layout (e.g., 4 KB chunks for fine-grained access to 32-byte nodes). This provides:
   - Hyperslab access to individual proof-path nodes ($O(\log N)$ positions) without loading the entire tree—critical for partial verification perfor-

mance.
- No object header bloat on the parent dataset.
- Independent chunk tuning for the tree's I/O pattern.
- Natural extensibility: appending chunks to the data dataset appends corresponding nodes to the companion dataset, using the same incremental I/O pattern.

Write amplification and parallel I/O contention. Because the companion dataset's 4 KB chunks each hold $4096/32 = 128$ tree nodes, updating a *single* data chunk requires rewriting its leaf hash plus the $O(\log N)$ ancestor nodes on the path to the root. If those ancestors span multiple 4 KB companion chunks (which occurs as soon as the tree depth exceeds one chunk boundary), a single data write triggers $O(\log N / \log 128) = O(\log N/7)$ companion-chunk I/O operations in addition to the data chunk write itself.

Under concurrent HPC writes—e.g., 10^4 MPI ranks each writing disjoint data chunks simultaneously—the upper-level companion chunks (which hold ancestor nodes shared by many leaves) become severe contention hotspots. Even with the Map-Reduce aggregation protocol, non-collective or asynchronous writes (e.g., SWMR independent-writer mode) can cause redundant rewrite traffic on the same companion chunks as multiple ranks converge toward the root. The recommended mitigation for high-concurrency workloads is **lazy in-memory tree maintenance**: each rank accumulates its leaf hashes in memory and defers companion dataset writes until a collective epoch boundary or file-close event, at which point a single collective I/O pass writes the full tree delta. This eliminates per-chunk companion I/O during the write epoch at the cost of holding the in-progress tree delta in memory; for P ranks each writing k chunks per epoch the memory overhead is $O(Pk \cdot 32)$ bytes. The Map-Reduce aggregator (§7) provides a framework for this batched flush. The write-amplification factor and the effectiveness of lazy flushing under various concurrency levels are explicitly characterized in RQ3.

For **small datasets** (under 256 chunks / ∼16 KB of tree), the full tree may be stored directly in the attribute, avoiding the overhead of a companion dataset. The implementation will select the appropriate strategy automatically based on chunk count.

Portability consideration. The companion dataset approach has one disadvantage: a reader unaware of the Merkle convention will see an extra dataset and ignore it, and a naïve file copy that omits the `_merkle` group breaks verification. The root hash attribute mitigates this—it travels with the dataset and still enables root-level integrity checking even if the companion is lost. The design document (§7, Phase 2) will specify conventions for tools to preserve companion datasets during copy and subset operations.

Downgrade-to-plaintext threat. This *fail-open* default is intentional for

backward compatibility but creates an active attack surface: a compromised storage node or Man-in-the-Middle can strip the `_merkle_root` attribute and the `/_merkle_global` companion dataset, causing an unaware reader to process tampered data without error. To close this gap, the S2-D2 framework's Data Protection Factory will enforce a **Strict Mode**: when a file is known to be governed by a provenance policy (via out-of-band trust metadata, filesystem extended attributes, or a policy registry), the *absence* of a valid Merkle root is treated as a fatal security exception rather than a graceful degradation. Strict Mode is opt-in for interoperability but mandatory in high-assurance deployment contexts.

Strict Mode bootstrapping. A subtlety in Strict Mode is the *bootstrapping problem*: the mechanism by which a verifier knows a file is policy-governed must itself be tamper-evident. If policy membership is stored solely inside the HDF5 file (e.g., as an attribute), an attacker who can strip the Merkle root can equally strip the policy flag, causing the verifier to silently fall back to non-strict mode. Strict Mode therefore requires an **out-of-band policy registry**—a separate, protected source of truth that maps file identifiers (e.g., cryptographic file fingerprints or storage URIs) to their required policy level. Concrete mechanisms include: filesystem extended attributes on the directory or mount point (xattr enforced by a MAC policy, e.g., SELinux or AppArmor); a site-managed policy server queried at file open; or an LDAP/SCIM directory entry for the dataset. The verifier consults this registry *before* attempting to read the file, so policy enforcement is independent of file content. If the registry is unavailable—for example, when a file is transferred to a disconnected environment—the verifier must either fail closed (reject all files) or fall back to an embedded policy hint, accepting the reduced security guarantee. The C HDF5 design document (§7, Phase 2) will specify the policy distribution interface and the required trust properties of the registry.

Crash consistency. The hybrid storage layout introduces a write-ordering requirement: if the application crashes after writing an encrypted chunk but before updating the companion Merkle dataset, the tree becomes inconsistent with the data. The required write-ordering protocol is:

1. Write the chunk data to disk (via the filter pipeline).
2. Write the updated Merkle tree nodes to the companion dataset (leaf hash, then recomputed path to root).
3. Update the root attribute (root hash, companion integrity hash, and dataset version number).

Steps 2 and 3 must be flushed before the write is considered committed. In the ClawHDF5 Rust prototype, explicit sync calls enforce this ordering at the I/O layer. In the C HDF5 library, however, file writes pass through the **Metadata Cache (MDC)**, which coalesces and evicts pages heuristically; simply issuing API calls in the correct order does not guarantee on-disk write ordering. Enforcing the three-step

sequence in C HDF5 requires either explicit write-barrier flushes via `H5Fflush()` after each step, or use of HDF5's internal *parent–child flush dependency* mechanism (see P2.6, §7) to express the ordering constraint to the MDC directly—causing it to flush child pages (companion dataset nodes) before parent pages (root attribute) during ordinary eviction. This distinction is noted here and addressed in detail in the Phase 2 C HDF5 design document. On recovery after a crash, the system detects the inconsistency (the root attribute will not match the companion dataset state). The recovery behavior depends on the signing context:

- **Unsigned datasets**: The tree may be rebuilt by rehashing the on-disk chunk data, since no authenticity guarantee was in force.
- **Signed datasets**: The system must **fail closed**—report the inconsistency and refuse to auto-rebuild. The risk differs by role: a *verifier* without the private key cannot re-sign anything and simply reports the inconsistency and halts. A *writer* with the private key (or with auto-signing enabled) faces the laundering risk: if the runtime automatically rehashes on-disk data and signs the result, it would compute and sign a new root over potentially tampered chunks, institutionalizing the corruption. In both cases the system flags the affected chunks as unverified and requires explicit operator intervention (re-signing from a trusted source or restoring from a known-good backup).

The C HDF5 design document (§7, Phase 2) will specify this protocol in detail, including interaction with HDF5's existing metadata journaling and flush semantics, and whether a cryptographic write-ahead log can provide stronger crash recovery guarantees for signed datasets without the blind-signing risk.

Error response and recovery. When `verify_chunk` or `verify_dataset` returns a `MerkleError` (P1.4, §7), the calling application faces a three-way decision:

Halt. Refuse all further access to the file and surface a hard error. This is the correct default for automated pipelines, archival ingest, and any context where consuming unverified data would propagate the compromise.

Quarantine. Mark the file as unverified, deny all writes, and allow read-only access only under explicit operator acknowledgement. Appropriate when forensic inspection of the affected data is required before disposal.

Alert and continue (degraded). Log the failure, notify an operator, and proceed in a read-only mode without the integrity guarantee. Acceptable only in interactive data-exploration workflows where the operator is immediately present and the downstream use is non-critical.

In all cases the error must be logged with the affected file path, dataset name, chunk index, and the full `MerkleError` variant before any further action is taken.

The `MerkleError` variant also provides a first-signal triage between accidental hardware corruption and deliberate tampering. `HashMismatch{chunk_idx}`—a single leaf discrepancy with an otherwise intact companion dataset—is the pattern most consistent with silent data corruption (bit rot, sector fault), though it cannot

exclude targeted modification. `CompanionTampered` is harder to explain as hardware failure, because ordinary faults rarely corrupt exactly the companion dataset while leaving chunk data intact; treat it as adversarial until storage diagnostics rule that out. `SignatureInvalid` requires access to the signing key and is not a pattern produced by hardware failure at all; it must be treated as adversarial. This triage is diagnostic, not definitive: a sophisticated attacker can mimic hardware-failure patterns, and `HashMismatch` on multiple non-contiguous chunks with no plausible write history is itself suspicious.

Although T4 (§4.1) characterizes rollback as an *attack* vector, rolling back to the last known-good signed version is also the **primary recovery path** after detected tampering in signed datasets. A verifier that has previously observed version v of a file and cached or logged its signed root can restore integrity by reverting to that snapshot, provided the backup predates the tampering event. This is a second reason why the system must checkpoint observed signed roots when freshness guarantees matter (see the version-counter discussion in §5.7): without a cached prior root the verifier has no reference point from which to recover, and the only remaining options are operator re-signing from a trusted source or full restoration from backup. The prototype implementation of this recovery path—version-counter persistence, the provenance journal, and `restore_to_version`—is specified as task P2.2b (§7, Phase 2).

5.6 Filter Pipeline Interaction

The position of hash computation in the filter pipeline has security implications. We propose the following ordering as the default:

$$\texttt{Application data} \xrightarrow{\text{shuffle}} \xrightarrow{\text{compress}} \xrightarrow{\text{encrypt}} \xrightarrow{\textbf{hash (Merkle leaf)}} \texttt{Storage}$$

This is an *Encrypt-then-MAC* (EtM) construction, which is the standard cryptographic best practice for authenticated encryption and offers two important properties:

- **Third-party integrity verification without decryption.** Because the Merkle leaf hashes cover the ciphertext, any party—a storage node, middleware, archival system, or researcher without the decryption key—can verify that stored chunks have not been modified. This is critical for data custody workflows where the verifier is not the data owner.
- **Early rejection of tampered ciphertext.** A reader encountering modified ciphertext can detect the tampering *before* spending CPU cycles on decryption. Under a Hash-then-Encrypt ordering, the reader must decrypt the (potentially attacker-controlled) ciphertext before discovering via the hash that the plaintext is invalid—wasting cycles and expanding the attack surface.

Trade-off: plaintext provenance. EtM hashes the ciphertext, not the plaintext. This means that rotating the encryption key (re-encrypting the same data with a new key) changes the ciphertext and invalidates the Merkle tree, requiring a full tree recomputation. If plaintext-stable provenance is strictly required (e.g., to allow key rotation without rebuilding the tree, or to prove that two differently-encrypted copies contain the same data), a secondary plaintext hash can be stored alongside the primary ciphertext-based tree. The implementation will support both orderings, with EtM as the default and Hash-then-Encrypt available via an explicit configuration flag, along with a clear warning that the HtE mode sacrifices early tamper rejection and third-party verifiability.

This ordering will be validated experimentally by measuring verification correctness and performance for both permutations.

5.7 Nonce Derivation for Stream Ciphers

Both AES-256-CTR and ChaCha20 are stream ciphers that generate a pseudorandom keystream XORed with the plaintext. If a chunk is updated in place using the same symmetric key and the same nonce/IV, the keystream is reused: an attacker who observes both the old ciphertext $C_{\text{old}} = P_{\text{old}} \oplus KS$ and the new ciphertext $C_{\text{new}} = P_{\text{new}} \oplus KS$ can compute $C_{\text{old}} \oplus C_{\text{new}} = P_{\text{old}} \oplus P_{\text{new}}$, destroying confidentiality. This is a **catastrophic vulnerability** if the nonce is derived statically from the chunk index alone.

To safely support in-place updates, we derive the nonce as:

$$\text{Nonce}(k, v) = \text{KDF}(\text{DEK},\ k \,\|\, v) \tag{1}$$

where k is the chunk index, v is a monotonically increasing **per-chunk version counter**, and DEK is the per-dataset Data Encryption Key described in §5.8. The version counter is stored in the companion Merkle dataset alongside the tree nodes (one 64-bit counter per leaf, adding 8 bytes per chunk of overhead). Every write to chunk k—whether an append or an in-place update—increments v, guaranteeing a unique nonce per (key, chunk, version) triple.

Crash-safe version counter increments (WAL requirement). A subtle conflict exists between the crash-consistency write order of §5.5 ("write chunk first, then update companion tree") and the nonce-safety requirement above ("v must be incremented *before* the chunk is encrypted and written"). If v is incremented only in the companion dataset after the chunk write, a crash between the chunk write and the companion update leaves v at its old value; the next write to the same chunk would then re-derive the same nonce, catastrophically reusing the keystream. Conversely, incrementing v in the companion dataset *before* the chunk write violates the crash-consistency write order.

The resolution is a **Write-Ahead Log (WAL)** for version counters:

1. Append a journal record ($chunk\_idx$, v_{new}) to a crash-durable WAL (e.g., a small dedicated HDF5 dataset or an OS-level append-only file) *before* computing the nonce.
2. Derive the nonce using v_{new}, encrypt the chunk, and write the ciphertext.
3. Promote the journal record into the companion dataset (update the leaf's stored version counter to v_{new}) and recompute the Merkle path to the root.
4. Mark the journal record as committed.

On crash recovery, any uncommitted journal record indicates an in-progress write: the system replays the chunk encryption (using the journaled v_{new}) and completes the companion update before declaring the file consistent. This guarantees the nonce is never reused regardless of crash timing. The C HDF5 design document (§7, Phase 2) will specify the exact WAL format and interaction with HDF5 flush semantics. The version counter also serves double duty for rollback detection (threat T4 in §4.1): the dataset-level version in the signed root is the maximum of all per-chunk versions, providing a monotonic timeline. The locally stored high-resolution timestamp τ in the signed payload R similarly defends against T4 by making rollback detectable to any verifier with a prior observation. However, it is important to note that **local timestamps provide no protection against key compromise**: an attacker who obtains the Ed25519 private key can generate a freshly signed payload with a backdated τ, fabricating an apparently old provenance record. True non-repudiation and key-compromise resilience require an *external* cryptographic timestamp from a trusted Timestamp Authority (TSA, RFC 3161 [2]), which is why the Sigstore + TSA token stretch goal (§5.8) is architecturally significant and not merely a deployment convenience.

HPC write-throughput concern. In high-throughput parallel write environments—checkpoint workloads where 10^4 or more MPI ranks write simultaneously—the per-chunk WAL protocol above imposes a disk sync before every chunk write, defeating asynchronous I/O and collective I/O pipelines. Two mitigations address this:

- **Epoch-based WAL batching.** A write epoch groups a bounded set of chunk writes. Counter increments for all chunks in the epoch are buffered in memory and flushed to the WAL as a single collective record at the epoch boundary, before any chunk data is written. On crash recovery the full epoch record is replayed. This converts k per-chunk sync operations into one per-epoch sync, amortizing the flush cost across the batch.
- **Random 96-bit nonces (recommended for high-frequency parallel writes).** An alternative is to generate a fresh cryptographically random 96-bit nonce per write and store it alongside the ciphertext, eliminating the stateful counter and WAL entirely. The birthday-collision probability for a single dataset is negligible up to 2^{32} writes (the standard bound for a 96-bit nonce), and the storage overhead is 12 bytes per chunk. This is the simplest path for HPC deployments where checkpoint I/O throughput is paramount, at the

cost of eliminating the monotonic version counter's secondary role in rollback detection—rollback resistance then relies solely on the dataset-level version counter in the signed root.

The C HDF5 design document (§7, Phase 2) will characterize the throughput impact of each option on representative HPC checkpoint workloads and specify which is the default for high-concurrency deployments. The implementation will support both the version-counter (WAL) and random-nonce schemes, selectable via the filter pipeline configuration.

5.8 Signing Authority and Key Management

A Merkle tree without a signed root is vulnerable to wholesale replacement (an attacker replaces both data and tree; see threat T1 in §4.1). The core signing mechanism is:

Trust assumptions. The security guarantees of this scheme rest on three assumptions that must hold simultaneously:

1. **The signing private key is secret.** The signing private key is the single point on which all integrity guarantees depend. If it is compromised, an attacker can produce a valid signature over any data, and no part of the scheme can detect this. All other defences (Merkle tree, companion integrity hash, version counter) become ineffective once the key is in attacker hands. Key revocation and rotation are therefore operational requirements, not optional features.
2. **The verifier obtains the public key from a source independent of the file being verified.** A public key read from the same file it authenticates provides *no adversarial guarantee*: an attacker who can modify the HDF5 file can substitute both the data and the `_provenance_pubkey` attribute, producing a file that verifies perfectly against the attacker-controlled key. Secure verification requires the verifier to hold the author's public key from a trusted out-of-band source—the Task 2 KeyStore, a prior cached observation, or an institutional key registry. The file-embedded key is a *convenience* for environments where the file's physical custody is already trusted (e.g., an institutional archive with access controls); it is not a substitute for independent key distribution in adversarial settings.
3. **The scheme does not detect a malicious legitimate signer.** The Merkle tree and signature verify that the data matches what the key-holder signed. They say nothing about whether the key-holder *should have* signed that data, or whether the signer is acting honestly. Authorisation policy—who is permitted to sign a given dataset—is outside the cryptographic scope of this design and must be enforced by the surrounding key-management infrastructure.

These assumptions are orthogonal to the post-quantum signing upgrade (§5.8): the hybrid Ed25519 + ML-DSA-65 scheme extends the lifetime of assumption 1 against

a future CRQC but does not change assumptions 2 or 3. They are also distinct from R8 (*no trusted setup*, §4.2): R8 guarantees that the *algorithm* needs no ceremony to generate its parameters, while assumptions 1–3 govern how a specific *deployment* of that algorithm must be operated. A scheme can satisfy R8 and still require careful operational trust management—and this one does.

- The file creator signs the root hash (and the companion integrity hash) with an **Ed25519 private key**. Ed25519 is chosen for its fast verification, small signature size (64 bytes), and resistance to timing side-channels. The signed payload includes the dataset version number and a high-resolution timestamp to defend against rollback attacks (threat T4).
- The corresponding public key is distributed via one of two mechanisms: (a) embedded in a file attribute (`_provenance_pubkey`) for self-contained verification in trusted-custody environments (see trust assumption 2 above), or (b) distributed via the Task 2 KeyStore (`HDF5_PLUGIN_KEYSTORE_DIR` / `HDF5_PLUGIN_KEYSTORE`), reusing the trusted-key infrastructure from PR #6198. **The KeyStore path is required for adversarial settings**; the file-embedded path is a convenience for deployments where file custody is already controlled.

Long-lived key distribution and trust bootstrapping. The two mechanisms above are sufficient for datasets shared within an institution or over a short-to-medium time horizon. However, R6 requires that *any* downstream collaborator—including a researcher verifying a dataset 50 years after signing—can authenticate the public key itself without any prior relationship with the originating laboratory. This is the classic key-distribution problem, and it becomes non-trivial at archival timescales where institutional email addresses, domain names, and certificate authority hierarchies may all have changed:

- **Institutional PKI (X.509).** The signing key can be certified by an institutional or community CA already used in research computing (e.g., the IGTF grid PKI, or a DOE Office of Science CA). The certificate chain anchors the public key to an auditable institutional identity and can be stored as an HDF5 attribute alongside the signature. Limitation: CA hierarchies are restructured on decadal timescales; a certificate issued under a deprecated root may require explicit trust-anchor pinning in a verifier's trust store to remain verifiable.
- **Decentralized Identifiers (DIDs) [66].** A W3C standard in which a self-sovereign identifier (e.g., `did:web:hdfgroup.org:keys:v1`) resolves to a *DID Document* listing current and historical public keys for that identifier. Because DID Documents can be anchored in content-addressed decentralized registries (IPFS, a consortium ledger, or an immutable web resource), they do not depend on the continued existence of any single institution's domain or CA. Storing a DID alongside the Rekor bundle provides a trust-bootstrapping path that survives institutional restructurings and CA deprecations. DID-

based key binding is scoped as a stretch goal and noted as an area for future work in the C HDF5 design document.

- **Sigstore / OIDC.** As noted in the Sigstore stretch goal below, Sigstore timestamps the signing event via Rekor and a TSA token but relies on OIDC identity (e.g., a university email). Sigstore therefore answers *when* a key was used, not *who controls it today*. For maximum long-term trust, the OIDC-backed Rekor bundle (timestamping) is best paired with either a PKI certificate or a DID (persistent identity).

For the Phase 1 prototype, the baseline design embeds the public key in a file attribute and relies on the Task 2 KeyStore for institutional deployments. Long-term trust binding via PKI or DIDs is deferred to the C HDF5 design document as a configurable extension.

Post-quantum hybrid signing (T5: harvest-now, forge-later). Scientific data is routinely retained for decades—NOAA climate baselines, EQSIM simulation outputs, genomic reference assemblies, and astronomical survey data are all held on archival timescales of 30–100 years. Within that horizon, a cryptographically relevant quantum computer (CRQC) is widely expected to render Ed25519 (and all elliptic-curve and RSA signatures) forgeable via Shor's algorithm. An adversary who archives today's signatures and waits—the *harvest-now, forge-later* threat (T5)—can later fabricate signed roots over tampered datasets attributed to the original signer. This proposal therefore promotes **post-quantum hybrid signing** to a first-class research deliverable. The signed root attribute carries a hybrid signature consisting of:

$$\begin{aligned} \sigma_{\text{root}} &= \big(\sigma_{\text{Ed25519}}(R),\ \sigma_{\text{ML-DSA-65}}(R)\big), \\ R &= \text{root} \,\|\, \text{companion\_hash} \,\|\, v_{\text{dataset}} \,\|\, \tau \,\|\, \text{AlgID} \end{aligned} \tag{2}$$

where R is the canonical signed payload (root, companion integrity hash, dataset-level version counter, timestamp, and a compact algorithm identifier). Including **AlgID**—a fixed-width encoding of the hash function (e.g., `BLAKE3`) and AEAD cipher (e.g., `ChaCha20-Poly1305`) in use—prevents *Algorithm Substitution Attacks*: an attacker who rewrites the `_merkle_root` attribute to specify a weaker hash or cipher cannot produce a valid signature over the modified payload, because the algorithm identifiers are covered by both σ_{Ed25519} and $\sigma_{\text{ML-DSA-65}}$. ML-DSA-65 (NIST FIPS 204 [54]) is NIST PQC Category 3, providing approximately AES-192 strength against both classical and quantum attacks; SLH-DSA-128s (NIST FIPS 205 [55]) is also implemented as an alternative for deployments preferring hash-based signatures whose security rests only on the underlying hash function. A verifier accepts the signature iff *both* components verify (strict-AND policy). This means that even today—before a CRQC exists—a verifier will reject if either signature is invalid; a file signed with Ed25519 alone will not be accepted. This is intentional: it prevents a downgrade attack where an adversary strips the PQ component. Until a CRQC

exists, Ed25519 dominates verification time (ML-DSA-65 verification is somewhat slower); once a CRQC exists, the ML-DSA-65 component preserves authenticity while the Ed25519 component is considered compromised.

One signature, all datasets. The hybrid signature is applied **once** to the global *file-level* root that aggregates all per-dataset roots (§5.5). Individual datasets contribute a 32-byte hash to the global tree; they do not each carry a 3.3 KB ML-DSA-65 signature. For a file with 10^5 datasets, the total signing overhead is one 3.3 KB signature (plus a ∼2 KB public key) stored in a single file-level attribute, not $10^5 \times 3.3$ KB = 330 MB spread across per-dataset object headers. Signing and verifying an individual dataset at rest still requires only $O(\log D)$ hash reads from the companion tree (where D is the number of datasets in the file), never a per-dataset PQ signature. The file-level tree nodes are stored in a dedicated companion dataset at the HDF5 file root (e.g., `/_merkle_global`), using the same chunked-companion layout as per-dataset trees (§5.5), so the global tree itself imposes no object-header overhead.

Why this still needs to be measured at HDF5 scale. ML-DSA-65 signatures are ∼3.3 KB and public keys are ∼2 KB, versus 64 B and 32 B for Ed25519. SLH-DSA signatures are larger still (∼8 KB to ∼50 KB depending on parameters; SLH-DSA-128s is ∼7.7 KB, SLH-DSA-256f reaches ∼49 KB). Even with a single file-level signature, the *object-header encoding* of the file-level root attribute (which contains the root hash, companion integrity hash, version counter, timestamp, and the hybrid signature blob) must remain within HDF5's attribute-storage limits. HDF5 object headers begin failing silently past ∼64 KB and cannot accommodate 330 MB. Verifying that the file-level root attribute stays within these limits for all supported parameter combinations, and measuring batch verification throughput at 10^4–10^7 chunks (covering 10 GB–10 TB datasets at 1 MB chunk size), is the substance of RQ7 (§5.10) and is intended to be a primary empirical contribution of the resulting paper, supplying the first published PQ-signature performance numbers for scientific data archives.

Fallback: signature material in a dedicated metadata dataset. If RQ7 measurements show that the combined file-level root attribute payload (root hash, companion integrity hash, version counter, timestamp, AlgID, Ed25519 signature + public key, ML-DSA-65 signature + public key, and optionally a Rekor bundle plus TSA token) approaches the ∼64 KB object-header limit, the design automatically falls back to the same chunked-companion pattern already used for tree nodes: all signature material is written to a dedicated 1D metadata dataset (e.g., `/_merkle_signature`), and the file-level root attribute is reduced to a single 32-byte **metadata-blob hash** that commits to that dataset's contents. The verifier fetches the metadata dataset in one read, confirms that its hash matches the attribute, and proceeds with normal signature verification—imposing

one additional sequential read but keeping the file-level root attribute under 64 bytes, well within any HDF5 object-header budget. **Non-circularity.** The signed payload $R = \text{root} \,\|\, \text{companion\_hash} \,\|\, v_{\text{dataset}} \,\|\, \tau \,\|\, \text{AlgID}$ *does not* include $H_{\text{sig\_ds}}$ (the metadata-blob hash stored in the attribute). This is intentional: $H_{\text{sig\_ds}}$ commits to the dataset that *contains* σ, so including it in R would require signing a hash of something that already contains the signature—a circular dependency with no fixed point. The verification chain is therefore a clean one-way dependency: $H_{\text{sig\_ds}} \rightarrow$ verify integrity of `/_merkle_signature` $\rightarrow$ extract $\sigma \rightarrow$ verify $\sigma(R)$, with no back-reference from R to $H_{\text{sig\_ds}}$. The fallback threshold is provisionally set at 32 KB ($\sim$50 % of the object-header limit) to preserve headroom for future attribute additions; the exact threshold will be validated empirically against the RQ7 benchmark results.

Symmetric key management. The asymmetric Ed25519 key provides signing (authenticity), but the encryption layer (AES-256-CTR, ChaCha20-Poly1305) requires symmetric Data Encryption Keys (DEKs). The full design of key management is a broader S2-D2 concern (spanning Tasks 2 and 3); for the ClawHDF5 prototype, we adopt the following baseline scheme: a single DEK per dataset is generated at dataset creation, wrapped (encrypted) with a Key Encryption Key (KEK) derived from user credentials via HKDF-SHA-256, and stored as an HDF5 attribute (`_provenance_wrapped_dek`) on the dataset. The KEK is never stored in the file. This supports the common case where one user or group encrypts a dataset; multi-party access (multiple KEKs wrapping the same DEK) and external KMS integration are deferred to the C HDF5 design document.

AEAD integration. When ChaCha20-Poly1305 is used, the cipher natively produces a 16-byte Poly1305 authentication tag per chunk. To avoid discarding this tag (which would waste a security property the cipher already provides), the Merkle leaf hash for a ChaCha20-Poly1305-encrypted chunk is computed over the concatenation of the ciphertext and the Poly1305 tag:

$$H_{\text{leaf}}(k) = H(\texttt{0x00} \,\|\, k \,\|\, \text{Ciphertext}_k \,\|\, \text{Tag}_k \,\|\, v_k), \tag{3}$$

where v_k is the per-chunk version counter (§5.7) and `0x00` is the leaf domain-separation prefix (§5.5). Explicitly binding the chunk index k into the leaf hash prevents *position-swapping attacks*: an adversary cannot substitute the ciphertext, tag, and version counter of chunk k' in place of chunk k, because the index is baked into the hash; the forged leaf will not match the honest leaf at position k in the tree, and Merkle path verification will reject it. **Length-prefixing requirement.** If compression is applied before encryption (as in the shuffle→compress→encrypt pipeline of §5.6), the ciphertext length is variable across chunks. Concatenating variable-length fields without explicit length delimiters can admit *boundary-shifting* (a.k.a. length-extension at the field boundary): a transcript $(\text{Ct}_1 \| \text{Tag}_1)$ with $|\text{Ct}_1| = n_1$

and $(\mathrm{Ct}_2 \| \mathrm{Tag}_2)$ with $|\mathrm{Ct}_2| = n_2$ may produce the same bit string if a suffix of Ct_1 is reassigned to the front of the tag field. The implementation must therefore encode the serialization as $\texttt{0x00} \| \mathrm{len}(k) \| k \| \mathrm{len}(\mathrm{Ciphertext}_k) \| \mathrm{Ciphertext}_k \| \mathrm{Tag}_k \| v_k$, where all length fields are fixed-width big-endian integers. Because the Poly1305 tag is always exactly 16 bytes and the version counter v_k is always a fixed-width integer, only the ciphertext length field needs to be explicit; nonetheless, including all length fields is the safest practice and is mandated in the implementation specification. This is especially important when variable-length compression is applied, since differing ciphertext lengths across chunks could otherwise introduce structural ambiguity in proof generation. Critically, including v_k in the leaf hash **cryptographically binds the version counter to the tree**: if an attacker selectively rolls back a chunk and its corresponding version counter in the companion dataset, the leaf hash changes, breaking the path to the signed root. For AES-256-CTR (which is not an AEAD cipher), the HMAC-SHA-256 authentication tag serves an analogous role: $H(\texttt{0x00} \,\|\, k \,\|\, \mathrm{Ciphertext}_k \,\|\, \mathrm{HMAC}_k \,\|\, v_k)$. For *plaintext* (unencrypted) chunks, the leaf hash binds the raw chunk data and version counter directly: $H(\texttt{0x00} \,\|\, \mathrm{len}(k) \,\|\, k \,\|\, \mathrm{len}(\mathrm{Chunk}_k) \,\|\, \mathrm{Chunk}_k \,\|\, v_k)$; the same length-prefixing rules from above apply whenever the chunk payload may be variable-length (e.g., after compression is applied in the filter pipeline). The index k is bound in all cases to prevent position-swapping attacks.

Implementation note: formalized tuple serialization. Rather than maintaining custom byte-packing logic, the implementation **should** adopt a standardized tuple-hashing scheme that eliminates the need to write length prefixes by hand. NIST SP 800-185 **TupleHash** [29] (built on SHAKE128/SHAKE256) provides a formally defined, collision-resistant method for hashing an ordered sequence of byte strings; it encodes each element's length internally, so the caller never constructs explicit `len()` fields or worries about field-boundary ambiguity. If BLAKE3 is the selected hash function, the same guarantee is achieved by constructing a single keyed-hash invocation whose input is the concatenation of length-prefixed fields—but here the prefix format is defined once in a helper and never duplicated across call sites. In either case, deferring canonicalization to a vetted library reduces the attack surface compared to hand-rolled length-prefixing that must be re-audited whenever the leaf-input field layout changes. The serialization in the preceding paragraph therefore represents the *logical* tuple $(k, \mathrm{Ciphertext}_k, \mathrm{Tag}_k, v_k)$; a conforming implementation *may* satisfy the canonicalization requirement by passing these four fields directly to TupleHash or to an equivalent library-provided multi-part hash API.

Stretch goal: Sigstore transparency log. As an extension, the signed root may be published to a **Sigstore transparency log** [48, 35], establishing a public, append-only audit trail of who signed what and when. This extends Task 3

provenance with a public, tamper-evident audit trail and is independent of Task 2's plugin-signing KeyStore (which does not use Sigstore). A practical consideration: Sigstore uses short-lived certificates issued via OIDC/Fulcio, which means the certificate will have expired by the time a researcher verifies the dataset months or years later. The implementation must therefore store the **Rekor inclusion proof** (transparency log bundle) as an HDF5 attribute alongside the signature, enabling offline verification against the log's Merkle root without requiring the original certificate to still be valid. For datasets archived for decades—a common requirement in scientific data management—institutional domain changes may render OIDC identity verification ambiguous. To support **Long-Term Validation** (LTV), the implementation will also store a trusted **Timestamp Authority** (TSA) token [2] alongside the Rekor bundle. The TSA token mathematically proves the signature occurred before the certificate expired, ensuring cryptographic validity for 50+ years independent of the signer's institutional identity. Sigstore integration involves external service dependencies and is therefore scoped as a stretch goal for Phase 2 (§7).

5.9 Security Analysis Methodology

A central weakness of prior provenance-style systems papers is that the security claim is asserted by intuition rather than argued formally, inviting reviewer rejection at security venues and leaving operational deployments without a defensible model. We therefore commit to producing either a symbolic protocol model or a game-based argument as part of this research:

Game-based argument (baseline, in scope). A written argument in the paper showing that the construction (EtM filter pipeline) + (version-counter nonce) + (hybrid signed root over companion-integrity hash) achieves *verified-subset soundness* (an EUF-CMA-style goal over the authenticated data structure: no adversary can produce an accepted $(\mathcal{C}', \pi')$ that differs from the honest covering set) against the adversary defined in §4.1, by reduction to (a) the second-preimage resistance of the underlying hash function, (b) the EUF-CMA security of the hybrid signature scheme, and (c) the per-chunk INT-CTXT security already provided by the AEAD cipher (for encrypted datasets) or by the Merkle leaf hash alone (for plaintext datasets). Note that the Merkle layer does not re-derive INT-CTXT at the chunk level—AEAD ciphers such as ChaCha20-Poly1305 already provide this via their authentication tag; instead, the Merkle layer adds *cross-chunk and cross-version binding* that no per-chunk AEAD can provide. We follow the Bellare–Namprempre framework for analyzing generic composition [6].

Symbolic model (stretch, preferred for security venues). A Tamarin Prover [40] model of the file-creation, update, and verification protocols, including the version-counter binding into leaf hashes and the hybrid-signature root, with machine-checked verification of the principal security lemmas (authenticity, freshness, subset soundness). Because Tamarin is

designed for finite protocol state machines and does not natively handle unbounded recursive data structures, the model will prove lemmas over a **depth-bounded instantiation** of the tree (e.g., depth $d \leq 30$, covering trees of up to 10^9 chunks). Security at arbitrary depth is argued by structural induction over the depth-bounded proof, a standard technique in symbolic tree analysis. The Tamarin model and all proof scripts would be released alongside the implementation.

The game-based argument is treated as a Phase 2 deliverable; the Tamarin model is a stretch deliverable whose completion would target a security venue (USENIX Security, ACSAC) for the resulting paper.

5.10 Research Questions

The systems research questions below (RQ1–RQ5) are posed *comparatively*: each metric is measured not only for the Merkle construction but also for the primitives whose characteristics come closest to it among the candidates of §4.3—per-chunk asymmetric signatures (the closest public-verifiable competitor, which shares partial verification, tamper localization, and public verifiability and differs mainly on proof size and build cost), per-chunk MAC/AEAD (which shares the chunk-level access structure but not public verifiability), and the flat whole-dataset hash that is ClawHDF5's current provenance feature. To our knowledge no prior work reports these metrics head-to-head for chunked, multi-terabyte *scientific* data on common hardware; establishing that comparative baseline is therefore both a prerequisite for interpreting the Merkle results and a contribution in its own right. The baseline harness (P1.2b, §7) is built and measured *concurrently* with the Merkle implementation, so that the cost and capability gaps of existing and comparable methods are established empirically rather than asserted. The three contribution-specific questions (RQ6–RQ8) concern properties of the subset-extraction, hybrid-signing, and adversarial-soundness work and are evaluated against the relevant per-chunk baselines where one exists.

RQ1 What is the storage overhead of the Merkle tree—and of the comparable per-chunk-signature, per-chunk-MAC, and flat-hash baselines—as a function of chunk size and dataset size, and at what chunk count does the hybrid companion-dataset approach outperform attribute-only storage in object header impact and partial verification access time?

RQ2 What is the partial verification latency for realistic scientific access patterns (sequential scan, strided hyperslab, random chunk access), measured for the Merkle tree, for per-chunk-signature and per-chunk-MAC verification, and for full-dataset rehashing?

RQ3 How does incremental tree extension perform for append-heavy workloads (time-series, streaming sensor data) compared to re-signing the entire dataset?

Additionally, what is the write-amplification factor—number of companion-dataset chunk I/Os per data chunk update—as a function of tree depth, companion chunk size, and writer concurrency, and does lazy in-memory tree maintenance (epoch-boundary collective flush) reduce amplification and upper-level contention to an acceptable level for parallel HPC writers compared with eager per-chunk companion writes?

RQ4 What is the throughput advantage of BLAKE3 and KangarooTwelve over SHA-256 for Merkle tree construction at HDF5-typical chunk sizes, and does BLAKE3's internal tree hashing mode offer additional speedup for whole-dataset root computation? (Note: BLAKE3's internal tree uses fixed 1 KB leaf chunks and does not expose intermediate nodes via its public API, so it cannot replace the external Merkle structure for partial verification. This question is therefore scoped to throughput comparison, not structural subsumption.)

RQ5 Can the Merkle tree detect and localize tampering injected into specific chunks of real-world scientific datasets (NOAA climate, EQSIM simulation [38])?

RQ6 (Subset extraction.) What is the achievable proof size $|\pi|$ for verifiable hyperslab subset extraction (§5.4) as a function of hyperslab shape (rectangular, strided, sparse), dataset chunk grid, and companion-dataset layout, and what is the proof-construction and proof-verification latency relative to the bandwidth saved over a naïve full-dataset re-download?

RQ7 (Post-quantum signing.) What are the storage, sign, and verify costs of hybrid Ed25519 + ML-DSA-65 (and Ed25519 + SLH-DSA-128s) signed roots at 10^4–10^5 datasets per file and 10^4–10^7 chunks per dataset (10 GB–10 TB at 1 MB chunk size), including object-header impact and end-to-end verification time on x86-64 (with and without AVX-512) and ARM?

RQ8 (Adversarial soundness.) Can a white-box adversary with full design knowledge and read/write access to the storage medium evade detection by the verification protocol within the stated threat model (T1–T8)? For each attack class (T1a, T1b, T2–T5, subset-extraction attacks), we report: whether the verifier rejects, the tamper-localization granularity (which chunks are flagged), and the detection latency. Any successful attacks are root-cause-analyzed and the design is updated. This is evaluated via the white-box red-team study described in §6.5.

6 Experimental Plan

6.1 Datasets

We will use publicly available scientific datasets identified in the S2-D2 proposal:

- **NOAA Climate Data Online**: Multi-variable gridded climate data, representative of regular-access time-series workloads.
- **EQSIM earthquake simulation** [38]: Large-scale simulation output from Lawrence Berkeley National Laboratory, representative of HPC checkpoint/restart patterns.
- **Synthetic datasets**: Parameterized by chunk size (64 KB to 16 MB), chunk count (10^2 to 10^7), dimensionality (1D to 3D), and compression ratio, for controlled benchmarking.

6.2 Hardware Platforms

- **Workstation**: Modern x86-64 with AVX-512 and SHA-NI for hardware-accelerated hashing.
- **ARM platform**: Apple Silicon or AWS Graviton, exercising NEON SIMD and ARM SHA extensions.

6.3 Evaluation Baselines

Comparing Merkle-tree partial verification ($O(\log N)$) against full-dataset rehashing ($O(N)$) favors the Merkle tree by construction and is therefore not, on its own, a persuasive baseline. The substantive baseline—listed first below—instead measures the *closest-characteristic cryptographic primitives* on the same metrics (§6.6), datasets, and hardware, so the comparison rests on measured numbers rather than asymptotic arguments. The remaining three baselines address specific questions reviewers most often raise:

Comparable cryptographic primitives (head-to-head). Per-chunk asymmetric signatures (Ed25519, plus an ML-DSA-65 post-quantum variant), per-chunk MAC (HMAC-SHA-256), and flat whole-dataset hashing are implemented as reference backends and measured on every applicable metric of §6.6: storage overhead, commit/build cost, partial-verification and tamper-localization latency, incremental append and in-place-update cost, and—where the primitive admits it—subset coverage-proof size and public verifiability. Per-chunk signatures are the closest public-verifiable competitor (they satisfy R1, R4, R5a, and R6 but differ sharply on R7 and R10, §4.3); the goal is to quantify *by how much* they differ on real scientific data rather than only in the asymptotic table. Where a primitive structurally cannot answer a query—flat hashing cannot verify a single chunk; a MAC cannot be verified without the secret key—that is recorded as an explicit capability entry, not omitted. We are not aware of prior measurements of these primitives head-to-head at HDF5 scientific scale, so this baseline characterizes the gap in existing and comparable methods directly. The harness is task P1.2b (§7), run concurrently with the Merkle implementation.

Filesystem-level checksumming (ZFS [14], Btrfs). Addresses *"Why do this in HDF5 when my filesystem already prevents silent data corruption?"* The answer is that filesystem checksums are stripped the moment a file is transferred over a network, uploaded to cloud storage, or moved to a different filesystem. Application-level (HDF5) Merkle trees *travel with the data* and can be verified by any recipient regardless of the underlying storage.

Cloud object-store checksums (S3 ETag, GCS CRC32C). Addresses *"S3 already gives me per-object integrity."* Object stores expose a flat per-object hash and lack hierarchical structure for partial verification, sub-object tamper localization, or non-repudiable signing. The cloud baseline (§6.4) quantifies this gap by comparing proof-of-chunk retrieval over S3 range-GETs against full-object re-download.

Per-chunk AEAD only (no Merkle tree). Addresses *"Why not just AEAD every chunk and be done?"* This baseline verifies that AEAD alone cannot answer "has *any* chunk been modified?" without decrypting all chunks, and cannot localize tampering without an external authenticated structure.

6.4 Cloud Object-Store Evaluation

A growing fraction of scientific HDF5 deployments runs over cloud object stores: HSDS [71], h5coro [47], and the ros3 VFD all access HDF5 over S3-compatible APIs. The companion-dataset design maps naturally to this environment: $O(\log N)$ proof-path nodes can be retrieved via HTTP range-GETs against the companion's chunked layout, without downloading the data chunks themselves. We evaluate this end-to-end for two cloud scenarios—intra-region (LAN-class latency) and cross-region (WAN, ∼80 ms RTT)—measuring (a) bytes transferred for verifying a hyperslab via subset proof versus full re-download, (b) wall-clock latency of the proof-then-verify path, and (c) the impact of HTTP/2 multiplexing on proof-path retrieval latency. This is the first reported measurement of authenticated partial verification for HDF5 over cloud storage, and directly grounds the "portability" benefit of application-level Merkle trees in concrete numbers.

6.5 Adversarial Red-Team Study

A common failure mode of integrity-system papers is that the "tamper detection" experiment injects modifications using the system's own write path and then reports near-100% true-positive / 0% false-positive rates. This trivially confirms that the verifier rejects what the writer breaks; it does not test whether a directed adversary can evade detection. We instead conduct a *white-box red-team study* in which the adversary has full design knowledge, full read/write access to the storage medium, and attempts to mount each threat in §4.1:

T1a (storage-level tampering, data). Targeted single- and multi-chunk modification, including modifications of compressed payloads designed to remain

valid after the filter pipeline.

T1b (storage-level tampering, internal tree nodes). Modification of internal nodes in the companion dataset without altering the root, targeting the companion-integrity-hash defense (§5.5).

T2 (silent data corruption). Single-bit and burst-error injection via deliberate fault injection on a known-good chunk.

T3 (provenance forgery). Substitution of the `_provenance_pubkey` attribute alone, and key-confusion attacks on the hybrid signature.

T4 (rollback). Substitution of an entire valid prior version of the file (data, companion, root attribute), and selective per-chunk rollback where the chunk and its version counter are both restored to a prior valid state. The latter targets the version-counter binding into the leaf hash (§5.7).

T5 (post-quantum). Simulated forgery of the Ed25519 component of the hybrid signature (assuming oracle access to a CRQC), to confirm that the ML-DSA-65 component blocks acceptance.

Subset-extraction attacks. (a) Omission of a chunk from the delivered set; (b) substitution of a chunk from a different version; (c) delivery of a covering set that does not match the requested hyperslab.

For each attack class we report whether the verifier rejects, the proof-of-failure granularity (which chunk(s) were flagged), and the wall-clock detection latency. Attacks that succeed are root-cause-analyzed and the design is iterated; this is treated as a research result, not a defect.

6.6 Metrics

Every metric below is reported for the Merkle construction *and* for the comparable-primitive baselines of §6 (per-chunk signature, per-chunk MAC, flat hash) wherever the primitive supports the operation, so that each result is read relative to the alternatives rather than in isolation.

- **Throughput**: MB/s for hash computation (SHA-256, BLAKE3, KangarooTwelve) and encryption (ChaCha20-Poly1305) at varying chunk sizes.
- **Partial verification latency**: Wall-clock time to verify a single chunk via Merkle proof vs. full-dataset rehash.
- **Subset-proof size and latency**: $|\pi|$ and end-to-end `extract`/`verify` time as a function of hyperslab shape.
- **WAN bandwidth savings**: Bytes transferred for proof-of-subset over S3 range-GETs vs. full re-download.
- **Hybrid-signature cost**: Sign and verify time, signature size, public-key size, and object-header impact for Ed25519 alone vs. Ed25519 + ML-DSA-65 vs. Ed25519 + SLH-DSA-128s.
- **Incremental update cost**: Time to append k chunks or overwrite k existing chunks and update the Merkle tree vs. re-signing the entire dataset.

- **Storage overhead**: Bytes consumed by Merkle tree, companion dataset, version counters, and provenance metadata as a fraction of dataset size.
- **Adversarial-evasion rate**: Number of red-team attack classes detected vs. undetected (§6.5); for undetected attacks, the time-to-fix in the design.

6.7 Statistical Protocol

Each benchmark configuration will be measured over a minimum of 30 trials preceded by 5 warmup runs (discarded). We report median and 95th-percentile latency, with 95% confidence intervals computed via bootstrapping. To control for variance from background system activity, benchmarks will be run with CPU frequency scaling disabled (`performance` governor), on an isolated set of cores (via `taskset`), and with filesystem caches dropped between trials for I/O-sensitive measurements.

7 Implementation Plan

This section describes the work as concrete, step-by-step tasks ordered to build progressively from a working ClawHDF5 environment through the three primary research contributions. Each task specifies what to build, which Rust crate to work in, how to test the result, and a clear *Done when* acceptance criterion. Items marked **(Move N)** are publication-unit contributions targeted at the resulting paper (Move 1 = verifiable subset extraction, Move 2 = PQ hybrid signing, Move 3 = adversarial red-team study, Move 4 = security analysis, Move 5 = append-only Merkle log, Move 6 = cloud evaluation). AES-256-CTR + HMAC-SHA-256 filters and live Sigstore service integration are deferred to Phase 3 to preserve focus on the critical path.

Artifact conventions. Commit all benchmark outputs under `benches/results/`, test vectors and known-good fixtures under `test-vectors/`, attack-harness results under `attack-results/`, and written design documents under `docs/`. Every benchmark file must begin with a header comment recording the hostname, CPU model, RAM size, and date so results remain interpretable after the machine changes. Do *not* commit large HDF5 files (>100 MB); commit the script that regenerates them instead. Each task below lists its *Artifacts* explicitly—all listed items must be committed before the task is considered done.

7.1 Phase 1: Core Architecture and Verifiable Subset (Months 1–4)

Background reading before starting Phase 1: read all of §2 (HDF5 data model, chunked storage, S2-D2 framework architecture, and ClawHDF5 crate layout), then all of §5 (the full Merkle-Tree Provenance design: threat model, why integrity

is independent of encryption, tree construction, hash algorithm selection, verifiable subset extraction, storage layout, and signing), with particular attention to §5.5 (hybrid attribute + companion-dataset layout and domain-separation prefixes `0x00`/`0x01`/`0x02`).

Implementation status (baseline as of this revision). To keep the plan honest about its starting point, we record what the ClawHDF5 prototype implements today versus what remains design. The `clawhdf5-format` crate currently implements the P1.2 core: the `MerkleTree` structure, the three hash algorithms (`Sha256`/`Blake3`/`K12`), tree construction, inclusion-proof generation and verification, the domain-separation prefixes, and the depth cap. Accordingly the `MerkleError` enum presently defines a *single* variant, `TreeTooDeep`; the expanded taxonomy in P1.4 (`HashMismatch`, `CompanionTampered`, `SignatureInvalid`, `NoncePending`, ...) and the five persisted-state verification functions are the deliverable of P1.4, not yet built. Likewise, the persisted `_merkle_root` attribute and companion dataset (§5.5), the dataset version counter (§5.7), hybrid signing (§5.8), and the crash-consistency WAL (§5.5, "Crash consistency") do not yet exist in the prototype—the present codebase has only non-cryptographic checksums and per-dataset SHA-256 provenance attributes, and its sole recovery primitive is a full-file atomic snapshot copy with no link to the root it certifies. Consequently, the error-response triage and rollback-as-recovery behavior described in §5.5 ("Crash consistency") presuppose the P1.4 verification API and the versioning/signing deliverables; they describe the *target* system and are realized progressively across Phase 1 and Phase 2.

P1.1 Platform evaluation / go–no-go gate (Weeks 1–2). Before writing any Merkle-tree code, confirm that ClawHDF5 supports all the I/O patterns this work requires. If any step below fails, activate the standalone-library fallback described in §11.

1. Clone the ClawHDF5 repository and run `cargo test --workspace`. All existing tests must pass with no warnings about missing features.
2. Generate a synthetic 10 GB HDF5 file that has at least three datasets with different chunk sizes (e.g., 64 KB, 256 KB, 1 MB). Use ClawHDF5's high-level API in `clawhdf5`.
3. Write a test that opens the file, iterates over every chunk of one dataset (reading raw chunk bytes), modifies one chunk in memory, and writes it back. Re-open the file and read the modified chunk to confirm the write succeeded without corrupting adjacent chunks.
4. Repeat step 3 on a real NOAA sample file from the provided test-data archive.
5. *Done when:* All four steps above complete without panics or data-corruption failures and the modified file re-opens cleanly.
6. *Artifacts:* Commit the generator script that produces the synthetic 10 GB file to `test-vectors/gen_synthetic.rs` (or `.py`); add the NOAA sample

filename and its download URL to `test-vectors/README.md`. Do *not* commit the HDF5 files themselves.

P1.2 `MerkleTree` struct and hash benchmarks (Weeks 2–5). Implement the core tree data structure and compare three hash functions across HDF5-typical chunk sizes (RQ4). Read §5.3 for the performance–compliance tradeoffs among SHA-256, BLAKE3, and K12.

1. In `clawhdf5-format/src/merkle.rs`, add:

```
pub enum HashAlg { Sha256, Blake3, K12 }

pub struct MerkleTree {
    pub leaves:     Vec<[u8; 32]>,  // leaf hashes, left to right
    pub nodes:      Vec<[u8; 32]>,  // complete binary tree, level-order
    pub root:       [u8; 32],
    pub alg:        HashAlg,
    pub padded_n:   usize,          // next power of 2 >= leaves.len()
    pub actual_n:   usize,          // true leaf count (chunks in dataset)
}
```

2. Implement `MerkleTree::build(leaves: &[[u8;32]], alg: HashAlg) -> Self`. **Tree shape:** scientific datasets rarely have power-of-2 chunk counts. Pad the leaf array with the computed null sentinel $H(\texttt{0x02} \,\|\, \texttt{null})$ (as defined in §5.5) up to the next power of 2 before building. This keeps the level-order index arithmetic (left $= 2i + 1$, right $= 2i + 2$) correct for all N. Store the actual leaf count in `actual_n` and the padded count in `padded_n`; proof paths must use `padded_n` for index arithmetic but must not deliver null-padded leaves to the verifier. **Maximum depth:** enforce $\lceil \log_2 \texttt{padded\_n} \rceil \leq 40$ (supporting up to $2^{40} \approx 1$ Trillion chunks); return `Err(MerkleError::TreeTooDeep)` for larger inputs to prevent out-of-memory attacks (threat T7 in §4.1). Apply domain-separation prefixes exactly as specified in §5.5: prepend `0x00` before hashing leaf inputs, `0x01` before hashing pairs of internal nodes, and `0x02` for unallocated sparse-chunk slots.
3. Add a helper `hash_chunk(data: &[u8], alg: HashAlg) -> [u8;32]` that computes the leaf hash (with the `0x00` prefix) over raw chunk bytes. **BLAKE3 API note:** use `Hasher::new().update(&[0x00]).update(data).finalize()` (the flat hash API); do *not* use BLAKE3's internal tree mode (`new_derive_key` or implicit 1 KB chunking), which does not expose intermediate nodes and cannot produce per-chunk proofs. BLAKE3 tree mode is benchmarked for throughput in RQ4 only.
4. Write unit tests covering at minimum:

- a single-leaf tree (the root must equal the leaf hash),
- a two-leaf tree (verify the root by hand),
- an eight-leaf tree whose expected root is pre-computed externally using a reference implementation.

5. Add a `cargo bench` microbenchmark comparing SHA-256, BLAKE3, and K12 at chunk sizes 64 KB, 256 KB, and 1 MB. Save results to `benches/results/hash-bench-$(hostname).txt` with CPU model and RAM size in the header.
6. *Done when:* All unit tests pass; benchmark output is committed.
7. *Artifacts:* `benches/results/hash-bench-$(hostname).csv` (step 5; save as CSV in addition to or instead of the `.txt` output so it can be parsed by a plotting script). Required columns: `alg` (`sha256`/`blake3`/`k12`), `chunk_size_kb` (64/256/1024), `throughput_mbs` (median over 30 trials), `ci95_low`, `ci95_high`, `platform` (`x86`/`arm`), `hostname`, `cpu_model`. **Feeds Table 3 (RQ4, §7.4):** present as a grid with rows = chunk size and columns = alg × platform; each cell reports median MB/s ± 95% CI. Also commit `test-vectors/merkle-vectors.json` containing the pre-computed expected roots for the single-leaf, two-leaf, and eight-leaf unit-test cases.

P1.2b Comparable-primitive baseline harness (Weeks 3–6, concurrent). *Dependency:* requires only the chunk-iteration path validated in P1.1 and the `hash_chunk` helper from P1.2 (step 3); it runs in parallel with P1.3/P1.3b. This task establishes the empirical baseline against which all Phase 1 Merkle results are reported (RQ1–RQ3, §5.10), so that existing and comparable methods are *measured* on the same data, software, and hardware rather than asserted to be worse. It directly answers the reviewer question of whether the metrics of §6.6 have been characterized for the primitives nearest the Merkle tree in Table 1.

1. In a new module `clawhdf5-format/src/baselines.rs`, implement three reference integrity backends over the same chunk-iteration path used by the Merkle code: (a) `flat_hash`—a single SHA-256 over the whole dataset (ClawHDF5's current provenance feature, the status quo); (b) `per_chunk_mac`—keyed HMAC-SHA-256 per chunk; (c) `per_chunk_sig`—Ed25519 per chunk via `ed25519-dalek` (an ML-DSA-65 variant is added once the P2.1 signing crate lands in Phase 2). Each backend exposes `commit`, `verify_chunk`, `verify_dataset`, `append`, and `update`, mirroring the Merkle verification API so a single harness can drive all backends identically.
2. On the 10 GB synthetic file from P1.1 and a real NOAA dataset, measure every applicable metric of §6.6 for each backend: bytes of integrity metadata (storage overhead), commit/build wall time, single-chunk verification latency, whole-dataset verification latency, append and in-place-update cost, and (for `per_chunk_sig`) the bytes a recipient must receive to verify a k-chunk hyperslab. Record operations a backend *cannot* perform (flat hash: no single-

chunk verification; MAC: no verification without the secret key) as an explicit `supported=false` capability entry, not a blank.

3. For `per_chunk_sig`, project storage and signing cost to $N = 10^7$ chunks from the measured per-chunk costs and confirm that the ∼640 MB (Ed25519) / ∼33 GB (ML-DSA-65) storage and ∼3-CPU-hour signing figures cited in §4.3 hold on the actual hardware (or report the corrected numbers).
4. *Done when:* all three backends commit and verify correctly on the synthetic and NOAA files; the baseline CSV is committed; and the capability matrix (backend × metric, with explicit "N/A—no public key" / "N/A—no partial access" entries) is filled in.
5. *Artifacts:* `benches/results/baselines-$(hostname).csv`. Required columns: `backend` (`flat`/`mac`/`sig_ed25519`/`sig_mldsa`), `metric`, `n_chunks`, `chunk_size_kb`, `value`, `unit`, `supported` (`true`/`false`), `source` (`synthetic`/`NOAA`), `trial` (1–30), `hostname`, `cpu_model`. **Feeds §7.1–§7.3 (RQ1–RQ3):** every Merkle figure and table in those subsections gains a baseline series, so the reader sees the Merkle tree *relative to* per-chunk signatures, per-chunk MAC, and flat hashing rather than in isolation.

P1.3 Hybrid storage layout (Weeks 4–6). *Dependency:* P1.3 can begin as soon as P1.2 steps 1–3 (the `MerkleTree` struct and domain-separation logic) are complete, which should happen by the end of Week 3. P1.2's benchmark runs (step 5) may proceed in parallel with P1.3.

Persist the Merkle tree alongside its dataset in the HDF5 file using the two-tier layout defined in §5.5: a small root-hash HDF5 attribute for fast checks, plus a companion dataset for the full node array when $N > 256$ chunks.

1. In `clawhdf5-format`, implement `write_merkle_attr(dataset: &Dataset, tree: &MerkleTree)`. Pack the root hash (32 bytes), algorithm identifier (1 byte), and companion-integrity hash (32 bytes, see step 3) into a fixed-width binary blob and write it as the HDF5 attribute `_merkle_root` on the dataset.
2. Implement `write_merkle_companion(file: &File, name: &str, tree: &MerkleTree)`. For datasets with more than 256 chunks, write the complete `nodes` array as a flat `u8` dataset at path `/_merkle/{name}`. For datasets with 256 or fewer chunks, pack all node hashes into the attribute directly (no companion dataset).
3. Compute the companion-integrity hash: after writing the companion dataset, compute SHA-256 over its full byte content and store the result in the `_merkle_root` attribute. This single extra hash catches any direct tampering with the companion dataset before the tree is walked.
4. Write a round-trip test: create a 1,024-chunk synthetic dataset, call the write functions, close the file, re-open it, read back the attribute and companion, and verify the companion-integrity hash matches the recomputed value.
5. Confirm the companion dataset is visible using `h5dump -n` on the test file and

that no existing ClawHDF5 tests regress (`cargo test --workspace`).

6. *Done when:* Round-trip test passes; `h5dump` shows the expected paths; no regressions.
7. *Artifacts:* The round-trip test committed as a named `#[test]` function (CI re-runs it on every commit); the `h5dump -n` output from step 5 saved to `test-vectors/p1.3-h5dump.txt` as a reference for the expected on-disk layout.

P1.3b Parallel leaf-hashing and Map-Reduce aggregation (Weeks 5–7). *Dependency:* requires P1.2 (tree struct) and P1.3 (storage layout). The write-amplification analysis (§5.5) and R11 (distributed parallel construction) both require that leaf hashing is embarrassingly parallel. Implement the Map-Reduce aggregator described in §5.5.

1. Add Cargo dependency `rayon` to `clawhdf5-format`.
2. Implement `MerkleTree::build_parallel(chunks: &[&[u8]], alg: HashAlg) -> Self`:

```
// Map phase: hash all leaves in parallel
let leaf_hashes: Vec<[u8; 32]> = chunks
    .par_iter()
    .map(|c| hash_chunk(c, alg))
    .collect();
// Reduce phase: single-threaded tree aggregation
MerkleTree::build(&leaf_hashes, alg)
```

   The reduce phase is sequential because each internal node depends on its two children; the contention is at the upper tree levels, not the leaves.
3. For the MPI-IO scenario (C HDF5 Phase 3), document the equivalent distributed protocol in `docs/mpi-protocol.md`. Cover two complementary strategies:
   - **Eager collective flush.** Each MPI rank computes leaf hashes for its own chunk subset; ranks perform an `MPI_Reduce` tree over the leaf-hash arrays using $\lceil \log_2 P \rceil$ rounds; the root rank aggregates the full tree and writes the companion dataset in a single collective I/O pass. No global lock is needed because each rank owns a disjoint leaf range.
   - **Lazy in-memory tree maintenance.** Each rank accumulates its leaf hashes in memory without touching the companion dataset during the write epoch. At epoch close (file flush or explicit barrier), a single collective I/O pass writes the full tree delta. This eliminates per-chunk companion I/O and the upper-level contention hotspots identified in §5.5, at the cost of holding the in-progress tree delta in memory across the epoch. It is the recommended strategy for high-concurrency checkpoint workloads.

4. Write a correctness test: verify that `build_parallel` and `build` produce identical roots for a 1,024-chunk synthetic dataset on at least 4 rayon threads.
5. Benchmark `build_parallel` vs. `build` at $N \in \{10^4, 10^5, 10^6\}$ on the workstation. Save to `benches/results/parallel-build-$(hostname).csv`.
6. *Done when:* Correctness test passes; benchmark is committed.
7. *Artifacts:* `benches/results/parallel-build-$(hostname).csv` (step 5). Required columns: `mode` (`sequential`/`parallel`), `n_chunks` (10^4/10^5/10^6), `wall_time_ms`, `n_threads`, `trial` (1–30), `hostname`. **Presentation:** compute the speedup ratio (sequential / parallel wall time) for each N and report it as a two-column table in the §7 narrative; this data does not merit a standalone figure but should appear as a paragraph with a small table. Commit the full MPI-IO protocol description from step 3 (both eager and lazy variants) to `docs/mpi-protocol.md`.

P1.4 Verification API (Weeks 7–10). Expose the `MerkleError` type and five public verification functions in `clawhdf5-format/src/merkle.rs`. Define the error enum first:

```
pub enum MerkleError {
    HashMismatch { chunk_idx: usize },  // leaf hash != recomputed value
    CompanionTampered,                  // companion-integrity hash mismatch
    SignatureInvalid,                   // hybrid sig verification failed
    MissingChunkGridMetadata,           // _merkle_root attribute absent
    HyperslabOutOfBounds { idx: usize },// chunk index exceeds dataset
    TreeTooDeep { depth: usize },       // depth > 40, reject to prevent OOM
    NoncePending,                       // WAL has uncommitted entry for chunk
}
```

Then expose the five public verification functions:

```
pub fn verify_root(d: &Dataset) -> Result<bool, MerkleError>;
pub fn verify_chunk(d: &Dataset, idx: usize) -> Result<bool, MerkleError>;
pub fn verify_dataset(d: &Dataset) -> Result<bool, MerkleError>;
pub fn extend_merkle(d: &Dataset, idx: usize, h: [u8;32]) -> Result<(), MerkleError>;
pub fn update_merkle(d: &Dataset, idx: usize, h: [u8;32]) -> Result<(), MerkleError>;
```

Also define a minimal *response policy* so that callers have a defined default behavior on a detected error, rather than each call site inventing its own (the response side of §5.5, "Crash consistency"):

```
pub enum VerifyResponse { Halt, Quarantine, Alert }

// Default policy: fail closed. Maps every error variant to a response.
// Rollback-based recovery (restore_to_version) is deferred to P2.2b;
// in Phase 1 the default is always Halt.
pub fn default_response(e: &MerkleError) -> VerifyResponse;
```

1. `verify_root`: reads the `_merkle_root` attribute, re-hashes the companion dataset bytes, and compares against the stored companion-integrity hash. Sub-second; does *not* re-hash chunk data—only detects tampering with the companion dataset.
2. `verify_chunk`: re-hashes the live chunk at index `idx`, reads the $O(\log N)$ sibling hashes from the companion dataset, and walks the proof path to the root, comparing the recomputed root to the stored value. If the WAL contains an uncommitted entry for `idx`, returns `Err(MerkleError::NoncePending)` immediately (the chunk may be partially written; the caller must replay or abort before verifying).
3. `verify_dataset`: re-hashes every chunk, rebuilds the tree from scratch, and compares the computed root to the stored root. This is the full $O(N)$ integrity check.
4. `extend_merkle`: appends a new leaf hash `h` at position `idx` and recomputes only the $O(\log N)$ nodes on the path to the root; leaves all other nodes unchanged.
5. `update_merkle`: same path recomputation as `extend_merkle` but for an in-place overwrite of an existing chunk.
6. `default_response`: return `VerifyResponse::Halt` for every variant in Phase 1 (fail closed). This is the seam that P2.2b replaces with the full halt/quarantine/alert decision and rollback recovery; defining it here ensures every verification call site has a single defined response path from the start, and that the variant is logged (file path, dataset, chunk index, variant) before halting.
7. Write tests using a 1,024-chunk synthetic dataset. Tamper one byte in one chunk on disk (using raw file I/O, bypassing ClawHDF5). Confirm: `verify_chunk` returns `Err(MerkleError::HashMismatch)` for the tampered chunk; `verify_dataset` also returns an error; `verify_root` returns `Ok(true)` (correct—it only checks the companion, not chunk data).
8. *Done when:* All five functions compile and pass tests; the tamper-detection test produces exactly the expected error values; and `default_response` re-

turns `Halt` for every variant.

9. *Artifacts:* The tamper-detection assertions from step 6 committed as a named `#[test]` that encodes all five expected `MerkleError` variants explicitly—this is the regression fixture for the verification API. Include an assertion that `default_response` maps each variant to `Halt`, so the P2.2b change to this policy is caught by a failing test. No output files beyond the code itself.

P1.5⋆ Verifiable subset extraction (Weeks 8–13, Move 1). This is the first primary research contribution. A recipient of a hyperslab must be able to prove cryptographically that the delivered chunks are a *complete and correct* subset of the signed parent dataset—not just that each individual chunk is intact. Read §5.4 thoroughly before starting; the coverage certificate is what distinguishes this from a plain Merkle inclusion proof.

1. In `clawhdf5-format/src/subset_proof.rs`, define:

```
pub struct ChunkGridParams {
    pub dims:        Vec<u64>,   // dataset dimensions
    pub chunk_shape: Vec<u64>,   // chunk dimensions
    pub grid_hash:   [u8; 32],   // H(dims || chunk_shape)
}

pub struct SubsetProof {
    pub chunk_indices:      Vec<usize>,
    pub leaf_hashes:        Vec<[u8; 32]>,
    // Deduplicated proof nodes keyed by 1D level-order tree index.
    // Using BTreeMap (not Vec<Vec<...>>) because after deduplication
    // adjacent proof paths share internal nodes; a flat nested Vec cannot
    // express which node sits at which tree position, breaking path
    // reconstruction at the verifier.  Key = level-order index in the
    // padded binary tree (root = 0, left child of i = 2i+1, right = 2i+2).
    pub proof_nodes:        std::collections::BTreeMap<u64, [u8; 32]>,
    pub grid_params:        ChunkGridParams,
    pub coverage_cert:      [u8; 32],  // H(sorted_indices || grid_hash)
}
```

2. Implement `extract_subset(dataset: &Dataset, hyperslab: &Hyperslab)` returning `Result<SubsetProof, MerkleError>`. The

function must support two leaf-linearization orderings selectable at build time via a `LeafOrder` enum (`RowMajor` and `Morton`), so that RQ6 can be answered empirically by running the same benchmark under both:

- **Row-major:** 1D leaf index $= \sum_i c_i \prod_{j>i} n_j$, where c_i is the chunk coordinate on axis i and n_j is the chunk count on axis j.
- **Morton (Z-order):** implement a helper `fn morton_index(coords: &[u64]) -> u64` that bit-interleaves the per-axis chunk coordinates: MortonIndex(x, y, z) = BitInterleave(x, y, z) (expand each coordinate's bits across the alternating bit positions of the output word). A correct 3D implementation passes the test vector MortonIndex$(1, 2, 3) = 0x15$ (binary `000010101`). Commit the test vector in `test-vectors/morton-vectors.json`.

Translate the hyperslab into a sorted list of chunk indices under the selected ordering; compute each leaf's proof path as a set of level-order tree indices; take the union of all path index sets and populate `proof_nodes` with one entry per unique index—this is the deduplication step that keeps $|\pi|$ compact for contiguous hyperslabs. Build the coverage certificate by hashing the sorted index list together with the grid-parameter hash. The verifier reconstructs each path by looking up the required nodes from `proof_nodes` using level-order index arithmetic, so every node that appears in more than one proof path is transmitted only once.

3. Implement `verify_subset(root: [u8;32], chunks: &[ChunkData], proof: &SubsetProof) -> Result<bool, MerkleError>`. On the recipient side: re-hash each delivered chunk, walk its sibling path to confirm it is included under `root`, and verify the coverage certificate confirms no requested chunk is missing or substituted.
4. Measure proof size $|\pi|$ (bytes) for three hyperslab shapes on a dataset with $N = 65{,}536$ chunks:
   - a contiguous rectangular slab (best case, many shared nodes),
   - a strided slab (fewer shared nodes),
   - a random sparse selection of k chunks (near-worst case).

   Record results and compare against the theoretical bound $O(k \log N)$ (RQ6). Save the table to `benches/results/subset-proof-size.csv`.
5. *Done when:* `verify_subset` returns `Ok(true)` for a correct proof; returns `Err` when one delivered chunk is modified; returns `Err` when one requested chunk is silently omitted from the delivered set. All three failure modes must be covered by explicit tests.
6. *Artifacts:* `benches/results/subset-proof-size.csv` (step 4). Required columns: `hyperslab_type` (`contiguous`/`strided`/`random`), `k_chunks` (chunks in the subset), `n_total` (65536), `proof_size_bytes`, `proof_time_ms`, `verify_time_ms`, `theoretical_bound_bytes` ($= k \cdot \lceil \log_2 N \rceil \cdot 32$, computed and filled in by the benchmark). **Feeds Figure 6 left panel (RQ6,**

§7.6): log-log line chart with k on the x-axis and `proof_size_bytes` on the y-axis; three series (one per hyperslab type) plus a dashed line for `theoretical_bound_bytes`—the contiguous series should fall well below the bound due to shared proof nodes. Also commit `test-vectors/subset-vectors.json` with three serialised `SubsetProof` structs (one per hyperslab shape) with their expected roots.

P1.6 Phase 1 benchmarks (Weeks 13–16). Compare Merkle-tree overhead *against the P1.2b comparable-primitive baselines* (flat SHA-256, per-chunk MAC, and per-chunk Ed25519 signature), not only against ClawHDF5's existing flat verification, to answer RQ1, RQ2, and RQ3 comparatively. Reuse the P1.2b harness so every scenario below is run for each backend that supports it.

1. Using the 10 GB synthetic file from P1.1, run and record wall-clock time and peak RSS for each of the following scenarios, for the Merkle tree and for each P1.2b baseline that supports the operation:
   - `verify_dataset` (full Merkle re-hash) vs. existing flat `verify_dataset()` vs. per-chunk-signature whole-dataset verification on the same file,
   - `verify_chunk` for a single chunk: Merkle proof vs. per-chunk-signature verification (flat hashing cannot do this—record as unsupported),
   - companion dataset I/O cost vs. attribute-only storage at chunk counts $N \in \{16, 256, 1{,}024, 65{,}536\}$.
2. Repeat on a real NOAA dataset.
3. Save all raw numbers, the hostname, CPU model, and disk type to `benches/results/phase1-$(hostname).csv` and commit the file.
4. *Done when:* CSV files are committed; numbers are ready to paste into the paper's evaluation tables (§6).
5. *Artifacts:* `benches/results/phase1-$(hostname).csv` (step 3). Required columns: `source` (`synthetic`/`NOAA`), `scenario` (`verify_dataset`/`verify_chunk`/`flat_verify`/`compa`
   `n_chunks`, `chunk_size_kb`, `wall_time_ms`, `peak_rss_mb`, `companion_bytes`, `trial` (1–30), `hostname`, `disk_type`. **Feeds three paper items:**
   - **Figure 4 (RQ2, §7.2):** horizontal bar chart; filter to `scenario` ∈ {`verify_chunk`, `verify_dataset`, `flat_verify`}; x-axis = latency (ms), bar groups = scenario, bar fill = source.
   - **Figure 5 (RQ3, §7.3):** line chart; filter to `scenario` ∈ {`extend_merkle`, `update_merkle`, `full_rebuild`}; x-axis = N (log scale), y-axis = wall time (log scale), one line per scenario.
   - **Table 1 (RQ1, §7.1):** filter to `scenario = companion_io`; rows = N, columns = `companion_bytes` and fraction of dataset size.

7.2 Phase 2: Security Contributions and Evaluation (Months 4–7)

Background reading before starting Phase 2: read §5.8 (Signing Authority and Key Management), paying particular attention to the "Post-quantum hybrid signing"

paragraph (§5.8) for the hybrid Ed25519 + ML-DSA-65 construction and payload encoding; then §5.6 (Encrypt-then-MAC filter pipeline ordering and why hash-then-encrypt is insecure), §5.7 (version-counter nonce derivation and why it prevents nonce reuse on in-place updates), and §5.9 (the game-based argument template, including the three component security assumptions).

P2.1⋆ Hybrid Ed25519 + ML-DSA-65 signing (Weeks 1–4 of Phase 2, Move 2). This is the second primary research contribution: a post-quantum-safe signing scheme that attaches a non-repudiable author identity to each Merkle root and survives the 50-year archival timescales typical of scientific data.

1. Create a `clawhdf5-sign` crate (or module inside `clawhdf5-format`). Add Cargo dependencies: `ed25519-dalek` and either `ml-dsa` or `pqcrypto-mldsa`.
2. Implement the canonical payload encoder (see §5.8 for field definitions):

```
fn canonical_payload(
    root:           &[u8; 32],
    companion_hash: &[u8; 32],
    version:        u64,        // dataset version
       counter
    timestamp:      u64,        // Unix seconds
    alg_id:         HashAlg,
) -> Vec<u8>
// Returns: big-endian fixed-width binary encoding, no
   separators.
```

3. Implement `sign_root(payload: &[u8], ed_key: &Ed25519SigningKey, ml_key: &MlDsaPrivateKey) -> HybridSignature`. Produce both signatures independently over the same payload and store them concatenated with a 1-byte version tag in the HDF5 attribute `_merkle_sig` on the dataset.
4. Implement `verify_sig(payload: &[u8], sig: &HybridSignature, ed_pub: &Ed25519VerifyingKey, ml_pub: &MlDsaPublicKey) -> Result<(), SigError>`. **Require both signatures to be valid**; accept the payload only if neither fails.
5. Measure sign and verify latency on x86-64 (with and without AVX-512) and on ARM. Vary dataset size from 10^4 to 10^7 chunks. Record signature size, public-key size, and object-header size impact (RQ7). Save to `benches/results/signing-$(hostname).csv`.
6. *Done when:* Round-trip sign/verify test passes; a test with one byte of the payload modified returns `Err(SigError::InvalidSignature)`; benchmarks are committed.
7. *Artifacts:* `benches/results/signing-$(hostname).csv` (step 5). Required columns: `scheme` (`ed25519`/`hybrid_mldsa65`/`hybrid_slhdsa128s`), `platform` (`x86_avx512`/`x86_noavx`/`arm`), `n_datasets` (10^4 to 10^7), `sign_ms`, `verify_ms`, `sig_size_bytes`, `pubkey_size_bytes`, `header_overhead_bytes`,

`trial` (1–30). **Feeds Table 4 (RQ7, §7.7):** rows = scheme, columns = platform; each cell reports median sign ms, verify ms, sig size, and pubkey size with 95% CIs on the timing columns. Also commit `test-vectors/hybrid-sig-vectors.json` with one known-good `HybridSignature` and its canonical payload (using a test-only key pair committed alongside it).

P2.2 ChaCha20-Poly1305 AEAD filter and nonce derivation (Weeks 2–5 of Phase 2). Add the encryption layer to `clawhdf5-filters` and implement the version-counter nonce derivation that makes in-place chunk updates safe. Read §5.6 and §5.7 before starting.

1. **Key management scope for P2.2.** ClawHDF5 does not yet have key management infrastructure; Task 2 of S2-D2 will provide the production KeyStore. For this prototype phase, implement a `MockKeyStore` that derives a static 32-byte DEK from the environment variable `CLAW_DUMMY_DEK` (hex-encoded):

```
fn mock_dek() -> [u8; 32] {
    let hex = std::env::var("CLAW_DUMMY_DEK")
        .expect("set CLAW_DUMMY_DEK for prototype");
    hex::decode(&hex).unwrap().try_into().unwrap()
}
```

   Do *not* attempt to build HKDF/KEK wrapping or multi-party key distribution in P2.2; that is deferred to Phase 3. All tests must set `CLAW_DUMMY_DEK` in the test environment (add it to `.cargo/config.toml` under `[env]`).
2. Add a `chacha20_filter` module to `clawhdf5-filters` with Cargo dependency `chacha20poly1305`.
3. Implement version-counter nonce derivation. The nonce for encrypting chunk i at write number v is: $nonce$ = BLAKE3-derive(DEK, $chunk\_idx \,\|\, v_{\text{chunk}}$). The per-chunk write counter v_{chunk} is stored as a `u64` in the companion dataset alongside each leaf hash. **Use the WAL protocol from §5.7**: journal $(chunk\_idx, v_{\text{new}})$ to a crash-durable log *before* deriving the nonce; write the chunk using v_{new}; then update the companion dataset and mark the journal record committed. Never increment v_{chunk} directly in the companion dataset before writing the chunk, and never reuse the old v_{chunk} after a crash. A nonce must never be reused under the same DEK.
4. Enforce the Encrypt-then-MAC pipeline order for filters: compress → shuffle → encrypt. The Merkle leaf hash must be computed over the ciphertext output of the filter chain, not over the plaintext (§5.6).
5. Bind the AEAD authentication tag and the version counter into the Merkle leaf: $\text{leaf}[i] = H(\texttt{0x00} \,\|\, \text{ciphertext} \,\|\, \text{tag} \,\|\, v_{\text{chunk}})$.
6. Write an end-to-end encrypted test: write a 256-chunk encrypted dataset, verify the Merkle tree, overwrite chunk 7 (which must increment $v_{\text{chunk}}[7]$ and update the leaf hash), then call `verify_dataset` and confirm the tree is still

consistent.

7. *Done when:* The end-to-end test passes. Add a short code comment proving nonce reuse is impossible by construction ("nonce reuse requires v_{chunk} rollback, which is prevented by ...").
8. *Artifacts:* The end-to-end encrypted test committed as a named `#[test]`; ensure `CLAW_DUMMY_DEK` is wired into `.cargo/config.toml` under `[env]` so the test runs in CI without manual setup. No output files beyond the code itself.

P2.2b Crash-consistency recovery and signed-root rollback (Weeks 3–6 of Phase 2). This task implements, in the Rust prototype, the recovery behavior specified in §5.5 ("Crash consistency")—the verifier-side counterpart to the signing (P2.1) and version-counter (P2.2) machinery. Until this task, the prototype can *detect* tampering but has no defined response or recovery path: rolling back to the last known-good signed version is the primary recovery mechanism, and it requires persisted version history that does not yet exist. Read §5.5 ("Crash consistency"), §5.7, and the T4 rollback entry in §4.1 before starting.

1. **Persist the version counter.** Store the dataset version as a `_merkle_version: u64` attribute, bumped on each committed mutation and bound into the signed payload (already a field of `canonical_payload`, P2.1). On open, a verifier rejects a file whose version is lower than the highest it has previously observed (T4).
2. **Enforce the three-step write order** from §5.5 in the prototype: chunk data → companion Merkle nodes → root attribute (root hash, companion-integrity hash, version), with explicit syncs between steps. Reuse the per-chunk WAL from P2.2 to carry the uncommitted `(chunk_idx, version)` record; `verify_chunk` returns `MerkleError::NoncePending` for a chunk with an uncommitted WAL entry (P1.4).
3. **Append-only provenance journal.** Add a journal (e.g. a `/_merkle/journal` dataset or a sidecar) recording one record per commit: `(version, signed_root, hybrid_sig, timestamp, snapshot_ref)`. Tie the existing full-file snapshot primitive (`clawhdf5-agent snapshot_file`) to the certified root so each snapshot is linked to the exact version and signature it certifies—a snapshot with no journaled root is not a valid rollback target.
4. **Implement `restore_to_version(v)`**: revert to the journaled snapshot for version v, then re-run `verify_dataset` and hybrid signature verification against the journaled signed root before the restored state is accepted. The default target is the highest version whose signature verifies ("last known-good signed version").
5. **Wire the error-response decision** from §5.5 into the calling path by replacing the Phase 1 `default_response` stub (P1.4, which always returns `Halt`) with the full policy: on `HashMismatch`/`CompanionTampered`/`SignatureInvalid`,

the caller logs the file path, dataset, chunk index, and variant, then follows the halt / quarantine / alert policy. For *signed* datasets the runtime must fail closed (never auto-rehash and re-sign on-disk data); for *unsigned* datasets it may rebuild by rehashing.

6. **Crash-vs-tamper test matrix.** Write tests that exercise each cause and confirm the expected variant and recovery: (a) kill the process between write-order steps 2 and 3, reopen, confirm the inconsistency is detected and that unsigned datasets rebuild while signed datasets fail closed; (b) flip one byte in one chunk → `HashMismatch`; (c) corrupt a companion node → `CompanionTampered`; (d) present a stale signature → `SignatureInvalid`; (e) roll a chunk and its version counter back to a prior valid state → rejected via the version-counter binding (T4 selective rollback). In each tampering case, confirm `restore_to_version` returns the file to a state that passes full verification.
7. *Done when:* The test matrix in step 6 passes, each case producing exactly the expected `MerkleError` variant (or clean recovery), and `restore_to_version` round-trips a tampered file back to a verifying state.
8. *Artifacts:* The crash-vs-tamper test matrix committed as named `#[test]`s (the regression fixture for recovery behavior); a short `docs/recovery-protocol.md` documenting the write order, journal record format, and the halt/quarantine/alert policy. No large output files.

P2.3 Game-based security argument (Weeks 4–7 of Phase 2, Move 4). This is a written deliverable, not a code task. Its goal is to give a rigorous argument that breaking the full scheme requires breaking one of its three components (AEAD, Merkle tree, or hybrid signature). Read §5.9 for the template.

1. Write the INT-CTXT (integrity under chosen-ciphertext attack) security game for the combined EtM + version-counter + hybrid-signed-root construction in the style of Bellare and Namprempre.
2. State the three component security assumptions: IND-CCA2 / INT-CTXT for ChaCha20-Poly1305, collision resistance for the Merkle hash, and CMA (chosen-message attack) security for Ed25519 and ML-DSA-65.
3. Give the reduction argument: an adversary who forges an integrity proof for a modified file must either (a) find a hash collision, (b) decrypt and re-encrypt a chunk without the DEK, or (c) forge a signature without the private key.
4. Write the argument as a self-contained appendix section in the paper draft and request a senior-researcher review before finalizing.
5. *Done when:* The section is reviewed and the paper's §5.9 is complete.
6. *Artifacts:* The security-argument text committed to the paper draft (inline in `Merkle-tree-HDF5.tex` or as a separate `paper/security-analysis.tex`); reviewer comments and your responses saved to `docs/security-review-notes.md` so the review history is auditable.

P2.4⋆ Adversarial red-team study (Weeks 5–9 of Phase 2, Move 3). *Dependency:* P2.1 (signing) must be complete before starting (done by end of Week 4). P2.2 (AEAD filter) finishes at end of Week 5; attacks involving the encryption layer (T1 compressed-payload attacks, T4 nonce-reuse attempts) should be deferred to Week 6 if P2.2 is not yet merged, while plaintext-dataset attacks (T2, T3, T5–T8) can begin at Week 5.

This is the third primary research contribution. Attempt to break the implementation on real scientific datasets using the eight threat classes defined in §4.1 (T1–T8).

1. Add an `attack-harness` binary to the repository. Each attack opens a test HDF5 file, performs one targeted modification using raw file I/O (bypassing ClawHDF5), then calls the verification API and asserts the expected result.
2. Implement and run each attack class on the NOAA dataset:
   - **T1** (storage tampering): overwrite raw chunk bytes using a hex editor; `verify_chunk` must detect it.
   - **T2** (silent corruption): flip one bit in the companion dataset; `verify_root` must detect it via the companion-integrity hash.
   - **T3** (root substitution): replace `_merkle_root` with a freshly computed root for the tampered file; `verify_sig` must reject it (no valid signature on the new root).
   - **T4** (replay): copy the `_merkle_sig` from an older file version onto the current file; the version counter in the canonical payload must cause rejection.
   - **T5**–**T8**: metadata tampering, missing-chunk substitution, and subset-extraction attacks as defined in §6.5.
3. Repeat the full harness on the EQSIM dataset.
4. Record the detected/undetected result for each attack on each dataset in `attack-results/matrix.csv`. Any *undetected* result requires either a code fix or a documented limitation explaining why the attack is out of scope.
5. *Done when:* All T1–T8 attacks are detected on both datasets; the matrix CSV is committed with zero unresolved *undetected* entries.
6. *Artifacts:* `attack-results/matrix.csv`. Required columns: `threat_class` (T1–T8), `attack_id` (e.g. `T1a`/`T1b`), `dataset` (`NOAA`/`EQSIM`), `detected` (`yes`/`no`), `verifier_fn` (which API call caught it), `latency_ms`, `root_cause` (filled in only for `detected = no`). **Feeds Table 5 (RQ5 + RQ8, §7.5 + §7.8):** present as a matrix with rows = threat class and columns = dataset; each cell shows ✓ + latency (ms) for detected attacks or × + root cause for any undetected ones. The `attack-harness` binary source must be committed to the repository so every attack is reproducible from a single `cargo run` command.

P2.5 Cloud / WAN evaluation (Weeks 8–11 of Phase 2, Move 6). Measure proof-path retrieval latency over S3 range-GETs. Read §6.4 for the full measurement protocol.

1. Wire the companion-dataset proof retrieval into the S3 range-GET path already present in `clawhdf5-io`. A single proof path for one chunk requires only $O(\log N)$ 32-byte node reads; issue them as one range request covering the relevant slice of the companion dataset.
2. Upload representative test files (1 GB and a larger surrogate for 1 TB-scale behavior) to the S3 test bucket.
3. Run the intra-region and cross-region WAN measurements from §6.4: record first-chunk proof latency, full-dataset verify latency, and total bytes transferred per verification call.
4. Save results to `benches/results/phase2-cloud.csv` and commit.
5. *Done when:* CSV is committed; numbers are ready for the paper's cloud-evaluation table.
6. *Artifacts:* `benches/results/phase2-cloud.csv` (step 4). Required columns: `method` (`merkle_range_get`/`full_redownload`), `measurement` (`first_chunk_proof_latency_ms`/`full_verify_latency_ms`/`bytes_transferred`), `dataset_size_gb`, `n_chunks`, `region_pair` (e.g. `us-east-1_to_eu-west-1`), `value`, `trial` (1–30). Record the S3 bucket name, region pairs, and file sizes in a header comment at the top of the file. **Feeds Figure 6 right panel (RQ6, §7.6):** grouped bar chart comparing `bytes_transferred` for `merkle_range_get` vs. `full_redownload` per dataset size; the bandwidth-savings ratio is the headline number for this figure.

P2.6 C HDF5 integration design document (Weeks 10–12 of Phase 2). A technical specification describing how the Merkle-tree storage format and verification API would be added to the official C HDF5 library.

1. Describe the on-disk layout: root-hash attribute (`_merkle_root`) plus companion dataset (`/_merkle/{name}`), with attribute-only fallback for $N \leq 256$ chunks (§5.5).
2. Specify proposed new public API functions: `H5Dverify_chunk`, `H5Dextract_subset`, and `H5Fget_integrity_root`, with C function signatures and error-code semantics.
3. Specify companion-dataset preservation semantics for `h5copy`, `h5repack`, and hyperslab-copy operations (the companion must be updated atomically with the data).
4. Specify the crash-consistency write order: chunks → companion dataset → root attribute. Writes in the reverse order must never be generated; document how each API call enforces this. Because the C HDF5 Metadata Cache (MDC) flushes pages heuristically, API call order alone does not guarantee on-disk ordering. The specification must choose between two enforcement mechanisms

and document the trade-offs:

- `H5Fflush` **barriers:** call `H5Fflush(fid, H5F_SCOPE_LOCAL)` after each step to drain the MDC to storage. Simple to implement and portable, but introduces additional I/O round trips and prevents MDC coalescing across steps.
- **MDC flush dependencies:** register parent–child flush dependency relationships (via HDF5's internal `H5AC_create_flush_dependency` API) so the MDC guarantees companion-dataset pages are written before root-attribute pages during ordinary eviction—without requiring explicit `H5Fflush` calls at each step. More efficient for high-frequency update workloads, but requires use of a library-internal API not exposed in the public HDF5 headers.

5. Specify the nonce scheme for encrypted datasets (§5.7). The document must choose among the three options now described in §5.7 and characterize their throughput impact on representative HPC checkpoint workloads:
   - **Per-chunk WAL** (prototype default): one crash-durable journal record per write; correct but adds a disk sync per chunk.
   - **Epoch-based WAL batching**: counter increments buffered per epoch and flushed collectively; amortizes sync cost across the batch.
   - **Random 96-bit nonces** (recommended for high-concurrency writes): eliminates the WAL entirely; collision probability negligible up to 2^{32} writes per dataset; 12-byte per-chunk storage overhead.

   Specify which scheme is the default for collective MPI-IO workloads and which for interactive/SWMR workloads.
6. Specify filter-pipeline ordering requirements from §5.6 and the on-disk layout of the hybrid signed-root attribute.
7. Get an internal review from a senior team member before submitting.
8. *Done when:* Design document is reviewed and all feedback addressed.
9. *Artifacts:* `docs/c-hdf5-integration-spec.md` containing the full on-disk format and proposed C API from steps 1–5; reviewer comments and responses saved to `docs/c-hdf5-integration-review.md`.

P2.7 HDF5 community RFC (Weeks 11–13 of Phase 2).

1. Summarize the on-disk format and proposed API from P2.6 in a self-contained document without implementation details or proofs. Target length: 4–6 pages.
2. Post to the HDF5 forum and record all substantive feedback in a `rfc/feedback.md` file in the repository.
3. *Done when:* RFC is posted and the feedback file is committed.
4. *Artifacts:* The RFC document committed to `rfc/`; `rfc/feedback.md` with each piece of substantive feedback logged as a separate entry with date and commenter so the response history is auditable.

7.3 Phase 3 / Stretch

These items are cut-able without affecting the three primary paper contributions. Begin them only after Phase 2 is complete and on schedule. Items are in priority order.

1. **Append-only log Merkle mode (Move 5):** implement the Certificate-Transparency-style append-only variant: each new dataset version appends leaves and stores a consistency proof that version n is a true prefix of version m. Targets time-series and streaming science workloads.
2. **Tamarin symbolic model:** encode the verification protocol in Tamarin Prover and prove authenticity, freshness, and subset-soundness lemmas mechanically. This elevates the P2.3 game-based argument to a machine-checked proof suitable for a security-venue submission.
3. **Sigstore / Rekor transparency-log integration:** publish hybrid signed roots to a Rekor log and store RFC 3161 TSA tokens for Long-Term Validation.
4. **AES-256-CTR + HMAC-SHA-256 filters:** FIPS-compliant alternative to ChaCha20-Poly1305 for deployments subject to FIPS 140-3.
5. **Provenance-graph integration:** bind the Merkle root as a W3C PROV [78] `prov:Entity` identifier so that derivation chains across datasets reference content-stable, signed roots.

8 Deliverables and S2-D2 Contributions

Table 3: Mapping of deliverables to S2-D2 tasks. The three primary research contributions targeted at the resulting publication are starred.

Deliverable	S2-D2 Task	Phase	Scope
Merkle-tree provenance implementation in ClawHDF5 (hybrid storage, companion-integrity-hash)	Task 3	1	Core
Hash benchmark: SHA-256, BLAKE3, KangarooTwelve at HPC chunk sizes	Task 3	1	Core
⋆ Verifiable subset extraction protocol and implementation (Move 1)	Task 3	1	Core
⋆ Hybrid Ed25519 + ML-DSA-65 signed roots with PQ measurements at HDF5 scale (Move 2)	Task 2, 3	2	Core
ChaCha20-Poly1305 AEAD filter with version-counter nonce derivation and AEAD-tag binding into the Merkle leaf	Task 3	2	Core
Game-based security argument for EtM + version-counter + hybrid-signed-root (Move 4)	Task 3	2	Core
⋆ Adversarial red-team study on NOAA and EQSIM datasets (T1–T8 + subset-extraction attacks) (Move 3)	Task 3	2	Core
Cloud object-store evaluation (S3 range-GET proof retrieval, WAN measurements) (Move 6)	Task 3	2	Core
C HDF5 integration design document and RFC	Task 3	2	Core
Append-only log Merkle mode with consistency proofs (Move 5)	Task 3	3	Stretch
Tamarin symbolic model of verification protocol	Task 3	3	Stretch
Sigstore transparency-log publication with Rekor + TSA storage	Task 3	3	Stretch
AES-256-CTR + HMAC-SHA-256 filters (FIPS-required deployments)	Task 3	3	Stretch
PROV [78] provenance-graph integration	Task 3	3	Stretch

8.1 Contribution to S2-D2 Goals

- **SHINES alignment**: The Merkle-tree prototype implements concrete technical mechanisms within the safety/security/privacy taxonomy and threat categorization (FMT, LIB, EXT, TCD) established by the HDF5 SHINES project [72], grounding the vocabulary in working code. The post-quantum threat T5 (§4.1) extends the SHINES taxonomy with a long-archival risk category not yet covered by the current vocabulary.
- **Task 2 connection**: Task 2 has implemented RSA plugin signature verification in the C HDF5 library (PR #6198), introducing a `KeyStore` for trusted public keys, an append-style signature footer on plugin binaries, a revocation blocklist, and the `h5sign` command-line signing tool. The present proposal shares this trust infrastructure: dataset-root signing keys can be managed through the same KeyStore mechanism, and the post-quantum hybrid pattern introduced here (Ed25519 + ML-DSA-65) provides a forward-compatible upgrade path for plugin signing on long-archival timescales, without requiring Sigstore or any external transparency service.
- **Task 3 contribution**: The Merkle tree provides a concrete, benchmarked data authentication mechanism—the "digital signatures for file object metadata" deliverable—with hybrid storage, EtM pipeline ordering, version-counter nonce derivation, and crash consistency protocol. The verifiable subset extraction protocol (§5.4) extends Task 3 beyond whole-dataset authentication to cryptographically verifiable partial data delivery.
- **Publication-shaped outputs**: The plan is structured around three contributions intended to be evaluated in the resulting paper—verifiable subset extraction (Move 1, RQ6), post-quantum hybrid signing at HDF5 scale (Move 2, RQ7), and the adversarial red-team soundness study (Move 3, RQ8)—each backed by an experimental section with novel measurements not currently in the literature.
- **Design transfer**: The Rust prototype's architecture serves as a reference implementation and test vector generator for the C HDF5 integration, even though the production C code will not use Rust directly. The design document bridges prototype to production.

9 Potential Venues

- **PDSW** (Parallel Data Systems Workshop at SC): Natural fit for the provenance and data integrity work in an HPC data context.
- **IPDPS / CCGrid**: Aligned with prior S2-D2 team publications.
- **HDF5 User Group (HUG)**: For the community RFC and design document presentation.

10 Limitations and Future Work

The proposed architecture is designed around HDF5's *chunked* storage layout, which provides the natural leaf granularity for the Merkle tree and is the dominant choice for large-scale HPC data. HDF5 also supports three non-chunked layouts—Contiguous, Compact, and Virtual Dataset (VDS)—each of which poses distinct research challenges that are out of scope for the present work but constitute natural and publishable extensions.

Logical paging for contiguous datasets. A contiguous dataset serializes its multi-dimensional array to a single sequential byte stream. A naïve Merkle tree over such a dataset has exactly one leaf, collapsing $O(\log N)$ verification to a full-file hash and making verifiable subset extraction impossible. A *virtual chunking* layer addresses this without altering the on-disk format: the byte stream is divided into fixed-size logical pages (e.g., 1 MB) that serve as Merkle leaves. The open research question is how to map an r-dimensional hyperslab proof onto the resulting 1D page grid. Because contiguous data is serialized in C-order (row-major), a rectangular 3D bounding box typically maps to many disconnected 1D byte ranges, amplifying the number of pages—and therefore proof witnesses— that must be fetched. Finding the page size that minimizes proof size times read-amplification cost across realistic HPC access patterns is a non-trivial optimization problem. Leaving this to future work is a deliberate scoping decision necessitated by the 7-month project timeline; it limits the immediate out-of-the-box utility of the prototype for legacy files whose datasets use contiguous storage, and this limitation is acknowledged explicitly in the paper's evaluation section.

Compact datasets and object-header compartmentalization. Compact datasets store raw data directly inside the HDF5 Object Header rather than the data heap (used for arrays typically <64 KB). Because the present proposal already incorporates a hash of the serialized Object Header into the file-level Merkle root (§5.5), compact datasets are *implicitly* protected: any mutation invalidates the Object Header hash and, transitively, the signed file-level root. The open problem is *cascading re-signing cost*: a single-value update to a compact dataset changes the Object Header and triggers a full PQ hybrid re-signature of the entire file. Future work should investigate Object Header compartmentalization strategies that isolate compact-data mutations from structural metadata, amortizing re-signing cost to $O(\log D)$ rather than $O(1)$ full-file re-signatures per update.

VFD-level block Merkle trees. HDF5 abstracts all file I/O through pluggable Virtual File Drivers (VFDs). An alternative architecture builds the Merkle tree at the VFD layer, treating the entire HDF5 file as a sequence of uniform 4 KB or 1 MB blocks— analogous to ZFS or IPFS. This secures all file content (metadata

B-trees, local heaps, contiguous data, and chunked data) without requiring HDF5-semantic awareness. The fundamental limitation is loss of *semantic provenance*: a VFD-level proof answers "blocks 400–512 are authentic" but cannot answer "this temperature hyperslab is authentic." Future research must define a translation layer that maps semantic HDF5 object queries to VFD block ranges and constructs the corresponding Merkle proof, recovering verifiable-subset semantics at the block level.

Authenticated Virtual Datasets. HDF5 Virtual Datasets (VDS) are views that map hyperslabs from multiple independent source files into a single logical dataset; they contain no data of their own. Authenticating a VDS requires a *Proof of Delegation* primitive: the VDS creator signs a manifest binding each virtual hyperslab to the Merkle root of its originating source file. When a consumer queries the VDS, the system fetches the appropriate subset proof from each source file and verifies (i) the source root against the signed manifest and (ii) the chunk witnesses against the source root. This transitive verification chain extends the integrity guarantees of the present proposal across file boundaries, and is directly relevant to federated scientific archives (e.g., multi-site climate ensembles) where data originates from many independently signed repositories.

11 Risks and Mitigations

ClawHDF5 maturity. ClawHDF5 has grown into an actively developed 15-crate workspace (∼46 commits as of mid-2026), but its primary focus has shifted toward an AI agent memory engine that uses HDF5 as a storage backend. To insulate this work from that churn, we depend solely on the `clawhdf5-format` crate—its `no_std`-compatible binary parser/writer—rather than the high-level API. This is the standalone-library strategy already described above, and it avoids re-exposure to ClawHDF5's agent-memory development cycle. To confirm the crate is fit for purpose, we retain the **go/no-go decision point at the end of Month 1**: `clawhdf5-format` must successfully open, iterate over all chunks of, and write back a 10 GB multi-dataset HDF5 test file without errors. If this gate is not met, the fallback is to vendor and patch the crate directly within this project's workspace.

Companion dataset portability. A reader unaware of the Merkle convention will see an extra dataset and ignore it, and a naïve file copy that omits the `_merkle` group breaks verification. The root hash attribute mitigates this (graceful degradation to root-level checking), and the design document will specify copy-preservation conventions.

Scope concentration. The plan is deliberately concentrated around three publication-shaped contributions (Moves 1, 2, and 3, plus the supporting security argument and cloud evaluation). Items not on the critical path—AES-256-CTR + HMAC-SHA-256 filters, live Sigstore service integration,

append-only log mode, the Tamarin model, and PROV graph integration—are explicitly demoted to Phase 3 / stretch and can be cut without affecting the core paper. The Merkle-tree provenance (§5), verifiable subset extraction (§5.4), hybrid signing (§5.8), and the adversarial red-team study (§6.5) are the highest-priority deliverables.

Cryptographic agility risk for ML-DSA / SLH-DSA. FIPS 204 and 205 were finalized in 2024; production-quality Rust implementations (e.g., `ml-dsa`, `pqcrypto-mldsa`) are still maturing. We mitigate by treating the signature scheme as an interchangeable trait behind a stable interface, allowing the classical-only Ed25519 path to remain functional while the PQ measurements catch up; if a chosen library proves unusable, the hybrid measurements are conducted with a different audited implementation (e.g., `liboqs` bindings) without disturbing the hybrid construction.

Adversarial-study scope. A white-box red-team study can in principle expand without bound. We bound it to the threat model of §4.1 (T1–T8) plus the three subset-extraction attack classes of §6.5; novel attack categories discovered during the study are documented as findings rather than chased to closure within this work.

Research Plan to Paper Mapping

7-month research plan → 9-section paper (∼13 pages) ⋆ *= primary research contribution*

Key structural principles

Single research direction, three named contributions. The paper makes one argument: HDF5 lacks sub-dataset integrity verification → a survey of tamper-detection primitives (error codes, hashes, MACs, signatures, authenticated structures, ZKPs, PoR/PDP, TEEs, and scientific-format precedents) shows that a Merkle tree with a hybrid-signed root is the candidate that best satisfies the full set of HDF5-specific requirements, with R8 (no trusted setup) and R9 (post-quantum upgrade path) as the discriminating constraints → here is a design that handles the HDF5-specific problems → here is the evidence. Three publication-shaped contributions sit inside that argument: ⋆ verifiable subset extraction (RQ6), ⋆ post-quantum hybrid signing at HDF5 scale (RQ7), and ⋆ a directed adversarial red-team study (RQ8). No two-direction framing.

Results organized by research question. Eight RQs (RQ1–RQ8), each a §7 subsection with question → data → interpretation. RQ1–RQ5 cover the systems contribution; RQ6–RQ8 cover the three primary contributions. Deeper treatment per RQ than in a single-sweep results section.

Security analysis as a first-class section. The game-based verified-subset soundness argument and adversarial red-team study are collected in §5 (Security Analysis), separate from the design (§4). This keeps §4 readable and gives security-venue reviewers what they look for.

STS is future work, not a companion result. The Security Task Scheduler is mentioned in §8 (Discussion) as an explicit future direction. Cleaner than trying to contribute to both Task 3 and Task 4 in one paper.

Target venue. PDSW @ SC'26 (10 pp) is the primary target at current scope; submission deadline is approximately August 2026. CCGrid / IPDPS (12 pp) is the backup if scope expands: both conferences are held in *May*, so submission deadlines fall in the preceding fall/winter (IPDPS 2027: ∼October 2026; CCGrid 2027: ∼November–December 2026). USENIX Security / ACSAC if the Tamarin model is completed.

Writing timeline. Draft §1–§4 after G2 (Month 4). Draft §5–§7 in Months 5–6. Write §8–§9 and revise §1 in Month 7. Submittable draft by Month 7.

Section-by-section mapping

§1 Introduction (∼1 pp)

Tasks / sources	Proposal §1
Research questions	—
Threats addressed	—

Content: Single-gap framing: no sub-dataset integrity verification in HDF5. Cites Peisert 2021. States three contributions up front. ClawHDF5 rationale: modular crate architecture, Rust memory safety for crypto, existing filter pipeline.
Writing strategy: One clear thesis; no two-direction framing. Abstract to 150 words. Draft early; revise after results to match headline numbers.

§2 Background, Survey, & Selection (∼2.5 pp)

Tasks / sources	Proposal §2–§4
Research questions	—
Threats addressed	—

Content: HDF5 chunked storage, filter pipeline, ClawHDF5 architecture. Survey of tamper-detection primitives: error-detection codes, cryptographic hashes, MAC/AEAD, classical and post-quantum signatures, hash trees and authenticated data structures (ZFS, Git, IPFS, CT, Sigstore, Tamassia), accumulators (RSA, bilinear), polynomial/vector commitments (KZG, Verkle, IPA, STARK), ZKP and PoR/PDP, TEE/fs-verity, cloud-store integrity, blockchain, scientific-format integrity (NetCDF, Zarr, ASDF, Parquet), provenance (W3C PROV). Then: requirements R1–R12 derived from HDF5 access patterns and archival horizon; comparison table; selection of Merkle-tree-with-signed-root.
Writing strategy: The survey → requirements → selection chain is what justifies the rest of the paper. Sharpen differentiation — what does this do that ZFS/IPFS/Git/Zarr-v3 don't? Answer: chunk-aligned trees with HDF5 hyperslab-aware partial verification, verifiable subset extraction, and PQ-ready signed roots. Target 30+ references.

§3 Threat Model (∼0.75 pp)

Tasks / sources	Proposal §4.1
Research questions	—
Threats addressed	T1 (storage tampering), T2 (silent corruption), T3 (provenance forgery), T4 (rollback), T5 (harvest-now/forge-later)

Content: Five threat categories from SHINES taxonomy. T5 extends the taxonomy for post-quantum archival risk. Adversary capabilities defined. Mechanism-to-threat mapping table. Companion integrity hash closes the partial-verification attack vector.
Writing strategy: Keep tight. The table mapping mechanism → threat is the money figure. Do NOT merge into §4.

§4 Merkle-Tree Design (∼4 pp)

Tasks / sources	Plan: Phase 1 items P1.2–P1.5; Phase 2 items P2.1–P2.2
Research questions	RQ1, RQ2, RQ3, RQ4, RQ6, RQ7
Threats addressed	T1, T2, T3, T4, T5

Content (planned paper subsections): (4.1) Hybrid storage — root attribute + companion dataset, 256-chunk crossover. (4.2) Domain separation + EtM pipeline ordering. (4.3) ⋆ Verifiable subset extraction — `extract_subset`/`verify_subset` protocol, proof π components, soundness sketch, proof-size scaling. (4.4) Nonce derivation — KDF with per-chunk version counter. (4.5) ⋆ Post-quantum hybrid signing — Ed25519 + ML-DSA-65, one signature per file-level root, key-size and attribute-encoding implications. (4.6) AEAD integration for ChaCha20-Poly1305 + DEK/KEK/HKDF key management. (4.7) Crash consistency — write ordering, fail-closed for signed datasets.
Writing strategy: Core contribution; takes most space. Lead with design figure, then storage layout, then security properties. 3 figures, 2 tables. Verifiable subset and PQ hybrid are the novel subsections reviewers will read first.

§5 Security Analysis (∼1 pp)

Tasks / sources	Plan: Phase 2 items P2.3–P2.4; Phase 3 item P3.2 (stretch)
Research questions	RQ8
Threats addressed	T1–T8

Content: (5.1) Game-based INT-CTXT argument (Bellare–Namprempre framework): EtM + version-counter + hybrid signed root reduces to second-preimage resistance, EUF-CMA of the hybrid scheme, and IND-CPA of the AEAD. (5.2) ⋆ Adversarial red-team study: white-box attack suite covering T1–T8 plus three subset-extraction attack classes; detection matrix; root-cause analysis of any successful attacks. Stretch: Tamarin depth-bounded symbolic model with machine-checked lemmas.

Writing strategy: Without the game-based argument this section risks rejection at security venues. Even for systems venues (CCGrid/PDSW), the argument upgrades the paper from "engineering report" to "security systems paper."

§6 Experimental Setup (∼1 pp)

Tasks / sources	Plan: Phase 1 item P1.6; Phase 2 item P2.5
Research questions	—
Threats addressed	—

Content: Datasets: NOAA, EQSIM, synthetic (parameterized by chunk size, count, dimensionality, compression ratio). Platforms: x86-64 (SHA-NI, AVX-512) and ARM (NEON, SHA extensions). Cloud: S3-compatible store for WAN measurements. Statistical protocol: 30 trials, bootstrapped 95% CIs, CPU isolation (`taskset`, `performance` governor). Metrics: hash throughput, partial-verification latency, subset-proof size/latency, PQ signing cost, WAN bandwidth, incremental update cost, storage overhead, adversarial-evasion rate.
Writing strategy: Factual and reproducible. Publish all benchmark scripts and tamper-injection harness as companion artifact.

§7 Results (∼3.5 pp)

Tasks / sources	Plan: Phase 1 items P1.2–P1.6; Phase 2 items P2.1, P2.4–P2.5
Research questions	RQ1–RQ8
Threats addressed	—

Content: §7.1 Storage overhead — Merkle vs. per-chunk-sig / per-chunk-MAC / flat baselines (P1.2b); hybrid vs. attribute-only crossover (RQ1). §7.2 Partial-verification latency by access pattern — Merkle vs. per-chunk-sig / per-chunk-MAC vs. full rehash (RQ2). §7.3 Incremental extension + in-place update cost — Merkle vs. per-chunk baselines vs. full rebuild (RQ3). §7.4 Hash throughput: SHA-256 / BLAKE3 / KangarooTwelve on x86 and ARM (RQ4). §7.5 Tamper detection TP/FP on NOAA + EQSIM (RQ5). §7.6 ⋆ Subset-proof size and latency vs. hyperslab shape; WAN bandwidth savings over S3 range-GET vs. full re-download (RQ6). §7.7 ⋆ PQ hybrid signing cost: key size, signature size, sign/verify time, object-header impact at 10^4–10^5 datasets per file; batch-verification throughput (RQ7). §7.8 ⋆ Adversarial red-team evasion matrix: attacks detected vs. undetected per threat class; localization granularity for detected attacks (RQ8).

Writing strategy: 8 subsections, each question → data → interpretation. RQ5, RQ6, RQ8 are the highlights. If tight on space, BLAKE3/KangarooTwelve comparison (RQ4) compresses to one paragraph. Expect 5–6 figures/tables.

§8 Discussion (∼1 pp)

Tasks / sources	Plan: Phase 2 items P2.6–P2.7 (design document + RFC)
Research questions	—
Threats addressed	—

Content: Design implications for C HDF5: filter pipeline ordering enforcement, companion dataset preservation semantics, fail-closed crash recovery integration with metadata journaling. Cloud deployment (HSDS, ros3 VFD). Limitations: single-platform scope, ClawHDF5-specific behaviors. Future work: C HDF5 integration, STS integration (explicitly future), append-only log mode, Tamarin model, multi-party key management / KMS.
Writing strategy: STS positioned as future work; discussion focuses on Merkle tree → C HDF5 production transfer. Design document (task 2.6) is the bridge artifact.

§9 Conclusion (∼0.5 pp)

Tasks / sources	—
Research questions	—
Threats addressed	—

Content: Three contributions: verifiable subset extraction, PQ hybrid signing at HDF5 scale, adversarial soundness study. Headline numbers: partial-verification speedup, subset-proof bandwidth savings, PQ signing overhead, adversarial-evasion rate. Design transfer via companion C HDF5 document. SHINES alignment.
Writing strategy: Short. One clear takeaway. Read §1 + §9 and get the full story.

Evidence flow: tasks → paper sections

Task	Description	§4 Design	§5 Sec	§7 Results	§8 Disc
P1.1	Platform eval go/no-go (ClawHDF5 gate)	✓			
P1.2	MerkleTree struct + hash eval (SHA-256, BLAKE3, K12)	✓		✓ (RQ4)	
P1.3	Hybrid storage layout + companion integrity hash	✓		✓ (RQ1)	
P1.4	Verification API (`verify_root/chunk/dataset`, `extend/update_merkle`)	✓		✓ (RQ1–3)	
P1.5⋆	Verifiable subset extraction protocol	✓		✓ (RQ6)	
P1.6	Phase 1 benchmarks (storage overhead, latency)			✓ (RQ1–4)	
P2.1⋆	Hybrid Ed25519 + ML-DSA-65 signing	✓		✓ (RQ7)	
P2.2	ChaCha20-Poly1305 AEAD + nonce derivation	✓			
P2.3	Game-based INT-CTXT security argument		✓		
P2.4⋆	Adversarial red-team study (T1–T8 + subset attacks)		✓	✓ (RQ8)	
P2.5	Cloud / WAN evaluation (S3 range-GETs)			✓ (RQ6)	
P2.6	C HDF5 design document				✓
P2.7	HDF5 community RFC				✓
P3.1*	Append-only log mode	✓			✓
P3.2*	Tamarin symbolic model		✓		
P3.3*	Sigstore + Rekor transparency log	✓			
P3.4*	AES-256-CTR + HMAC-SHA-256 filters	✓			
P3.5*	PROV graph integration	✓			

⋆ = primary research contribution. * = Phase 3 stretch goal.

Planned figures and tables

Figures

- Figure 1: Merkle tree over 8-chunk dataset with proof-path highlight (existing TikZ) — §4
- Figure 2: Hybrid storage layout — root attribute, companion dataset, integrity hash chain — §4
- Figure 3: Verifiable subset extraction proof π for a strided 2-D hyperslab (proof witnesses + coverage certificate) — §4
- Figure 4: Partial-verification latency vs. full rehash by access pattern (bar chart, RQ2) — §7
- Figure 5: Incremental update cost vs. full rebuild at varying N (line chart, RQ3) — §7
- Figure 6: Subset-proof size vs. hyperslab shape; WAN bandwidth savings vs.

full re-download (RQ6) — §7

Tables

- Table 1: Merkle tree storage overhead for representative dataset sizes (existing) — §4
- Table 2: Threat model — mechanism → threat mapping (T1–T8) — §3
- Table 3: Hash throughput SHA-256 / BLAKE3 / KangarooTwelve on x86 and ARM (RQ4) — §7
- Table 4: PQ hybrid signing cost — key size, signature size, sign/verify time, object-header impact (RQ7) — §7
- Table 5: Tamper detection TP/FP + adversarial evasion matrix on NOAA + EQSIM (RQ5, RQ8) — §7

Writing timeline

When	Sections	Action
May–June 2026 (Phase 1 stable)	§1–§4	Draft introduction, background, threat model, and Merkle design. Stable after P1.1 go/no-go gate. Circulate for internal feedback.
July 2026	§5, §6, §7	Security analysis; experimental setup; write results subsection-by-subsection as benchmark and red-team data arrive.
Early August 2026	§8, §9, §1 rev	Discussion + conclusion. Revise §1 with headline numbers. Final consistency pass. Internal review.
∼August 2026	**Submit**	**PDSW @ SC'26 submission deadline (primary target).** If rejected or scope expands: CCGrid / IPDPS 2027 deadlines fall Oct–Dec 2026 (conferences are in May).

Venue targeting

- **PDSW @ SC'26 (10 pp) — primary target:** SC'26 (November 2026, Chicago). Natural fit for HPC data provenance; aligns with S2-D2 team community. Achievable at current scope by compressing PQ signing and verifiable subset to one paragraph each and dropping the cloud / WAN evaluation if needed. *Paper submission deadline: approximately August 2026.*
- **CCGrid / IPDPS (12 pp) — backup if scope expands:** Full 8-RQ version with comfortable page budget. Both conferences are held in May; submission deadlines for the 2027 editions fall in the preceding fall/winter: IPDPS 2027 ∼October 2026, CCGrid 2027 ∼November–December 2026. These deadlines are compatible with the August 2026 PDSW submission: if PDSW is rejected, a revised version can be submitted to CCGrid/IPDPS on the same cycle.

- **USENIX Security / ACSAC:** Viable if the Tamarin symbolic model (Phase 3 stretch) is completed, giving machine-checked security lemmas. Requires the game-based argument in §5 as a minimum.
- **HDF5 User Group (HUG):** Community RFC and design document. Not peer-reviewed but critical for adoption and C HDF5 integration buy-in.

References

[1] Ryan Abernathey et al. Kerchunk: Cloud-friendly access to legacy data, 2022. GitHub repository; see also VirtualiZarr for the successor implementation.

[2] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato. Internet X.509 public key infrastructure time-stamp protocol (TSP). RFC 3161, 2001.

[3] Amazon Web Services. Checking object integrity in Amazon S3. https://docs.aws.amazon.com/AmazonS3/latest/userguide/checking-object-integrity.html, 2024.

[4] Apache Software Foundation. Apache Parquet: Columnar storage format. https://parquet.apache.org, 2024.

[5] Giuseppe Ateniese, Randal Burns, Reza Curtmola, Joseph Herring, Lea Kissner, Zachary Peterson, and Dawn Song. Provable data possession at untrusted stores. In *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*, pages 598–609, 2007.

[6] Mihir Bellare and Chanathip Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. In *Advances in Cryptology — ASIACRYPT 2000*, pages 531–545, 2000.

[7] Mihir Bellare and Chanathip Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm (journal). In *Journal of Cryptology*, volume 21, pages 469–491, 2008.

[8] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. In *Cryptology ePrint Archive, Report 2018/046*, 2018.

[9] Josh Benaloh and Michael de Mare. One-way accumulators: A decentralized alternative to digital signatures. In *Advances in Cryptology — EUROCRYPT 1993*, pages 274–285, 1993.

[10] Juan Benet. IPFS—content addressed, versioned, P2P file system. In *arXiv preprint arXiv:1407.3561*, 2014.

[11] Guido Bertoni, Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. KangarooTwelve: Fast hashing based on Keccak-p. https://keccak.team/kangarootwelve.html, 2018.

[12] Eric Biggers. fs-verity: Read-only file-based authenticity protection. Linux Kernel Documentation, 2019.

[13] Dan Boneh and David Mandell Freeman. Homomorphic signatures for polynomial functions. In *Advances in Cryptology — EUROCRYPT 2011*, pages 149–168, 2011.

[14] Jeff Bonwick, Matt Ahrens, Val Henson, Mark Maybee, and Mark Shellenbaum. The zettabyte file system. In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST)*, 2003.

[15] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy (S&P)*, pages 315–334, 2018.

[16] Victor Costan and Srinivas Devadas. Intel SGX explained. In *Cryptology ePrint Archive, Report 2016/086*, 2016.

[17] Rasmus Dahlberg, Tobias Pulls, and Roel Peeters. Efficient sparse Merkle trees: Caching strategies and secure (non-)membership proofs. In *Cryptology ePrint Archive, Report 2016/683*, 2016.

[18] Mike Folk, Albert Cheng, and Kim Yates. HDF5: A file format and I/O library for high performance computing applications. In *Proceedings of Supercomputing*, volume 99, pages 5–33, 1999.

[19] Google. dm-verity: Device-mapper block integrity checking. Linux Kernel Documentation, 2012.

[20] Google. Trillian: A transparent, highly scalable and cryptographically verifiable data store. https://github.com/google/trillian, 2024.

[21] Perry Greenfield, Michael Droettboom, and Erik Bray. ASDF: A new data format for astronomy. In *Astronomy and Computing*, volume 12, pages 240–251, 2015.

[22] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology — EUROCRYPT 2016*, pages 305–326, 2016.

[23] Herman Haverkort and Freek van Walderveen. Locality and bounding-box quality of two-dimensional space-filling curves. *Computational Geometry: Theory and Applications*, 43(2):131–147, 2010.

[24] Andreas Huelsing, Denis Butin, Stefan-Lukas Gazdag, Joost Rijneveld, and Aziz Mohaisen. XMSS: extended merkle signature scheme. RFC 8391, 2018.

[25] INCITS Technical Committee T10. SCSI protection information (T10-DIF): SPC-4 and SBC-3 standards. INCITS/T10 Standards, 2014.

[26] Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, 2017.

[27] Ari Juels and Burton S. Kaliski. PORs: Proofs of retrievability for large files. In *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*, pages 584–597, 2007.

[28] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology — ASIACRYPT 2010*, pages 177–194, 2010.

[29] John Kelsey, Shu jen Chang, and Ray Perlner. SHA-3 derived functions: cSHAKE, KMAC, TupleHash and ParallelHash. NIST Special Publication 800-185, National Institute of Standards and Technology, 2016.

[30] Philip Koopman. 32-bit cyclic redundancy codes for Internet applications. In *Proceedings of the International Conference on Dependable Systems and Networks (DSN)*, pages 459–468, 2002.

[31] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Advances in Cryptology — CRYPTO 2022*, pages 359–388, 2022.

[32] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. HMAC: Keyed-hashing for message authentication. RFC 2104, 1997.

[33] John Kuszmaul. Verkle trees. https://math.mit.edu/research/highschool/primes/materials/2018/Kuszmaul.pdf, 2018.

[34] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. In *RFC 6962*, 2013.

[35] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. In *RFC 6962*, 2013.

[36] Jay Lofstead, Ivo Jimenez, Carlos Maltzahn, Quincey Koziol, John Bent, and Eric Barton. DAOS: A scale-out high performance storage stack for storage class memory. In *Supercomputing Frontiers — SCFA 2020*, pages 40–60, 2020.

[37] Petros Maniatis, Mema Roussopoulos, T. J. Giuli, David S. H. Rosenthal, and Mary Baker. The LOCKSS peer-to-peer digital preservation system. *ACM Transactions on Computer Systems*, 23(1):2–50, 2005.

[38] David McCallen, N. Anders Petersson, Arthur Rodgers, et al. EQSIM—a multidisciplinary framework for fault-to-structure earthquake simulations on exascale computers. *Earthquake Spectra*, 37(2):707–735, 2021.

[39] David McGrew, Michael Curcio, and Scott Fluhrer. Leighton-micali hash-based signatures. RFC 8554, 2019.

[40] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN prover for the symbolic analysis of security protocols. In *25th International Conference on Computer Aided Verification (CAV)*, pages 696–701, 2013.

[41] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, pages 369–378, 1987.

[42] Meta Engineering. LtHash: Homomorphic hashing for efficient incremental integrity. https://engineering.fb.com/2019/03/01/security/homomorphic-hashing/, 2019.

[43] Simon Miles, Paul Groth, Steve Munroe, and Luc Moreau. PrIMe: A methodology for developing provenance-aware applications. *ACM Transactions on Software Engineering and Methodology*, 20(3):1–42, 2011.

[44] Luc Moreau, Ben Clifford, Juliana Freire, et al. The open provenance model core specification (v1.1). In *Future Generation Computer Systems*, volume 27, pages 743–756, 2011.

[45] G. M. Morton. A computer oriented geodetic data base and a new technique in file sequencing. Technical report, IBM Ltd., 1966.

[46] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. In *Whitepaper*, 2008.

[47] NASA SlideRule Project. h5coro: A cloud-optimized HDF5 reader for scientific data. https://github.com/SlideRuleEarth/h5coro, 2024.

[48] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2353–2367, 2022.

[49] Lan Nguyen. Accumulators from bilinear pairings and applications. In *Topics in Cryptology — CT-RSA 2005*, pages 275–292, 2005.

[50] Yoav Nir and Adam Langley. ChaCha20 and Poly1305 for IETF protocols. RFC 8439, 2018.

[51] NIST. NIST SP 800-38d: Recommendation for block cipher modes of operation: Galois/Counter mode (GCM) and GMAC. NIST Special Publication 800-38D, 2007.

[52] NIST. FIPS 180-4: Secure hash standard (SHS). Federal Information Processing Standards Publication 180-4, 2015.

[53] NIST. FIPS 202: SHA-3 standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication 202, 2015.

[54] NIST. FIPS 204: Module-lattice-based digital signature standard (ML-DSA). Federal Information Processing Standards Publication 204, 2024.

[55] NIST. FIPS 205: Stateless hash-based digital signature standard (SLH-DSA). Federal Information Processing Standards Publication 205, 2024.

[56] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. https://blake3.io, 2020.

[57] OpenLineage. OpenLineage: Open standard for data lineage collection. https://openlineage.io, 2024.

[58] Ekaterina Ovsyannikova, Rosa Filgueira, and Simon Sherwood. Towards trusted data pipelines for earth system science: Data integrity in NetCDF. In *2023 IEEE eScience Conference*, pages 1–10, 2023.

[59] Stavros Papadopoulos, Kushal Datta, Samuel Madden, and Timothy Mattson. TileDB: Managing massive dense and sparse multi-dimensional array data. *Proceedings of the VLDB Endowment*, 10(4):349–360, 2017.

[60] QuantumClaw. ClawHDF5: Pure-rust HDF5 library. https://github.com/quantumclaws/clawhdf5, 2026.

[61] Irving S. Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2):300–304, 1960.

[62] David S. H. Rosenthal, Thomas Robertson, Tom Lipkis, Vicky Reich, and Seth Morabito. Requirements for digital preservation systems: A bottom-up approach. *D-Lib Magazine*, 11(11), 2005.

[63] Hans Sagan. *Space-Filling Curves*. Springer-Verlag, 1994.

[64] Reiner Sailer, Xiaolan Zhang, Trent Jaeger, and Leendert van Doorn. Design and implementation of a TCG-based integrity measurement architecture. In *Proceedings of the 13th USENIX Security Symposium*, pages 223–238, 2004.

[65] Dafna Sheinwald, Julian Satran, Pat Thaler, and Vince Cavanna. Internet protocol small computer system interface (iSCSI) cyclic redundancy check (CRC)/checksum considerations. RFC 3385, 2002.

[66] Manu Sporny, Dave Longley, Markus Sabadello, Drummond Reed, Orie Steele, and Christopher Allen. Decentralized identifiers (DIDs) v1.0. W3C Recommendation, 2022. https://www.w3.org/TR/did-core/.

[67] Marc Stevens, Elie Bursztein, Pierre Karpman, Ange Albertini, and Yarik Markov. The first collision for full SHA-1. In *Advances in Cryptology — CRYPTO 2017*, pages 570–596, 2017.

[68] Roberto Tamassia. Authenticated data structures. In *Algorithms — ESA 2003*, pages 2–5, 2003.

[69] Houjun Tang, Quincey Koziol, Jyotsna Ravi, and Suren Byna. Transparent asynchronous parallel I/O using background threads. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):891–902, 2022.

[70] The HDF Group. HDF5 library and file format. https://www.hdfgroup.org/solutions/hdf5/.

[71] The HDF Group. HSDS: Highly scalable data service for HDF5 in the cloud. https://github.com/HDFGroup/hsds, 2024.

[72] The HDF Group. HDF5 SHINES: Securing HDF5 for industry, national security, engineering, and science. https://www.hdfgroup.org/projects/hdf5-shines/, 2026. NSF Safe-OSE Award No. 2534078.

[73] Peter Todd. Merkle mountain ranges. https://github.com/opentimestamps/opentimestamps-server/blob/master/doc/merkle-mountain-range.md, 2017.

[74] Linus Torvalds and Junio C. Hamano. Git: A content-addressable filesystem. https://git-scm.com/book/en/v2/Git-Internals-Git-Objects, 2005.

[75] Trusted Computing Group. TPM 2.0 library specification. Trusted Computing Group Standard, 2019.

[76] TODO: verify authors from DOI. Todo: verify title – see https://link.springer.com/chapter/10.1007/978-3-642-04846-3_9. In *Information Security Theory and Practice. Smart Devices, Pervasive Systems, and Ubiquitous Networks*, volume 5746 of *Lecture Notes in Computer Science*. Springer, 2009.

[77] TODO: verify authors from URL. Todo: verify title – see https://proceedings.neurips.cc/paper/2020/file/8ce6fc704072e351679ac97d4a985574-Paper.pdf. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[78] W3C. PROV-dm: The PROV data model. https://www.w3.org/TR/prov-dm/, 2013.

[79] wgpu: Safe and portable GPU abstraction in Rust. https://wgpu.rs, 2026.

[80] Mark D. Wilkinson et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016.

[81] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Ethereum Yellow Paper, 2014.

[82] Zarr Developers. Zarr: An implementation of chunked, compressed, N-dimensional arrays. https://zarr.dev, 2024.